uc3m | Universidad Carlos III de Madrid

Master Degree in Statistics for Data Science

2025-2026

Master Thesis

"Bayesian VAR vs. Temporal Convolutional Networks: A Simulation Study on Forecasting Accuracy and Uncertainty Calibration"

Zeno Fontaine

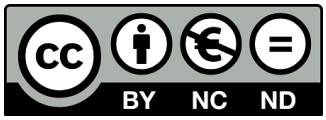
Juan Miguel Marín Diazaraque

María Helena Lopes Moreira Da Veiga

Madrid, 2026

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

This work aims to assess whether traditional Bayesian Vector Autoregression(BVAR) methods remain competitive, in terms of forecasting and uncertainty quantification, given recent advancements in deep learning algorithms for sequential data. To above question is explored via a simulation study that compares two statistical frameworks in their most common architectures: BVAR with Minnesota prior against Temporal Convolutional Networks (TCN) with Monte Carlo Dropout. The simulation study generates 315 datasets, each containing 6,250 observations across three variables, based on a stochastic model that aims to simulate minute-level trading data with three regimes that are modelled using a proposed *Excitement* parameter. The design combines the following elements: Stochastic Volatility via Ornstein-Uhlenbeck process, Jump-Diffusion, Cholesky-based Correlation Structure, and Regime-Dependent Correlations via *Excitement* parameter. The study utilised a two-stage design: the first stage found the best-performing architecture of each model family across 15 datasets, the second stage compared the two best-perforimng architectures across 300 datasets.

The results suggest that, despite being the older framework, BVAR dominated in accuracy (27-39% better RMSE, 38-53% better MAE), uncertainty calibration, and interval width (TCN intervals approximately 2.7 times wider). BVAR achieved near-nominal coverage whilst TCN remained severely overconservative due to MC Dropout being a Bayesian approximation method. Despite raw performance, the results are more nuanced. While BVAR remains competitive under suitable conditions and in cases where calibrated uncertainty is non-negotiable, TCN was approximately 9 times faster which makes it more suitable when computational speed and deployment over much larger datasets without strong structural assumptions takes priority.

Keywords: Bayesian VAR, Simulation Study, Minnesota Prior, Temporal Convolutional Networks, Monte Carlo Dropout, High-Frequency Forecasting, Forecasting Accuracy, Uncertainty Callibration

DEDICATION

I would like to express my gratitude to all those who have accompanied me on my journey that has culminated in the writing of this work. Namely, to Professor Charles Nolan, the Bonar MacFie Chair in Economics, who introduced me to VAR modelling and its use in financial econometrics and who supervised my first dissertation during my studies at the University of Glasgow; to my friends and family as I would have never been able to produce this work without their support; and last but not least, to Professors Juan Miguel Marín Diazaraque, María Helena Lopes Moreira Da Veiga and many other staff members of the UC3M's Department of Statistics, to whom I owe many thanks for all their assistance and their willingness to spend hours answering my questions.

CONTENTS

1. INTRODUCTION 1
2. LITERATURE REVIEW 3
2.1. Bayesian Vector Autoregression (BVAR) 3
2.1.1. Likelihood: Classical VAR 3
2.1.2. Prior: the foundational Minnesota Prior 4
2.1.3. Using MCMC Samplers with BVAR and Hamiltonian Monte Carlo 5
2.2. Neural Networks, TCN and Bayesian Approximation 7
2.2.1. Simple Neural Network Overview 7
2.2.2. History of Neural Networks used for Time-Series 10
2.2.3. Temporal Convolutional Networks (TCN) 11
2.2.4. Bayesian Approximation and Monte Carlo Dropout 14
2.2.5. Competing Architectures to TCN 15
3. METHODOLOGY 16
3.1. Simulated Data 16
3.1.1. Model Specification 17
3.1.2. Price Process 18
3.1.3. Statistical Properties 19
3.1.4. Parameter Calibration and Excitement Hyperparameter 19
3.2. Models and Simulation Study Procedure 23
3.2.1. Bayesian VAR with Minnesota Prior Implementation 24
3.2.2. TCN with Monte Carlo Dropout Implementation 26
3.2.3. Pretraining 27
3.2.4. Forecasting Test 28
3.2.5. Metrics, Comparison Method and Hardware Architecture 28
3.2.6. Aggregation Methodology Explained 29
3.3. Limitations 31

4. RESULTS AND DISCUSSION 32
4.1. Pretraining Results 32
4.1.1. BVAR 32
4.1.2. TCN+MC Dropout 33
4.2. Forecasting Test 34
4.2.1. Results for Excitement Level 0.25 34
4.2.2. Results for Excitement Level 0.50 35
4.2.3. Results for Excitement Level 0.75 35
4.2.4. Forecasting Results Discussion 36
4.3. Case Study: Comparison Between Model-Specific Inference Techniques . . . 39
4.3.1. Case Study Dataset: Excitement Level 0.50 - Pretraining Dataset 1 39
4.3.2. BVAR Inference Showcase 40
4.3.3. TCN+MC Dropout Inference Showcase 42
4.3.4. Strengths and Weaknesses Compared Across Models 47
5. CONCLUSION 49
BIBLIOGRAPHY. 51
A. TCN+MC DROPOUT ARCHITECTURE MAPPING FOR GRID SEARCH . .
B. FORECASTING TEST RESULTS . . . . . .
B.1. Results for Excitement Level 0.25 . . . . . .
B.2. Results for Excitement Level 0.50 . . . . . .
B.3. Forecasting Test Results for Excitement Level 0.75 . . . . . .
C. PRETRAINING DATASET VISUALISATIONS . . . . . .
D. PRETRAINING RESULTS . . . . . .
D.1. BVAR. . . . . .
D.2. TCN+MC Dropout . . . . . .
E. SIMULATION STUDY DESIGN TABLE . . . . . .
F. BVAR IMPLEMENTATION PIPELINE AND CODING SPECIFICATIONS . .
G. TCN+MC DROPOUT IMPLEMENTATION AND CODING SPECIFICATIONS
H. METRICS EQUATIONS AND HARDWARE CONFIGURATION TABLE . . .
H.1. Root Mean Square Error(RMSE) . . . . . .

H.2. Mean Absolute Error(MAE) . . . . . .
H.3. Prediction Interval Coverage Rate (80%, 90%, 95%) . . . . . .
H.4. Average Prediction Interval Width (80%, 90%, 95%) . . . . . .
H.5. Hardware Configuration for Model Training . . . . . .
H.6. Deviance Information Criterion (DIC). . . . . .
H.7. Mean Across Datasets . . . . . .
H.8. Standard Deviation Across Datasets . . . . . .
H.9. Aggregated (Overall) Mean. . . . . .
H.10. Aggregated (Overall) Standard Deviation . . . . . .
I. DERIVATION OF BAYES THEOREM . . . . . .
J. PRIOR DISTRIBUTIONS FOR COVARIANCE MATRICES . . . . . .
J.1. Inverse-Wishart Distribution . . . . . .
J.2. LKJ Distribution . . . . . .
K. MCMC ALGORITHMS SHOWCASE . . . . . .
L. INFERENCE SHOWCASE: DIAGNOSTIC PLOTS . . . . . .
M. CODE REPOSITORY AND AUTHOR ALIAS DECLARATION . . . . . .

LIST OF FIGURES

2.1 Four-layer Neural Network Architecture 8

2.2 12

4.1 BVAR(Lag 5) - IRF Plots 44

4.2 BVAR(Lag 5) - FEVD Plots 44

4.3 TCN+MC Dropout (Model 16) - Uncertainty Predictions 44

4.4 TCN + MC Dropout (Model 16) - Shock Propagation Plots 48

4.5 TCN + MC Dropout (Model 16) - Uncertainty Regime Detection 48

C.1 Pretraining Dataset: Excitement 0.25 Dataset 1 . . . . . .

C.2 Pretraining Dataset: Excitement 0.25 Dataset 2 . . . . . .

C.3 Pretraining Dataset: Excitement 0.25 Dataset 3 . . . . . .

C.4 Pretraining Dataset: Excitement 0.25 Dataset 4 . . . . . .

C.5 Pretraining Dataset: Excitement 0.25 Dataset 5 . . . . . .

C.6 Pretraining Dataset: Excitement 0.50 Dataset 1 . . . . . .

C.7 Pretraining Dataset: Excitement 0.50 Dataset 2 . . . . . .

C.8 Pretraining Dataset: Excitement 0.50 Dataset 3 . . . . . .

C.9 Pretraining Dataset: Excitement 0.50 Dataset 4 . . . . . .

C.10 Pretraining Dataset: Excitement 0.50 Dataset 5 . . . . . .

C.11 Pretraining Dataset: Excitement 0.75 Dataset 1 . . . . . .

C.12 Pretraining Dataset: Excitement 0.75 Dataset 2 . . . . . .

C.13 Pretraining Dataset: Excitement 0.75 Dataset 3 . . . . . .

C.14 Pretraining Dataset: Excitement 0.75 Dataset 4 . . . . . .

C.15 Pretraining Dataset: Excitement 0.75 Dataset 5 . . . . . .

F.1 BVAR Implementation Pipeline and Coding Specifications . . . . . .

G.1 TCN Implementation Pipeline and Coding Specifications . . . . . .

L.1 BVAR(Lag 5) - Diagnostic Plots . . . . . .

L.2 TCN+MC Dropout (Model 16) - Diagnostic Plots . . . . . .

LIST OF TABLES

3.1 Simulation Parameters by Excitement Level 21
3.2 Descriptive Statistics of Simulated Prices by Excitement Level 24
3.3 Minnesota Prior Hyperparameters 24
3.4 MCMC Sampling Configuration 25
3.5 Parameter Grid for Pretraining 28

4.1 Approximate 95% Prediction Interval Widths Across Excitement Levels . 36
4.2 Comparison of Runtimes Across Models 36
4.3 Per Variable Statistics % Change from BVAR to TCN 37
4.4 Overall Statistics % Change from BVAR to TCN 37

A.1 TCN Configuration Architecture Mapping . . . . . .

B.1 Overall Performance Metrics (Excitement = 0.25) . . . . . .
B.2 Prediction Interval Coverage Rates (Excitement = 0.25) . . . . . .
B.3 Prediction Interval Widths (Excitement = 0.25) . . . . . .
B.4 Root Mean Squared Error by Variable (Excitement = 0.25) . . . . . .
B.5 Mean Absolute Error by Variable (Excitement = 0.25) . . . . . .
B.6 Overall Performance Metrics (Excitement = 0.50) . . . . . .
B.7 Prediction Interval Coverage Rates (Excitement = 0.50) . . . . . .
B.8 Prediction Interval Widths (Excitement = 0.50) . . . . . .
B.9 Root Mean Squared Error by Variable (Excitement = 0.50) . . . . . .
B.10 Mean Absolute Error by Variable (Excitement = 0.50) . . . . . .
B.11 Overall Performance Metrics (Excitement = 0.75) . . . . . .
B.12 Prediction Interval Coverage Rates (Excitement = 0.75) . . . . . .
B.13 Prediction Interval Widths (Excitement = 0.75) . . . . . .
B.14 Root Mean Squared Error by Variable (Excitement = 0.75) . . . . . .
B.15 Mean Absolute Error by Variable (Excitement = 0.75) . . . . . .

D.1 BVAR Pretraining Results: Model Selection (Overall Metrics) . . . . . .

D.2 BVAR Pretraining Results: Uncertainty Quantification (Overall Metrics) .
D.3 BVAR Pretraining Results: Per-Variable Performance (Means) . . . . . .
D.4 TCN Pretraining Results: Model Selection (Overall Metrics) . . . . . .
D.5 TCN Pretraining Results: Uncertainty Quantification (Overall Metrics) .
D.6 TCN Pretraining Results: Per-Variable Performance (Means) . . . . . .

E.1 Simulation Study Design Summary . . . . . .

H.1 Hardware Configuration for Model Training . . . . . .

LIST OF ABBREVIATIONS

- **ADF** – Augmented Dickey-Fuller (test)
- **APIW** – Average Prediction Interval Width
- **BNN** – Bayesian Neural Networks
- **BVAR** – Bayesian Vector Autoregression
- **CNN** – Convolutional Neural Networks
- **DCC** – Dynamic Conditional Correlation
- **DIC** – Deviance Information Criterion
- **FEVD** – Forecast Error Variance Decomposition
- **GRU** – Gated Recurrent Unit
- **HMC** – Hamiltonian Monte Carlo
- **IQR** – Interquartile Range
- **IRF** – Impulse Response Function
- **KPSS** – Kwiatkowski-Phillips-Schmidt-Shin (test)
- **LKJ** – Lewandowski-Kurowicka-Joe (distribution)
- **LSTM** – Long Short-Term Memory Networks
- **MAE** – Mean Absolute Error
- **MC Dropout** – Monte Carlo Dropout
- **MCMC** – Markov Chain Monte Carlo
- **MLP** – Multilayer Perceptron
- **NUTS** – No-U-Turn Sampler
- **PFN** – Prior-data Fitted Networks
- **PICR** – Prediction Interval Coverage Rate
- **RMSE** – Root Mean Squared Error
- **RNN** – Recurrent Neural Networks
- **SHAP** – SHapley Additive exPlanations
- **TCN** – Temporal Convolutional Network
- **VAR** – Vector Autoregression

1. INTRODUCTION

The field of Bayesian Deep Learning represents the bleeding-edge of contemporary Bayesian statistics, but perhaps now more than ever, the field remains plagued with one major issue: computational efficiency [1]. Bayesian Neural Networks (BNN) are conceptually simple: using prior distributions over weights rather than point estimates. However, this simple addition exponentially expands even modest networks into computationally intractable models [2]. These challenges become more pronounced in time-series applications, where traditional dimensionality and temporal dependencies are exacerbated by neural network parametrisation [3]. So far, approaches to this challenge fall into two categories: **(1)** approximation methods that make Bayesian inference tractable, including Monte Carlo(MC) Dropout [4], Deep Ensembles [5], and variational inference [6]; and **(2)** alternative frameworks that bypass traditional inference, such as Prior-data Fitted Networks(PFN) [7]. In the case of time-series analysis, neural networks can now use non-sequential architectures for temporal data, making them a feasible alternative to traditional time-series models. the recent innovations that skip sequential processing that was previously required for temporal data, now allows for neural network models to be a feasible alternative to traditional methods. One such alternative are Temporal Convolutional Networks (TCN) [8] which can also be used with MC Dropout to create Bayesian approximation model for time-series analysis.

The classical approach for multivariate time-series analysis is the use of Vector Autoregression (VAR) models that have been a staple of multivariate time-series analysis since 1980 [9]. One of the more common approaches is the use of Bayesian VAR (BVAR), which addresses the shortcomings of point-estimation in VAR models by introducing uncertainty quantification via contemporary Bayesian framework.

The standard informed prior for BVAR is the Minnesota prior [10] which addresses the overparametrisation problem, that are common in standard VAR models, by introducing a simple framework. Nevertheless, the Minnesota prior's simplicity might be counterproductive and may perform worse compared to data-driven priors as its design may limit its ability to capture complex dependencies. However, data-driven priors can also be sensitive to their assumptions on top of the correlation structures within the historical data that may be irrelevant to forecasting, which is why Minnesota remains a standard choice within BVAR framework.

Despite Bayesian time-series analysis experiencing a renaissance thanks to the formulation of more advanced models, such as Time-Varying Parameter Factor Augmented VAR [11], its dominance is now being challenged by time-series neural networks models. Recent works suggest that neural networks can enhance forecasting abilities of Bayesian models [3], [7]. The main motivation behind this paper is to answer the following questions: Can traditional BVAR frameworks still remain competitive with modern deep learn-

ing techniques in terms of forecasting? What would be the impact on uncertainty quantification if BNN approximation methods are used? This paper aims to answer the above questions by carrying out a simulation study on 315 simulated datasets whose design is based on high-frequency trading data. The models compared are BVAR with Minnesota prior against TCN with MC Dropout. These models and their parameters were chosen to reflect two relatively simple and commonly used implementations. While more complex BVAR frameworks exist, so do more advanced TCN with Bayesian approximation models. The focus is on commonly used versions, not to compare the most advanced models available. Additionally, while not the main aim, the work contains a small showcase case study that briefly compares model-specific inference techniques to illustrate key differences between the models.

In brief, the simulation-study is set up as follows:

1. Generate 315 dataset (6250 observation each) where 15 datasets are generated under a different seed than the other 300 datasets
2. *Pretraining* Section: using the 15 datasets for model selection to select the best performing model within each category: TCN+MC Dropout(27 models) and BVAR (7 models for lags 3-9). Due to the dataset size, scale of simulation study, and runtime a simple 80%/20% train-test split was used
3. *Forecasting Test* Section: the best performing models from BVAR and TCN are trained and generate forecast on the remaining 300 datasets. The comparison metrics for each datasets are aggregated to identify the best performing model across 300 datasets.

In the first chapter, "Literature Review" presents a brief summary of history and development of both frameworks. The next chapter then presents methodology that goes in detail how: simulation was implemented, how the models were built, and detailed description of the simulation process. Following the "Methodology" chapter, the chapter "Results and Discussion" contains results of both *Pretraining* and *Forecasting Test* sections as well as a brief case study showcase of inference methods.

Information About Code Repository

The code repository is publicly available at `https://gitlab.com/zfontaine/uc3m-master-thesis-bvar-vs-tcn` under the alias Zeno Fontaine. For more information, please consult *Appendix M*.

2. LITERATURE REVIEW

Despite the great leaps in Bayesian statistics, at its core remains the same Bayes equation that was published posthumously in 1763[12]. Major advancements in Bayesian statistics occurred during the 20th century. The first major step was the formulation of conjugate priors by Raiffa and Schlaifer [13] which was later expanded by DeGroot [14] who generalised the theory and systemically extended the understanding for both exponential family and beyond into discrete distributions and multiparameter cases. Their combined efforts meant that for the first time, Bayesian statistics could be done analytically. Further developments were in the field of prior design and include: the invention of objective priors [15]; their use in regression [16], [17]; and independent development of reference priors by Berger and Bernardo [18] who formalised the distinction between objective and non-informative priors (objective are derived from formal rules while non-informative aim to have minimal effect on posterior thus hoping to "let data speak for itself"). Throughout the years, various solutions for prior design have been proposed including weakly informative priors [19], empirical Bayes [20], [21], and modern shrinkage methods [22], [23]. Despite of the above, the major issue of going beyond conjugate prior was computation. It was the invention of Metropolis-Hastings [24] and later Gibbs Sampling [25] algorithms that made Bayesian statistics viable in practical applications. Since Bayes Theorem is at a heart of Bayesian modelling, for completeness, the derivation based on Kolmogorov axioms [26] and summary of basic terms is included in *Appendix I*.

2.1. Bayesian Vector Autoregression (BVAR)

2.1.1. Likelihood: Classical VAR

Vector autoregression (VAR) originates from the original work on autoregressive modelling by Yule [27] that was later formalised by Walker[28] and aided by establishing of the notion of stationarity by Wold [29]. Their work was later synthesised into a practical framework by Box and Jenkins [30]. However, the key idea of vectorising the AR came from Sims [9] who enhanced econometric modelling of their era by treating all variables as endogenous within a reduced-form system.

In essence, it is a relatively simple equation of using Ordinary Least Squares (OLS) regression. This is illustrated using a simple VAR(1) with $k = 3$ variables:

$$\begin{bmatrix} y_{1,t} \\ y_{2,t} \\ y_{3,t} \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} + \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} y_{1,t-1} \\ y_{2,t-1} \\ y_{3,t-1} \end{bmatrix} + \begin{bmatrix} \varepsilon_{1,t} \\ \varepsilon_{2,t} \\ \varepsilon_{3,t} \end{bmatrix} \tag{2.1}$$

where: $y_{i,t}$ is the observation of variable i at time t; c_i is the intercept for variable i;

a_{ij} captures the effect of variable j at time $t-1$ on variable i at time t; and $\varepsilon_{i,t}$ is the error term for variable i, with $\boldsymbol{\varepsilon}_t \sim N(\mathbf{0}, \boldsymbol{\Sigma})$.

This model can then be rewritten via matrix notation into a general VAR(p) model with p lags:

$$\mathbf{y}_t = \mathbf{c} + \sum_{i=1}^{p} \mathbf{A}_i \mathbf{y}_{t-i} + \boldsymbol{\varepsilon}_t, \quad \boldsymbol{\varepsilon}_t \sim N(\mathbf{0}, \boldsymbol{\Sigma}) \tag{2.2}$$

where: $\mathbf{y}_t$ is a $k \times 1$ vector of observations at time t; $\mathbf{c}$ is a $k \times 1$ vector of intercepts; $\mathbf{A}_i$ is a $k \times k$ matrix of coefficients for lag i; and $\boldsymbol{\Sigma}$ is the $k \times k$ error covariance matrix.

However, VAR becomes problematic as the number of variables and lags increases. The total number of parameters in $\mathbf{A}_1, \ldots, \mathbf{A}_p$ is $k^2 p$. As the number of parameters increases, there is a rapid decrease in the available degrees of freedom. In other words, there is a risk of overparametrisation that leads to imprecise estimation. Furthermore, VAR and BVAR are built on fundamental assumptions: **1.** the dataset must be stationary (constant mean, variance and autocovariance over time); **2.** the relationships between variables are linear (no regime switches or threshold effects); **3.** errors are normally distributed with zero mean, with no autocorrelation, are homoskedastic, but may be correlated across variables; and **4.** standard modelling assumptions including weak exogeneity, sufficient sample size, and no perfect multicollinearity.

2.1.2. Prior: the foundational Minnesota Prior

Later developments from objective and non-informative priors led to realisation that such priors do not work well in specialised fields. One such solution was the introduction of systemic priors, one of which was the foundational Minnesota prior: an informative prior that is used within BVAR framework[10]. Minnesota prior was developed at Federal Reserve Bank of Minneapolis by Doan, Litterman and Sims[31]. Minnesota prior was designed to address high-dimensionality and overfitting issues associated with objective priors in higher dimensions and is based on the assumptions: **Random walk assumption:** each variable approximately follows a random walk meaning the coefficient for its own first lag is near 1; **Decay structure:** there is a decay towards zero as lag length increases, meaning the most recent lags are considered more important; and **Lag importance:** variable's own lag is more important that other variables' lags.

$$\mathrm{Var}(\beta_{ii,\ell}) = \left(\frac{\lambda}{\ell^{\delta}}\right)^2 \tag{2.3}$$

$$\mathrm{Var}(\beta_{ij,\ell}) = \left(\frac{\lambda}{\ell^{\delta}}\right)^2 \cdot \frac{\sigma_i^2}{\sigma_j^2} \cdot \frac{1}{\tau} \tag{2.4}$$

Minnesota prior has a relatively simple setup. The prior is first placed on coefficient

vector β with prior mean equal to 1 for its own first lag, but 0 for other variables. Assuming variable i is the starting variable, then equation 2.3 represents $i's$ prior variance formula for its own lags. For other variable j, the equation 2.4 represents the prior variance formula for cross-variable (i to j) lags. The parameters utilised by equation 2.3 and 2.4 are: λ which specifies overall tightness; τ which represents cross-variable shrinkage; δ which is lag decay rate; and σ_i^2/σ_j^2 is the ratio of residual variances for unit scaling. These terms allow for clear mechanics where: low λ creates stronger shrinkage which prevents overfitting; high τ results in cross-variable shrinkage to be higher than own-variable shrinkage; and finally δ then shrinks the amount of effective lags as distant lags start to matter less. This effectively reduces degrees of freedom without dropping variables which is exactly why it addresses the curse of dimensionality that was common with objective priors and classic VAR models. However, when working with multivariate modelling, the covariate structure needs its own prior formulation as well. Minnesota was originally designed to combine with Inverse-Wishart distribution which forms a prior for covariance matrix. This setup leads to conjugacy via multivariate normal likelihood as it implies the posterior belongs to the same distributional family. Thus, the full conjugate formulation is as follows:

$$\underbrace{p(\mathbf{B}, \mathbf{\Sigma} \mid \mathbf{Y})}_{\text{Posterior}} \propto \underbrace{\mathcal{N}(\mathbf{B} \mid \mathbf{B}_0, \mathbf{\Sigma} \otimes \mathbf{\Omega}_0)}_{\text{Minnesota Prior}} \times \underbrace{\mathscr{W}^{-1}(\mathbf{\Sigma} \mid \mathbf{S}_0, \nu_0)}_{\text{Inverse-Wishart Prior}} \times \underbrace{\prod_{t=1}^{T} \mathcal{N}(\mathbf{y}_t \mid \mathbf{B}\mathbf{x}_t, \mathbf{\Sigma})}_{\text{VAR Likelihood}} \tag{2.5}$$

where: $\mathbf{B}$ is the $k \times (1 + kp)$ coefficient matrix; $\mathbf{\Sigma}$ is the $k \times k$ error covariance matrix; $\mathbf{B}_0$ is the prior mean for coefficients (encoding random walk beliefs); $\mathbf{\Omega}_0$ encodes the Minnesota prior variance structure from Equations 2.3 and 2.4; $\mathbf{S}_0, \nu_0$ are the Inverse-Wishart scale matrix and degrees of freedom; and $\otimes$ denotes the Kronecker product.

2.1.3. Using MCMC Samplers with BVAR and Hamiltonian Monte Carlo

While using conjugate prior may seem appealing, in practice Inverse-Wishart often results in limited utility in edge cases; being overly informative in higher dimensions; and overly simplified assumptions about correlation/variance structures. Alternative is to use non-conjugate priors, but if a non-conjugate prior is used, then the model needs to include a sampling method. Very common alternative to Inverse-Wishart is Lewandowski-Kurowicka-Joe (LKJ) distribution [32]. The LKJ prior has key advantages over Inverse-Wishart by providing more control over correlation shrinkage and allowing separate prior on correlation and variances. LKJ prior is most often used within Bayesian Hierarchical modelling and is often included as a default setting in Stan and PyMC. A brief showcase of LKJ and Inverse-Wishart distributions can be seen in *Appendix J*.

Markov Chain Monte Carlo (MCMC) algorithms are the backbone of modern Bayesian statistics. For many years, it was impossible to compute posterior distributions, not even

computing marginal likelihoods was a feasible task. Furthermore, the "Evidence" often end up being a high-dimensional integral and as such, it was computationally intractable prior modern MCMC implementations.

The first step toward contemporary MCMC and other Bayesian sampling methods was the foundational paper by Metropolis at al. [24] from 1953. The key innovation was the acceptance-rejection mechanism that the paper proposed. In essence, Metropolis algorithm creates a Markov chain where proposed values are either accepted or rejected based on distribution ratios. The key here is that rejection means the current state is still kept. As such, through repeated sampling, the Markov chain converges to the target distribution based on the acceptance/rejection criteria. In 1970, Hastings generalised the algorithm in such a way that allowed proposals from asymmetric distributions [33]. This made the algorithm more accessible in other fields as it made the algorithm more flexible by allowing the use of any distribution. The next step was the Gibbs Algorithm which can be thought of as a special case of Metropolis-Hastings algorithm. Under Gibbs Algorithm, if the conditional distributions are known, then they can be sampled from, regardless if joint distribution is known or not. It was first proposed by Geman and Geman in 1984 [25], but it was not until six year later when Gelfand and Smith explicitly demonstrated its use in Bayesian statistics and showed that marginal densities can be approximated via sampling from conditional distributions [34]. This creates clear computational advantages and simplifications to the entire sampling process as: firstly, there is no acceptance/rejection process like in previous algorithms which simplifies computing; and secondly, the sampling process from conditional distributions greatly reduces computational cost as, instead of working with the product of likelihood and priors, the conditional densities allow for working through each variable consecutively. However, this requires that at least conditional distributions are known, which is not always the case.

The above samplers often perform poorly in high-dimensional spaces. A contemporary solution is Hamiltonian Monte Carlo (HMC) which simulates Hamiltonian dynamics via auxiliary momentum variables that lead to appropriate proposals from high-dimensional space [35]. In another words, HMC uses gradients of the target distribution to guide the sampling process. However, the original HMC requires manual selection of two hyperparameters: step size and number of leap frog steps. Since inappropriate choices lead to bad sampling, No-U-Turn Sampler(NUTS) was designed to automate parameter selection[36]. NUTS is an adaptive extension of HMC through a recursive tree-building algorithm that automatically determines path length by identifying when the path gets stuck. This is why NUTS is currently a popular choice within Bayesian programming tools for a sampler. MCMC algorithm showcase in *Appendix K*, but not containing HMC, may be used to observe the differences across MCMC more clearly.

Extensions and Modern Developments Outside of technological advancements, there has been significant development since the introduction of the BVAR framework. Kadiyala and Karlsson developed efficient computational methods [37] and Sims and Zha further

extended their work to include structural identification within BVAR [38]. In 21st century Koop and Korobilis formulated a comprehensive treatment for macroeconomic applications [11] while Giannone et al. proposed data-driven hyperparameters selection[39]. The above is just a tip of the iceberg as BVAR continues to be one of the most active research fields in econometrics to this day. Furthermore, there has been a trend of proposals that skip the MCMC architecture entirely through various alternative methods, such as bootstrap likelihood approaches [40] or integrated nested Laplace approximations [41].

2.2. Neural Networks, TCN and Bayesian Approximation

With the recent advancement of deep learning algorithms and neural networks that are marketed as "Artificial Intelligence", the machine learning research field has become one of the most popular while at the same time being the least understood by non-practitioners. Neural networks, despite their recent popularity, have their origins in the 20th century. They were first modelled by McCulloch and Pitts in 1943 [42] in the field of neurophysiology. The authors proposed axioms for their new calculus whose traces can be found within the models today: In 1958, Rosenblatt built on these axioms when building the first learning algorithm: The Perceptron [43]. The Perceptron could automatically adjust its weights based on identifiable errors in order to not repeat them. It worked by finding a linear decision boundary that separated data by a straight line or hyperplane. The innovation was not perfect as, if a solution did not exist, the algorithm would get stuck in an infinite loop. This was fully explored in 1969 by Minsky and Papert [44] who explained and proved that single-layer perceptrons have computational limitations. Through geometry, they showed that the Perceptron cannot linearly separate non-linear problems. Their work, perhaps unintentionally, effectively killed the field of neural networks as a viable area of machine learning research for over a decade. It was not until backpropagation was introduced that neural networks returned as a viable algorithm. However, there are other two major factors that contributed to neural network's popularity in 2020's: **(1.)** computational power and **(2.)** better and larger datasets. Combining these two means that neural networks can now be trained on terabytes of data while having a GPU-based architecture to do so. GPU allows for parallel linear algebra computations that are extremely fast. Combination of both factors meant that previously infeasible deep neural networks became a reality in the 21st century. This naturally led to technical advancements. Modern neural networks are capable of learning their own features and representations rather than relying on engineers' hand-crafted specifications. Architecture improved to a point where there are families of neural networks specialising on specific tasks.

2.2.1. Simple Neural Network Overview

Backpropagation was introduced in 1986 by a collaboration between Rumelhart, Hinton and Williams [45] and was crucial to all networks that came afterwards. The precise mech-

anism and its importance is best illustrated on a simple model of a neural network that is described below.

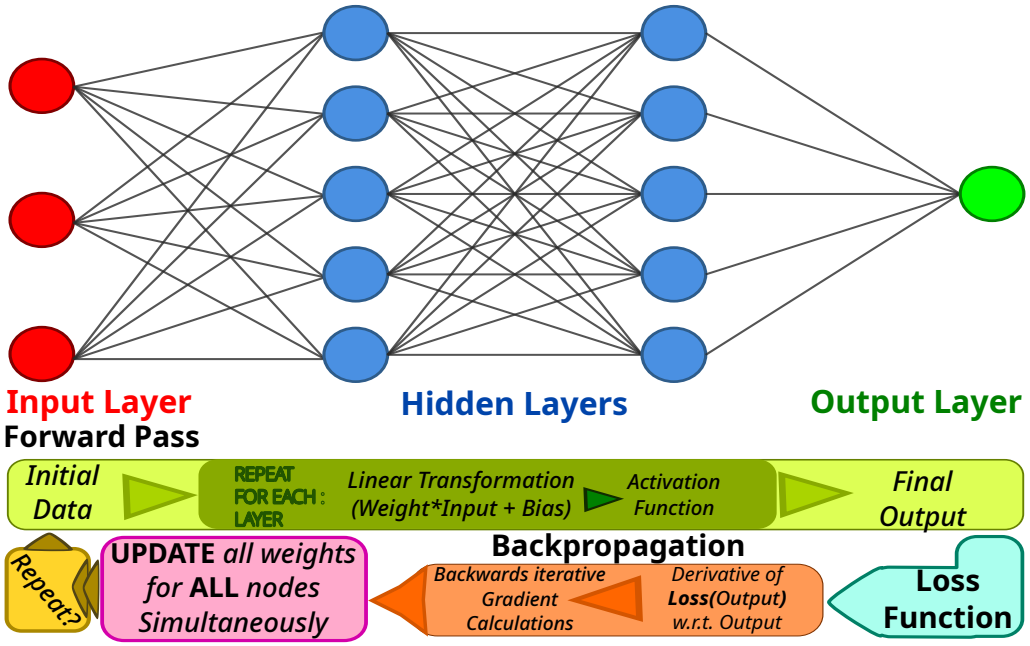


Fig. 2.1. Four-layer Neural Network Architecture

The diagram 2.1 is a simple example of a four-layer network with 3 input nodes and one output node. For simplicity, it is easier to assume the output node contains an integer that is being point-estimated from 3 input nodes. There are two layers in-between input and output layers which are referred to as hidden layers. As seen in the diagram, there are five nodes for each hidden layer. Each node can be thought of as storage for intermediate representations of the input. Connections between nodes carry weights, with the first assignment of weights being randomised. The are two weights used within the system: *weight matrix* that multiplies values passed between layers, and *bias* terms that are additive constants applied within each layer. What then follows is a multiple training loop runs that converge to a specific output.

Using the illustrative example, a single training loop does the following: **1.** forward pass in hidden layer 1, using data from input layer, applying linear transformation and activation function to compute node values; **2.** forward pass in hidden layer 2, using output from hidden layer 1, applying linear transformation and activation function; **3.** forward pass in output layer (often with task-specific activation function) using hidden layer 2 values to generate final prediction; **4.** calculate error via loss function comparing prediction to expected output; **5.** backpropagate through layers from output towards input, computing gradients via the chain rule; **6.** update all weights simultaneously using gradients and learning rate; and **7.** use convergence mechanism to decide if training should continue. The forward and backward passes repeat, adjusting weights after each epoch, until convergence. The convergence mechanism could be as simple as a fixed number of epochs.

As explained above, forward pass consists of linear transformation that is then fol-

lowed by activation function that leads to new node values through a sequential process across layers. As the pass moves from layer to layer, it takes data from previous nodes and then applies linear transformation to all nodes via:

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$$

where $\mathbf{W}$ is the weight matrix, $\mathbf{x}$ is the input vector from the previous layer, and $\mathbf{b}$ is the bias vector. This is then passed into the activation function to produce the new node values within the layer:

$$\mathbf{a} = f(\mathbf{z})$$

where f is the activation function. The output layer very often has its own activation function that generates desired outputs. In general, activation functions should be non-linear and differentiable for neural networks to work properly. The output is then evaluated via a loss function, which compares predicted versus actual values, most commonly via MSE (regression) or cross-entropy (classification). The derivative of the loss function with respect to the output is the starting point of backpropagation:

$$\frac{\partial \mathscr{L}}{\partial \mathbf{a}^{(out)}}$$

Compared to the developments within activation functions research (e.g. sigmoid [44] for binary classification, softmax for multiclass[46], ReLU [47]), the idea behind back-propagation remains essentially the same: backpropagation calculates gradients by applying the chain rule iteratively backwards through layers:

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}^{(3)}} \rightarrow \frac{\partial \mathscr{L}}{\partial \mathbf{W}^{(2)}} \rightarrow \frac{\partial \mathscr{L}}{\partial \mathbf{W}^{(1)}}$$

Once backpropagation is finished, iteratively going backward from output layer to input layer, all weights and biases are updated simultaneously:

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \cdot \frac{\partial \mathscr{L}}{\partial \mathbf{W}}$$

$$\mathbf{b}_{new} = \mathbf{b}_{old} - \eta \cdot \frac{\partial \mathscr{L}}{\partial \mathbf{b}}$$

where η is the learning rate.

While most academic literature includes the updating as a part of the backpropagation process, it is valuable to think of it as a separate step which prevents the false notion that weights are updated iteratively. Backpropagation thus elegantly addresses the issue of credit assignment and layer design simultaneously. Networks without backpropagation had to be single-layer, but backpropagation's automatic weight adjustment allowed

multi-layer networks to be feasible, which in turn allowed networks to learn non-linearities through the combination of multiple layers with non-linear activation functions. This simple idea is what gave rise to deep networks and the developments towards large language models. Thanks to the modular design of individual components, the basic neural network model is now largely illustrative, but its foundations are present, albeit in spirit only, across nearly all neural network models. However, despite all developments since backpropagation's conception, the risk of non-convergence and the algorithm getting stuck in local minima still exists. These risks have been greatly reduced thanks to advancements in gradient optimisation tools and much more complex architectures. Another drawback that remains is that inference about what led to the final prediction,and why the network assigned the weights the way it did, is much harder to obtain.

2.2.2. History of Neural Networks used for Time-Series

Neural networks encountered the same problem all statisticians encountered when working with time-series data: temporal dependence. In other words, the previously described model of neural network cannot be applied for time-series as it assumed independence between samples and fixed-size of inputs. Backpropagation was originally proposed to solve this problem with sequential data via Recurrent Neural Networks(RNN) [45] and later developed by Elman [48] into a viable form. The challenge with sequential data was to figure out a way for networks to have memory. RNN solution was the introduction of single hidden state which shared parameters across time, being functionally a single hidden layer network that is applied sequentially. While simple, the design revealed critical limitation. The repeated backpropagation across identical weights means that the the gradient is multiplied by weights repeatedly which led to gradients either vanishing, when weights are below 1, or exploding for weights larger than 1. This means that long-term memory is impossible to achieve with RNN as it cannot learn dependencies beyond approximately 10 iterations [49].

To address the issue of vanishing gradients, Hochreiter and Schmidhuber proposed a Long Short-Term Memory networks (LSTM) in 1997 [50]. They proposed a cell state that would be separate from hidden state and would function across the entire sequence. It made it possible for information to be stored, updated, forgotten and output. This process is controlled by three gates(sigmoid functions with values between 0 and 1): **Forget gate:** controls what information to discard from previous cell state; **Input gate:** controls what new information to store in a cell state; and **Output gate:** controls what parts of cell state to expose as hidden state. While this approach largely solved the gradient vanishing problem as gradients could now propagate across much larger time spans, in practice however, there is significant computational cost due to the impossibility of using parallel computing as each time-step has to be analysed separately. Furthermore, the architecture still struggled with long sequences beyond 500 timesteps as there are limits to what the cell state can learn from large data, making it unsuitable for high-frequency time-series analysis.

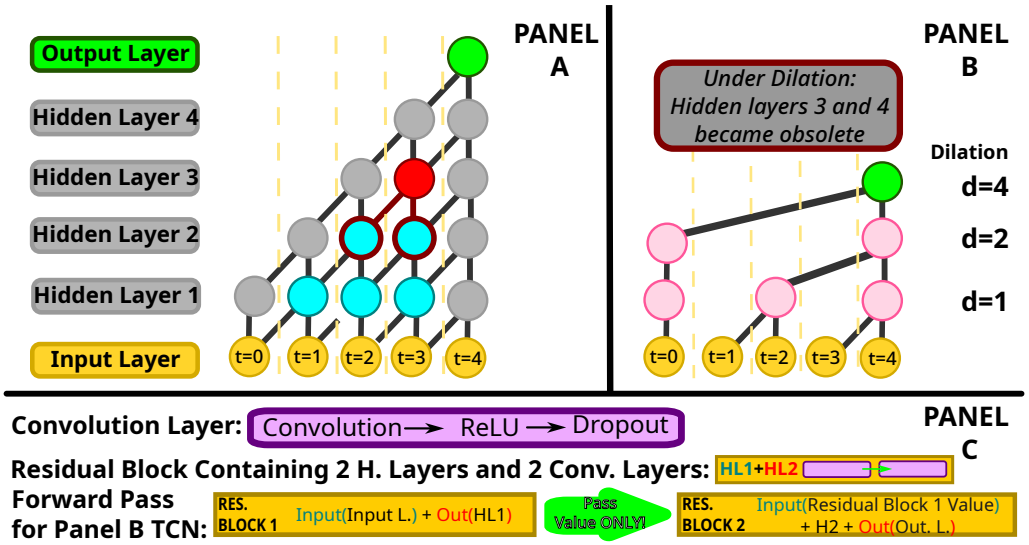
One proposed solution to solve the computational issue of LSTM was Gated Recurrent Unit (GRU) that greatly simplified the architecture [51]. GRU merged the cell state and hidden state into one and only uses two gates: **reset gate**, which dictates how much information to "forget"; and **update gate**, which decides how much information to keep or add. This simplified architecture allows for 25% less parameters than LSTM and has comparable performance to LSTM. In essence, GRU model prefers computational efficiency for at times lesser performance of LSTM. The choice between GRU vs. LSTM comes to specific application requirements where GRU is generally preferred for situations when computational resources are limited.

Despite the two decades worth of progress, the fundamental problem for using neural networks for time-series analysis remained: sequential processing. The fact that all timesteps had to be processed in a sequence meant that there was no possibility for vectorisation and GPU acceleration for linear algebra or use parallel computing for different timesteps. Therefore, the longer the sequential data, the longer the training time.

2.2.3. Temporal Convolutional Networks (TCN)

In 2018, a solution to the sequential processing problem was proposed by Bai, Kolter and Koltun who demonstrated that Convolutional Neural Network (CNN) architecture [52] can not only be used for any sequential dataset, but that it systematically outperforms RNN paradigm [8]. CNN are often used on images that can be considered as two dimensional space. CNN then uses a kernel, which is a function applied to a subspace (can be thought of as localised forward pass on an area of pixels) to identify key features of a said space. After CNN is done, it outputs a feature map that is build from individual kernel results and effectively shrinks the entire image space into its kernel mapping. With each extra layer of kernels, CNN is able to extract more complex features. Bai et al. adapted CNN architecture for 1-dimensional sequential data, thus formalising the modern TCN architecture. TCN consists of four key elements: causal convolutions, dilations, residual blocks and dropout:

Causal Convolutions The obvious issue when using kernels in one dimensional sequential data is that kernels would by design include future nodes which would violate the nature of temporal data. Without adjustment, kernel of size 3 at time t would encompass the kernel space $[node_{t-1}, node_t, node_{t+1}]$. This violates the assumption of linear time. To solve this issue, the algorithm creates a causal rule where a kernel cannot access future nodes. In effect, the new kernel space would look like $[node_{t-2}, node_{t-1}, node_t]$. Looking at figure 2.2 of a naive TCN in *Panel A*, a causal convolution network can be build resembling the classic pyramid-like structure of TCN. From the diagram, it can be seen that every node is associated with specific time which is shown by showing to which column does the node belong to. From the perspective of a red node and kernel of size 3 being applied to it, the kernel space includes the two red-circled cyan nodes in hidden layer



Note: **Panel A:** Naive TCN with kernel size 3(shown by red connections) where from red node's perspective, input nodes $t = 0, 1, 2, 3$ and cyan nodes are its receptive field; **Panel B:** Naive TCN with dilation; **Panel C:** graphical representation of the forward passes hierarchical architecture.

Fig. 2.2

2 as well as the red node. Combining all the cyan nodes with input layer nodes where $t = 0, 1, 2, 3$ creates what is known as a receptive field. In other words, the receptive field contains all past nodes that have direct influence on a current node. This naive TCN is naive because it does not include the additional architecture that makes this a viable network. In fact, the network grows linearly with depth to a point where, for 100 timesteps, approximately 5000+ nodes and 50 layers are needed just to generate a one-point forecast with kernel size 3.

Dilation This is solved via dilated convolutions. In simple terms, due to the dependent relationships between nodes, the network does not need to cover every connection to remain meaningful. For each layer l after the first hidden layer, dilation d is calculated as $d = 2^{l-1}$, where the first hidden layer has dilation one. This means that the first hidden layer connects to all adjacent input nodes. In Figure 2.2 *Panel B*, it is shown that for hidden layer 2, $d = 2$. For a node at time t with dilation d and kernel size k, the dilated convolution accesses positions $\{t, t-d, t-2d, \ldots, t-(k-1)d\}$. Thus, for example the pink node at $t = 4$ with $d = 2$ and $k = 3$, the dilated layer includes nodes from layer 1 at times $t = \{0, 2, 4\}$. While dilations are effective for capturing long-range dependencies without excessive computational cost, they do not eliminate the risk of overfitting.

Regularisation via Dropout Despite dilation, the network will remain overly complex. This might seem counter intuitive, but overly complex networks do not in fact lead to better inference. Instead, it leads to overfitting, which implies the network learns dataset-specific patterns that do not generalise well to other datasets, even from the same source. This is

why regularisation techniques are important for neural networks. One such technique is dropout [53], which is an elegantly simple technique. Dropout is active during training, where it randomly assigns 0s to values stored inside nodes by sampling from Bernoulli distribution, where the probability that the original value is kept is p. Effectively, this is identical as if during training, the neural network was using random sampling from an exponential number of different thinned networks. Then during testing, the impact of all those weightings is averaged into a single unthinned network that as a consequence, has much smaller weights than without using dropout.

Residual Blocks, Convolution Layer, Forward Pass and Backpropagation To simplify computational cost, prevent network degradation, and eliminate the risk of vanishing gradients, a hierarchical structure is implemented and is illustrated in Figure 2.2 Panel C. The smallest structure is a convolution layer. If there were no hierarchical structure, a single convolution layer could work as a forward pass. Convolution layers within a residual block are connected sequentially: the output of the first layer is fed directly into the input of the second layer as part of a single forward pass. Residual blocks are then subnetworks that consist of a selection of layers for which the convolution layers are trained. The residual block then passes its values to the next block that uses the new inputs to train the next subnetwork. For this to work effectively, ReLU [47] is used as activation function. ReLU is defined as $\text{ReLU}(x) = \max(0, x)$. For positive inputs, the derivative is always 1, thus mitigating the vanishing gradient problem, while negative inputs produce zero output, creating sparse activations that increase computational efficiency. Combining the above with the residual connection function that connects two blocks, $\text{output} = \text{Activation}(x + F(x))$, ReLU not only allows for simple residual block architecture, but the connection also allows gradients to flow directly through the identity shortcut (where gradients are 1) without vanishing in deep networks, thus enabling effective backpropagation. Relating this back to the simple neural network model: the entire forward pass is separated into residual blocks that contain convolution layers. The layers contain the TCN transformation $F(x)$ which comes from the convolution architecture and ReLU activation function. The loss function and backpropagation work via the same concept, but ReLU allows backpropagation to work effectively on deep networks.

The above text illustrates the basic conceptual blocks behind TCN, but it should be noted that the architecture is very modular and allows for many additions. There has been a significant number of publications, with the most recent paper on "ModernTCN"[54] showing that, through much more sophisticated convolution and block design, not only does TCN remain competitive, under certain circumstances is able to outperform all neural network competition. However, TCN needs to be extended even further to allow for uncertainty quantifications.

2.2.4. Bayesian Approximation and Monte Carlo Dropout

One obvious extension to neural networks is the idea of including uncertainty quantification via Bayesian methods. BNN was first proposed by MacKay [55] and later underwent a comprehensive treatment by Neal in their doctorate thesis [56]. In essence, they introduced the idea that, instead of using fully deterministic forward pass and backpropagation system, there could be a stochastic element where the weights are instead represented by posterior distribution. This idea, while innovative, faces one key issue: computational tractability. Neural networks are powerful algorithms because they allow for parallel matrix operations, but modern Bayesian systems almost always require sequential sampling via MCMC algorithms. This makes MCMC largely impractical within neural networks and unless there is a parallel matrix computation alternative, BNN will remain largely intractable in their pure form. Instead, the focus is currently on Bayesian approximation methods that would allow for some uncertainty quantifications. Such examples include variational inference[2], deep ensembles [57] and MC Dropout[4], which is a Bayesian approximation method chosen for this study.

The key difference from regular dropout is that MC dropout continues to conceptually sample from thinned distributions during testing. Normally, during testing, networks run a forward pass only a single time to predict values. However, when using MC Dropout, the network is run T times with dropout enabled, thus changing the deterministic process to stochastic. As each forward pass is based on different thinned architecture, it creates different predictions. Sample mean and variance of T predictions can be calculated to approximate predictive distribution, which is derived from variational approximating distribution, such as Bernoulli, which is fixed. This is the key difference: the model does not approximate distributions the way MCMC algorithms do to gain true posterior. MC dropout instead approximates the posterior over weights via variational inference of a fixed distribution. The nature of this difference is why it is a Bayesian approximation. The method might be controversial as its uncertainty estimates are not based on approximation of true posterior distribution achieved via Bayes Theorem. Furthermore, since it is sampling from a fixed variational distribution, there is a diminishing return after 100 samples as it is resampling from a fixed variational distribution. Gal and Ghahramani [4] indeed acknowledge this and admit that this approach produces non-calibrated uncertainty estimates that can be either overconfident or overconservative, depending on a choice of variational distribution[58]. This is because different approximating distributions used in dropout will lead to different uncertainty estimates and thus a choice of variational distribution remains a subjective choice. Despite all of these clear weaknesses, MC Dropout remains a relevant and useful Bayesian approximation tool as, while non-calibrated, it is computationally cheap, easy to implement, and the source of its stochastic nature is more transparent than when using other approximation methods, such as Ensembles that lack explicit Bayesian justification.

2.2.5. Competing Architectures to TCN

The field of neural networks for time-series analysis is highly dynamic with many competing architectures and no clear sign of dominance. Unlike with VAR-based models, there is not a consensus on the main model architectures. All architectures within the field propose their own unique way to solve the sequential processing issue that make LSTM and GRU impractical in larger and more complex datasets. There are other two major approaches that compete directly with TCN: transformer-based and LLM-based models. LLM-based models are relatively recent and currently include models such as TimeGPT [59] and Chronos [60]. The "established" competition to TCN are transformer-based models that are briefly described below.

Transformers-based Models Vaswani et al.[61] proposed solution to the sequential processing problem that let every timestep directly engage with every other position simultaneously. The authors coined the term "Self-Attention". Briefly, the algorithm computes for each timestep how much it should "attend" to all other timesteps via three vectors(Query, Key and Value); then through vector-based architecture, calculate attention scores that measure relevance between positions and normalise it via softmax for creating a probability distribution over the sequence; apply the scores to values stored for each timestep. To introduce natural ordering, sine/cosine functions are used to create position signals for each input which creates positional encoding. The entire process allows to run the network with multiple parallel attention mechanisms where each "Attention Head" was able to learn different relationship from the data. Transformer-based models thus allow sequential data models to: utilise parallel processing; learn long-range dependencies; eliminate the problem of vanishing gradients; and learn complex and flexible patterns across entire datasets. This of course came with new problems such as: quadratic complexity $O(n^2)$; fixed sequence length due to position encoding limitations; no inherent temporal dependence where the models had to learn temporal relationships from scratch. For time-series specifically, Zhou et al. [62] proposed a model named *Informer*. This model is able to simplify the computational cost from $O(n^2)$ to $O(nlogn)$ by identifying the most "important" pairs. Essentially, the model can be thought of as an informed transformer which allowed transformer-based architecture to be employed on very large datasets.

However, while TCN, transformer and LLM-based architectures are most visible within the field, there are many new architectures, that have the potential to cause a paradigm shift within the next two years. Such models include: Mamba[63], which represents long sequential data via continuous dynamical system; and TimeMixer [64], which is based on Multilayer Perceptron (MLP) that decomposes time-series into different scales, and then applies mixing algorithms across them. It should therefore be noted that, unlike in other fields, time-series neural networks do not yet offer architecture that dominates the entire field and thus choice of architecture is in practice decided on case-by-case basis.

3. METHODOLOGY

This chapter focuses on discussing the methodology behind simulation procedure and individual model implementations. It is important to note the overall design philosophy behind this simulation study. The key here is to compare two relatively simple and commonly used architectures and their performance. This is why BVAR with Minnesota prior is being compared to TCN with MC Dropout with standard default setting. While it is true more complex TCN architecture could be utilised, the same could be said about BVAR implementation within this study. The focus is to compare the commonly used variations of both models, not their best-performing, complex versions.

3.1. Simulated Data

In order to properly compare BVAR and TCN, a simulation study has to be carried out. However, to ensure a realistic comparison, a complex simulation design is necessary as simpler linear-model based simulations would not introduce the non-linearity that is common in trading data. As such, the comparison needs to be based on a realistic model that generates high-frequency time series.

Prior experience with trading data was heavily utilised in designing the following simulation model. In order to meet computational requirements, earlier attempts of creating realistic minute trading data for a period of 3 to 12 weeks was abandoned in favour of a more stylistic design. The underlying issue was scaling: trading week contains approximately 1950 minutes thus creating a realistic dataset where each day contains 390 minutes with simulated day-to-day dynamics quickly becomes computationally intensive. Instead, the idea of modelling high-frequency time series based on minute data for prices was kept, but now stylised to 6250 observations that create reasonable 5000-1250 split. Thus, the simulation model creates a single data set that contains 6250 observations for 3 variables where each variable starts at indexed value of 100. This model can then be run multiple times, while using seed for reproducibility, to produce a range of randomised datasets. The underlying model is based on combination of two components: Stochastic Volatility proposed by Taylor [65] and Jump-Diffusion proposed by Merton[66].

To achieve this, a system of equations for asset returns is used as a backbone for the model and then the prices are extracted from the log-returns. The following section explains the theoretical model. The contribution of this paper in terms of the model creation on top of the two aforementioned components are the following:

1. Integration and implementation of existing frameworks for Stochastic Volatility and Jump-Diffusion

2. Creating “Cholesky-based Correlation Structure” where correlation matrix is implemented via Cholesky decomposition with regime-dependent correlation
3. Scaling the model to achieve minute-level parametrisation
4. Introducing the “Excitement Parameter” to simulate different market conditions
5. Creating numerical stability mechanism via reasonable bounds
6. Implementing simple Return-to-Price conversions

3.1.1. Model Specification

The return process for asset i at time t is given by:

$$r_{i,t} = \sigma_{i,t}\epsilon_{i,t} + J_{i,t} \tag{3.1}$$

where: $r_{i,t}$ represents the log-return; $\sigma_{i,t}$ is the stochastic volatility component; $\epsilon_{i,t}$ is the diffusion shock; and $J_{i,t}$ is the jump component.

Stochastic Volatility Process

The log-volatility follows an Ornstein-Uhlenbeck process:

$$h_{i,t+1} = h_{i,t}(1-\kappa) + \kappa\bar{h} + \eta_{i,t} \tag{3.2}$$

where: $h_{i,t} = \log(\sigma_{i,t}^2)$ is the log-volatility; $\kappa = 0.1(1 - 0.5\lambda)$ is the mean reversion speed; $\bar{h} = 0$ is the long-term mean of log-volatility; $\eta_{i,t} \sim \mathcal{N}(0, \sigma_v^2)$ with $\sigma_v = 0.05+0.1\lambda$; and $\lambda \in [0, 1]$ is the Excitement parameter.

The actual volatility is recovered as:

$$\sigma_{i,t} = \exp(\frac{h_{i,t}}{2}) \tag{3.3}$$

Cholesky-based Correlation Structure with Regime-Dependent Correlations

The diffusion shocks are correlated across variables via Cholesky decomposition:

$$\boldsymbol{\epsilon}_t = L\boldsymbol{z}_t \tag{3.4}$$

where $\boldsymbol{z}_t \sim \mathcal{N}(\mathbf{0}, I_3)$ is a vector of independent standard normals and L is the Cholesky decomposition of the correlation matrix Σ, whose off-diagonal elements are defined as $\rho = 0.2 + 0.4\lambda$ and thus can be represented as follows:

$$\Sigma = \begin{bmatrix} 1 & \rho & \rho \\ \rho & 1 & \rho \\ \rho & \rho & 1 \end{bmatrix} \quad \text{where} \quad \rho = 0.2 + 0.4\lambda \tag{3.5}$$

The above method ensures that ϵ_t has a covariance structure $Cov(\epsilon_t) = \Sigma$ across 315 simulated datasets. Thus the complete system that combines stochastic volatilities with the correlation structure can be represented as follows:

$$\mathbf{\Sigma}_t^{1/2} = \text{diag}(\sigma_{1,t}, \sigma_{2,t}, \sigma_{3,t}) \cdot L \tag{3.6}$$

While there are more complex and arguably more realistic approaches to achieve multivariate correlation structures, such as dynamic conditional correlation(DCC)[67], the use of regime-dependent parameter (ρ includes λ) allows for a balance between realistic simulation and computational tractability that is suitable as a basis for a comparative study focusing on forecasting abilities of two distinct models.

Jump Component

The jump process follows a compound Poisson distribution with correlated jump sizes:

$$J_{i,t} = \begin{cases} \sqrt{\rho_J} Y_t^{(c)} + \sqrt{1-\rho_J} Y_{i,t}^{(id)} & \text{with probability } p_J \\ 0 & \text{with probability } 1 - p_J \end{cases} \tag{3.7}$$

where: $p_J = 0.0005 + 0.001\lambda$ is the jump probability; $\rho_J = 0.3 + 0.6\lambda$ is the jump correlation; $Y_t^{(c)} \sim \mathcal{N}(0, \sigma_J^2)$ is the common jump component; $Y_{i,t}^{(id)} \sim \mathcal{N}(0, \sigma_J^2)$ is the idiosyncratic jump component; and $\sigma_J = 0.002 + 0.003\lambda$ is the jump volatility.

Complete System Dynamics

The full multivariate system for $\mathbf{r}_t = [r_{1,t}, r_{2,t}, r_{3,t}]'$ can be written as:

$$\mathbf{r}_t = \mathbf{\Sigma}_t^{1/2} \boldsymbol{z}_t + \mathbf{J}_t \tag{3.8}$$

3.1.2. Price Process

Since the entire simulation process utilises log-returns, a simple mathematical transformation can be applied. By definition, the log-return at time t is:

$$r_t = \log(P_t) - \log(P_{t-1}) \tag{3.9}$$

After rearranging the formula for $\log(P_t)$, the equation is then transformed to represent the total change from time 0 to t. Due to log-returns being additive, all individual returns can be summed from $s = 1$ to t. This simplifies to the following equation that represents the total change from time 0 to t:

$$\log(P_t) = \log(P_0) + \sum_{s=1}^{t} r_s \tag{3.10}$$

the price at time t can be recovered by exponentiating both sides:

$$P_t = P_0 \cdot \exp\left(\sum_{s=1}^{t} r_s\right) \tag{3.11}$$

Finally, the price series is constructed from the cumulative returns:

$$P_{i,t} = P_{i,0} \exp\left(\sum_{s=1}^{t} r_{i,s}\right) \tag{3.12}$$

where initial price $P_{i,0} = 100$ for all assets. This Return-to-Price process is only carried out after the entire time-series for returns has been generated. Otherwise, there would be a risk of the simulation producing negative prices with non-cumulative effect, which would mean prices would not be simulated properly.

3.1.3. Statistical Properties

The proposed model generates returns with the following stylised facts:

- **Volatility clustering**: Periods of high/low volatility tend to persist due to the Ornstein-Uhlenbeck stochastic volatility process
- **Fat tails**: According to theory, excess kurtosis should be present from both stochastic volatility and jumps. Later section will show whether there is indeed kurtosis present or if the tails are just slightly larger than expected.
- **Correlation dynamics**: Correlations increase during market stress as "Excitement Hyperparameter" increases

3.1.4. Parameter Calibration and Excitement Hyperparameter

To simplify testing of different simulated market conditions, this paper proposes "Excitement Parameter" that manages the levels that are used to generate datasets representing different market regimes:

- $\lambda = 0.25$: Calm market conditions

- $\lambda = 0.50$: Normal market conditions
- $\lambda = 0.75$: Stressed market conditions

Originally, there was a plan to include a self-exciting dynamics, which would more accurately reflect the speed of the market fluctuations during crisis. In theory, for higher excitement levels, the model could be calibrated in such a way where a single jump temporarily increases probability for a future jump. This idea was abandoned as in practice this led to cascading jumps that generated datasets where variables either rapidly collapsed to zero or exploded to infinity. However, the "Excitement Hyperparameter" that was a part of the self-exciting dynamics works well on its own to simulate different market conditions. Especially its presence within the correlation structure is able to accurately represent different market conditions. When the market is calm, there is less correlation with less jumps, but if the market is stressed, not only are the jumps more pronounced, the assets are more likely to move together which reflects market reality for assets within the same asset-category.

Another challenge was calibrating the parameters. While literature for stochastic volatility [65] and Jump-Diffusion [66] contains standard specifications, they were not enough to realistically capture minute-scale data. Instead, intensive iterative experimentation was used to calibrate parameter values that are used within the model. Although the initial parameter rangers were based on estimation studies such as by *Ait-Sahalia et al.*[68] or *Chib et al.* [69], the priority for numerical stability and reproducibility across 315 simulated datasets with minute-data frequencies meant the parameters diverged significantly from the initial ranges. This was largely due to scaling.

For the simulation to achieve its stated goal of representing high-frequency minute-scale data, it required its parameter ranges to be scaled down. For that *minute_scale* parameter was introduced with the value of 0.001. This scaling produces approximately 0.1% return per observation which approximates minute-scale price movements. As stated previously, the trading day has approximately 390 minutes (for example: New York Stock Exchange ran from 9:30 AM to 4:00PM during 2025). Thus, the return-per-minute in percentage terms can be approximated using the following method:

Let daily return be r_{daily} and minute-level return be $r_{\text{minute},i}$, where the time period of a day is $T = 390$. Then:

$$r_{\text{daily}} = \sum_{i=1}^{390} r_{\text{minute},i} \tag{3.13}$$

Assuming independence across minute intervals, the variance of the daily return is:

$$\text{Var}(r_{\text{daily}}) = \sum_{i=1}^{390} \text{Var}(r_{\text{minute},i}) = 390 \cdot \sigma^2_{\text{minute}} \tag{3.14}$$

where σ^2_{minute} denotes the variance of minute-level returns. Taking standard deviations:

$$\sigma_{\text{daily}} = \sqrt{390 \cdot \sigma^2_{\text{minute}}} = \sqrt{390} \cdot \sigma_{\text{minute}} \tag{3.15}$$

Rearranging yields the minute-level volatility as a function of daily volatility:

$$\sigma_{\text{minute}} = \frac{\sigma_{\text{daily}}}{\sqrt{390}} \approx \frac{\sigma_{\text{daily}}}{19.75} \approx \frac{\sigma_{\text{daily}}}{20} \tag{3.16}$$

If its assumed that r_{daily} is between 1 and 2%, then it can be deduced that $r_{\text{minute},i}$ is approximately $0.05-0.1\%$ This scaling is then implemented via *minute_scale* parameter of 0.001, which when applied to the diffusion component produces returns of this magnitude:

$$r^{\text{diffusion}}_{i,t} = \text{minute_scale} \cdot \sigma_{i,t} \cdot \epsilon_{i,t} = 0.001 \cdot \sigma_{i,t} \cdot \epsilon_{i,t} \tag{3.17}$$

This ensures that simulated returns reflect realistic minute-level price movements while maintaining numerical stability.The above system of equations is build on the assumption of independent returns and is based on approaches that are common in econometrics and mathematical finance that can be found in "Analysis of Financial Time Series"[70].

However, *minute_scale* parameter alone does not achieve numerical stability. Thus reasonable limits were introduced where Log-Volatility bounds are set as [-2, 2] and Return bounds [-0.05, 0.05]. The former prevents volatility explosion where the variables would move too far apart. The latter then ensures that the returns are not defined by single observation and are thus set to 5% maximum per observation. This prevents large cumulative price errors from forming.

Parameter	**Symbol**	$\lambda = 0.25$	$\lambda = 0.50$	$\lambda = 0.75$	**Formula**
Jump Parameters					
Jump Probability	p_J	0.075%	0.10%	0.125%	$0.0005 + 0.001\lambda$
Jump Size (Volatility)	σ_J	0.275%	0.35%	0.425%	$0.002 + 0.003\lambda$
Jump Correlation	ρ_J	0.45	0.60	0.75	$0.3 + 0.6\lambda$
Stochastic Volatility Parameters					
Vol of Vol	σ_v	0.075	0.10	0.125	$0.05 + 0.1\lambda$
Mean Reversion Speed	κ	0.0875	0.075	0.0625	$0.1(1 - 0.5\lambda)$
Long-run Mean (log)	$\bar{h}$	0	0	0	–
Correlation Structure					
Cross-asset Correlation	ρ	0.30	0.40	0.50	$0.2 + 0.4\lambda$
Scale and Safety Parameters					
Base Return Scale	–	0.001	0.001	0.001	`minute_scale`
Log-vol Bounds	h_t	$[-2, 2]$	$[-2, 2]$	$[-2, 2]$	Clipped
Return Bounds	r_t	$[-0.05, 0.05]$	$[-0.05, 0.05]$	$[-0.05, 0.05]$	Clipped

TABLE 3.1. SIMULATION PARAMETERS BY EXCITEMENT LEVEL

For clarity, table 3.1 presents a summary of parameters for each Excitement level that are used for simulation. Additionaly for better understanding, diagnostic plots for the 15 *Pretraining* datasets are included in *Appendix C*. However, since graphical analysis of 315 datasets would not allow for thorough analysis, proper exploratory data analysis was carried out across all 315 datasets.

The table 3.2 contains descriptive statistics that were the result of the exploratory data analysis. It can be observed that for all variables and Excitement levels, means and medians are near the initial value 100. This implies the returns have near-zero mean which is consistent with standard financial time series models. Standard deviations also reveal interesting pattern: from Excitement level 0.25 to 0.5, standard deviations increases by 7.96% for variable 1, 12.54% for variable 2 and decrease by 2.55%. From Excitement level 0.50 to 0.75, standard deviation increases by 3.85% for variable 1, decreases by 7.74% for variable 2 and increases by 4.58. While this reveals a clear pattern of larger increases from excitement level 0.25 to 0.5, more can be revealed by calculating averages. If average standard deviation is calculated for each excitement level, then $\lambda = 0.25, \lambda = 0.50$, $\lambda = 0.75$ are associated with values 2.97, 3.14 and 3.14 respectively. This implies that standard deviation plateaus at excitement level 0.75. This is likely caused by imposed limits via clipping that constrain the standard deviation from increasing further.

Looking at Interquartile Ranges (IQR), minima and maxima, it can be observed that the set limits are working as designed. The bounds are preventing explosions while allowing for a realistic price movement. The minima are hovering around 94.5 while maxima are around 107 with ranges being slightly wider at excitement level 0.75. If average IQR are calculated for each excitement level, then 4.60, 4.95, 4.84 are for λ 0.25, 0.50 and 0.75 respectively. This suggest that there is also a plateau in IQR and again suggest that the model works as intended.

Skewness for all variables and Excitement levels is very close to 0 which suggest symmetric distributions. Excess kurtosis values are around -0.5 suggest platykurtic distribution which means that, compared to Gaussian, the distributions have flatter peaks and lighter and shorter tails with a very few outliers. This goes against the initial assumption of fat tails and is likely caused by the introduced limits.

The mean correlations show that the model's Excitement parameter works as intended as mean correlations increase from 0.285 (λ = 0.25) to 0.384 (λ = 0.50) to 0.465 (λ = 0.75), representing increases of 34.74% and 21.09% respectively. Again, the lower percentage increase suggests that the higher excitement level's effect on correlation dynamics is limited by the set bounds. The set bounds thus prevent the extreme co-movement that might be observed during financial shocks.

The Jarque-Bera tests systematically reject normality with passing rate being approximately zero. The approximately zero rates suggest the presence of non-Gaussian features in simulated returns. This is because the Jarque-Bera test being sensitive to any deviation from normality which includes platykurtic features. Combined with graphical analysis of

the first 15 datasets in *Appendix C*, it can be said that the distributions approximate the normal but with clear platykurtic features: flatter (and sometimes asymmetric) peaks and lighter tails as a consequence of the imposed bounds. Overall,it can be argued that simulated data creates fair representation for a distributions that are commonly observed in financial data, but without the fat tails that are more common in financial datasets.

What does the presence of platykurtic distribution and plateaus in standard deviation,IQR, and correlation dynamics imply regarding the model? It implies the acceptance of a deliberate trade-off for a large simulation study between realism and computational tractability. The safety mechanisms preserve the relative differences across excitement levels while preventing a price-crashing mechanics to occur. In effect, it limits volatility at $\lambda = 0..75$ from being too extreme. In theory, jump component and stochastic volatility should create occasional large moves that should generate fat tails, but the set limits prevent them from manifesting.The trade-off is exchanging fat tails for numerical stability and realistic price ranges. This trade-off allows for successful generation of 315 datasets with clean, reproducible results, but without computational intractability in terms of infinities, price explosions and missing values. The sacrifice is clear, no fat tails, no extreme events and lack of unconstrained stress dynamics at $\lambda = 0.75$. If the bounds were to be loosened, there is a risk of compounding jumps and price explosions which could corrupt datasets. However, the set bounds still preserve the overall model structure and thus allow for fair comparison of BVAR and TCN as they are affected in the same manner. The trade-off between realism that would be achieved by fat tails and computational tractability is necessary for large-scale simulation study. These constraints however still allow for a fair comparison as the imposed constraints still represent a a typical intraday price movements, but without major shocks.

3.2. Models and Simulation Study Procedure

The simulation study is separated into two key parts: **Pretraining** and **Forecasting Test**. These parts are discussed below in more details. The names were chosen to avoid confusion between different training and testing parts. *Pretraining* and *Forecasting Test* include their own individual train/test split to avoid data leakage.

The method of choice for validation for this study is the 80-20 percent training-testing split. Traditional cross-validation is not possible for temporal datasets as observations are not independent. While there are methods to address this issue such as moving cross validation, due to total number of observations, the 80-20 split is sufficient for the scope of this simulation study. Metrics described below are kept for each dataset and aggregated as averages for all the models. The 80-20% split is done in both parts for validation. Before discussing the experimental design in more detail, it is necessary to spend time on specifics and implementations of each model. Design summary table can be found in *Appendix E*.

Statistic	$\lambda = 0.25$ **(Calm)**			$\lambda = 0.50$ **(Normal)**			$\lambda = 0.75$ **(Stressed)**		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
Mean	100.78	100.37	100.10	100.17	100.29	99.95	101.17	100.90	100.51
Median	100.68	100.42	100.12	100.19	100.41	99.79	101.21	100.83	100.35
Std. Deviation	2.89	2.87	3.14	3.12	3.23	3.06	3.24	2.98	3.20
IQR	4.44	4.33	5.01	4.96	5.07	4.81	4.99	4.54	4.99
Minimum	94.90	94.29	93.64	93.85	93.75	93.84	94.56	94.64	94.11
Maximum	107.09	106.58	106.58	106.59	107.09	106.59	108.08	107.41	107.28
Skewness	0.11	0.00	-0.01	0.01	-0.01	0.08	0.04	0.01	0.08
Excess Kurtosis	-0.57	-0.49	-0.68	-0.71	-0.56	-0.60	-0.60	-0.54	-0.61
Aggregate Statistics									
Mean Correlation	0.285			0.384			0.465		
JB Test Pass Rate (%)	0.00			0.32			0.32		
N (Observations)	6,250			6,250			6,250		
N (Datasets)	105			105			105		

Note: Statistics computed as averages across all datasets within each Excitement level. Prices start at 100. Mean Correlation represents the average pairwise correlation between the three assets. Jarque-Bera tests for normality are represented by "JB Test Pass Rate" that indicates the percentage of datasets where the Jarque-Bera test fails to reject normality at $\alpha = 0.05$.

TABLE 3.2. DESCRIPTIVE STATISTICS OF SIMULATED PRICES BY EXCITEMENT LEVEL

3.2.1. Bayesian VAR with Minnesota Prior Implementation

Theoretical Specification

This section describes the theoretical choices that were made during the implementation of Minnesota prior and MCMC.

Minnesota Prior

Symbol	Parameter	Value	Meaning
λ	Overall tightness	0.2	Controls shrinkage strength
τ	Cross-variable tightness	10	Shrinks cross-effects relative to own
δ	Lag decay	1	Rate of decay toward zero

TABLE 3.3. MINNESOTA PRIOR HYPERPARAMETERS

The hyperparameters from table 3.3 were selected through iterative experimentation. The initial values were based on standard choices that are available in literature [11], [71] and R BVAR package [72], but the iteration method and steps were strongly inspired by the original Minnesota research [10], [31]. While the initial value for λ was kept as the standard recommendation at 0.2[71], the iteration final result for cross-variable shrinkage $\tau = 10$ and lag decay $\delta = 1$ represents a balance between a model stability and moderate

shrinkage appropriate for the three-variable simulation study.

The Minnesota prior is fitted for each individual dataset through a data-driven approach where for each dataset, the first lag β is fitted through AR(1) stochastic process. Thus, instead of relying on arbitrary choice of fixing the prior mean at 1 for own-variable first lag, the AR(1) coefficient is estimated via OLS for each variable that then are used as prior means. This approach allows the prior to reflect the unique persistence characteristics of each dataset, which enhances the Minnesota prior's lag calibration and better captures the underlying system dynamics.

The prior for the covariance matrix is based on LKJ prior [32]. This was recommended by PyMC package [73] as it is considered an appropriate choice for modern Bayesian computation. LKJ addresses limitations of the traditional BVAR's conjugate prior, Inverse-Wishart, which couples correlations and variances. This coupling tends to shrink correlations toward zero in higher dimensions. LKJ explicitly separates the correlation matrix from variances, which not only allows for more flexible prior specification but also addresses the high-dimensional issues of Inverse-Wishart. Thus, LKJ is often the preferred choice for multivariate models, while sacrificing conjugacy. The LKJ places a prior directly on the correlation matrix, which combined with *Exponential(1)* priors on standard deviations, induces a prior on Σ. The implementation uses $\eta = 2$, which slightly favours weaker correlations compared to the uniform $\eta = 1$ and is thus more suitable for high-frequency applications.

MCMC Implementation

MCMC was implemented with 2000 posterior samples with 1000 burnins per chain. The number of chain was chosen to be 4 with an acceptance rate 0.9. The posterior sample size is smaller than what is normally used (4000 to 10000 range), but iterative experimentation showed the 2000 is a sweet spot between computational speed and MCMC stability. Since the sample size is smaller, more conservative acceptance rate of 0.9 was chosen to balance the smaller sample size. Estimation was performed using NUTS sampler [36], which was implemented via PyMC [73].

Setting	Value	Notes
Draws	2000	Posterior samples per chain
Tune	1000	Warmup/burnin samples
Chains	4	For R-hat convergence diagnostics
Target accept	0.9	Conservative (default 0.8)
Sampler	NUTS	Via PyMC

TABLE 3.4. MCMC SAMPLING CONFIGURATION

Coding Specification

Unlike with TCN, BVAR requires a stationary dataset. As it would be complicated to manually check each dataset if it is stationary, a subroutine was written that checks if the dataset is stationary which includes Augmented Dickey-Fuller(ADF) test ($p<0.05$) and Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test ($p>0.10$). If the test are not met, the dataset is automatically differenced and tests are rerun. If it is still non-stationary, the dataset is differenced a second time. The code keeps track of how many times the dataset was differenced in order to revert the differencing back to prices for point forecasts and quantiles.

The code was written with maximum modularity in mind. Essentially, there are three layers: orchestration script, that is in charge of parallelisation and sequential execution; pipe script that is called by the orchestration script and which parses parameters (in BVAR's case its mainly lag, but other arguments such as for plot diagnostics are available) to the main BVAR script; and finally BVAR python code which accepts passed parameters and runs on designated .csv file and produces desired outputs(price forecasts, intervals, plots etc.). The python code for BVAR was written to be modular which allowed it to be used in all task from *Pretraining* to *Inference Showcase*. While GPU acceleration is not available, the code was able to achieve faster runtimes via vectorisation of the forecasting subroutine.

The pipe and orchestration scripts were written in Bash and followed UNIX principles, but instead of using native Linux FIFO files that require *stdin/out*, the Bash pipeline created its own standalone output files inside the working directory that allowed easier error handling and debugging. Furthermore, this setup allowed For utilising native Linux parallelisation which allowed for more BVAR models to be run at once for *Pretraining* or splitting workload for *Forecasting Test* section. The implementation pipeline and coding tools/libraries used can be found in *Appendix F*.

3.2.2. TCN with Monte Carlo Dropout Implementation

Theoretical Specification

Since TCN uses dropout by default, there was no need to adjust the model as MC Dropout is a readily available variation on the dropout architecture. Thus, unlike with BVAR, there was little need for iterative experimentation to fine-tune hyperparameters to achieve numerical stability. Instead, the choices for hyperparameters follow standard practices and recommendations:

1. **Kernel Size (3)** - standard choice in TCN as it captures local temporal patterns without excessive parameters [8]
2. **Dilation (2^i)** - standard design choice as exponential growth gives logarithmic receptive field growth [8]

3. **Lookback (50)** - standard choice is in range from 25 to 100 for high-frequency data. The 50 was chosen as a) its more computationally efficient, and b) there were only non-consequential improvements when using 100.
4. **MC Samples (100)** - standard choice as there are diminishing returns beyond 100 samples [4]
5. **Early Stopping (10)** - Given the rapid convergence in training loss, early stopping after 10 consecutive non-improving epochs is appropriate
6. **Batch Size (32)** - standard deep learning default that balances gradient noise and computation [74]
7. **Learning Rate (0.001)** - Adam optimiser's default
8. **Loss Choice (MSE)** - Matches the BVAR's MSE utilisation and is a standard metric used in regression

These choices were fine-tuned and validated iteratively with metrics and diagnostic plots, where illustrative example of a diagnostic plot is presented in figure L.2.

Coding Specification

TCN+MC Dropout design also follows a modular approach. Since the implementation is relatively simple through PyTorch, there was no need for advanced optimisation and fine-tuning techniques that were required for BVAR. Coding the modular python implementation of the model was perhaps the fastest and simplest of all codes that was written for the study and shows a clear advantage of neural network models in terms of prototyping. Furthermore, native support of PyTorch for GPU acceleration via CUDA meant that the runtimes for TCN+MC Dropout models were very fast. As seen in figure G.1, the logic is straightforward. First, the sequence is prepared with lookback=50 and then separated into train/test split without a shuffle. The model is then initiated with desired hyperparameters and trained via Adam optimiser. Forecasts are then made with MC Dropout inference with 100 dropout-activated forward passes. Quantiles are then computed from the MC Dropout sample and are then used as a part of the code output alongside forecasts.

To utilise the modular design and enable parallel computing, Bash pipes were also used. The advantage of this approach is clearest in its utilisation in *Pretraining* grid search. Grid search required to generate values for 27 models which was easily implemented via GNUParallel's parallelisation via Cartesian product of 3^3 variables. Both pipeline and coding tools/libraries can be seen in *Appendix G*.

3.2.3. Pretraining

It is computationally intractable to try to tune parameters of either model for all datasets. Therefore this paper uses a pretraining on 15 generated datasets where each Excitement parameter generates 5 datasets. Visualisation for each of the 15 dataset can be seen in the

Appendix C. The simulation code thus generates 15 distinct datasets, using seed *42* for reproducibility. In this section, the metrics are calculated across all excitement levels to find the best performing model across 15 datasets.

BVAR-MINNESOTA

For BVAR-MINNESOTA, the key metric to fit is the number of lags. Minnesota prior is fitted for each dataset. For finding the optimal number of lags, models with 3, 4, 5, 6, 7, 8 and 9 lags are fitted and than compared via DIC averages for all 15 datasets. DIC is further complemented with the metrics described below.

TCN-MC Dropout

For TCN-MC Dropout, the following variables are pre-tuned: number of dilated convolution layers (2, 3, 4); channels per layer (16, 32, 64); and dropout rate (0.1, 0.2, 0.3). Due to fast training speeds, the method used for finding the best model among all 27 configurations is Grid Search. Its key advantage is that it provides full information without build-in assumptions. Its results are only affected by the choice of comparison metrics.

Model	Parameter	Values Tested
BVAR	Lags (p)	{3, 4, 5, 6, 7, 8, 9}
TCN	Layers	{2, 3, 4}
	Channels	{16, 32, 64}
	Dropout	{0.1, 0.2, 0.3}

TABLE 3.5. PARAMETER GRID FOR PRETRAINING

3.2.4. Forecasting Test

After Pretraining is finished, two models are chosen from each category. These two are then compared head to head in the *Forecasting Test*. Both models are then trained and tested again on a new collection of datasets. For each Excitement parameter, 100 datasets are randomly generated with 6250 observations. The final scope of the study is 300 datasets containing in total 1875000 observations. This scale of simulation was chosen to achieve statistically significant results as this is the primary contribution of this study. For each Excitement parameter, comparison metrics are calculated and then compared to see if there is any difference between Excitement levels.

3.2.5. Metrics, Comparison Method and Hardware Architecture

In order to compare the models across simulations, metrics are calculated for each dataset and then averaged across datasets. For example, for final BVAR model, 100 RMSE values are calculated per Excitement level. These are then averaged. The aggregation process is discussed below. In terms of Total Runtime, the runtimes from each simulation are added together so that the total runtime is created for a model. The equations as well as hardware configuration table can be found in *Appendix H*.

Root Mean Square Error (RMSE) RMSE is a standard metric used in statistics. It penalises large errors due to the squared terms. It is also sensitive to outliers and is in the same units as the original data. For RMSE, lower is better.

Mean Absolute Error (MAE) MAE is a robust alternative to RMSE, but with less sensitivity to outliers. However, it has a much more straightforward interpretation than RMSE. In terms of interpretation, lower is also better for MAE. What also matters is the RMSE:MAE ratio. If RMSE is much larger than MAE, then there are large errors present, whereas if they are roughly the same, then errors are roughly uniform.

Prediction Interval Coverage Rate (80%, 90%, 95%) The prediction interval coverage rate (PICR) is a metric that measures the proportion of actual observations that are within the predicted interval. Well calibrated model should have PICR equal to prediction interval size.

Average Prediction Interval Width (80%, 90%, 95%) The average prediction interval width (APIW) aims to quantify the precision of forecast. Narrower imply higher precision. APIW should be used in conjunction with other interval-based metrics.

Total Runtime (Seconds) Total Runtime measures the computational cost required to fit each model and generate forecast, recorded in seconds. This metric facilitates comparison of computational efficiency across model architectures. Its a simple method to measure computational efficiency and has to be used with information about hardware architecture in table H.1. While more advanced methods exist, this one is the easiest to implement and interpret.

Deviance Information Criterion (DIC) DIC is a metric used uniquely in *Pretraining* for BVAR. It was chosen for its simplicity and its membership in a family of metrics appropriate when MCMC methods are employed as DIC was specifically designed for MCMC architectures. It is easy to compute from posterior's samples, balances fit against complexity and does not require strong assumptions about "true" models and asymptotic assumptions. Major disadvantage is that it might favour too complex models compared to alternative metrics such as WAIC. Ease of implementation was one of the major contributors for choosing this metric as it can be easily coded from scratch.

3.2.6. Aggregation Methodology Explained

Since the evaluation involves multiple levels of aggregation across datasets and variables, this section aims to describe the precise computational steps.

Per-Dataset Metrics For a single dataset d with test set of size $n_{test} = 1250$ observations and $k = 3$ variables, forecast evaluation metrics are computed for each variable independently. Thus,

$$\mathscr{M} = \{\text{RMSE}, \text{MAE}, \text{Coverage}_{80}, \text{Coverage}_{90}, \text{Coverage}_{95}, \text{Width}_{80}, \text{Width}_{90}, \text{Width}_{95}\}$$

denotes the set of metrics.

Aggregation Across Datasets For a given model configuration, *Pretraining* involves $N_{pre} = 15$ datasets, while final forecasting involves $N_{final} = 100$ datasets per Excitement level. For each metric $m \in \mathscr{M}$ and variable i, mean across datasets(equation H.6) and standard deviation across datasets (equation H.7) are computed.

Aggregation Across Variables To obtain overall performance metrics that summarise behaviour across all three variables, the mean of per-variable means and as well as the mean of per-variable standard deviations are computed (equations H.8 and H.9. This provides an average measure of variability across datasets for each variable.

Summary Statistics per Model Configuration

For each model configuration, the final reported metrics are:

- **Per-variable statistics:** $\overline{m}_i$ and σ_{m_i} for $i \in \{1, 2, 3\}$
- **Overall statistics:** $\overline{m}_{overall}$ and $\sigma_{m,overall}$
- **Total Runtime:** Total computational time summed across all datasets: $T_{total} = \sum_{d=1}^{N} T_d$
- **DIC (BVAR only):** Mean and standard deviation of DIC across datasets
- **Distribution of Orders of Differencing (BVAR only):** Count of datasets by stationarity requirement, reported as "n_0;n_1;$n_2(N)$" where n_0, n_1, n_2 represent datasets requiring 0, 1, or 2 differences respectively, and $N = n_0 + n_1 + n_2$ is the total

Rationale for Aggregation Approach

The choice to aggregate via simple means across variable rather than pooling all observations or computing weighted averages is justified by the following considerations:

Firstly, each variable represents an asset of equal importance and thus averaging gives each variable equal weight in performance metrics. Secondly, computing per-variable statistics allows to identify any potential asymmetries in model performance. Finally, overall means and standard deviation have simple interpretations where the former represents expected performance across variables, and the latter is an indicator of average variability in performance across datasets.

The alternative approach of using pooling where all observations across variables and datasets would be put together would work as follows:

$$\text{RMSE}_{pooled} = \sqrt{\frac{1}{3 \cdot N \cdot n_{test}} \sum_{i=1}^{3} \sum_{d=1}^{N} \sum_{t=1}^{n_{test}} (y_{i,t,d} - \hat{y}_{i,t,d})^2} \tag{3.18}$$

This approach however completely obscures variable-specific performance and information about performance across datasets which are important for assessing model's robustness.

3.3. Limitations

While the model selection is fair and justified, the core limitation is in the model design philosophies. Despite the simulation study, comparing two uncertainty quantifying models, the study cannot be considered as a comparison study between two Bayesian models. MC Dropout is not a true Bayesian method which limits the study contribution to the area of applied statistics.

Another key limitations are regarding the simulation design. As discussed before, more advanced correlation structures can be utilised, but require much more complicated modelling and do not guarantee computational tractability. Similarly, the safety bounds, while allowing for computational tractability, lead to thin and short tails. Real financial crises, and even intense day movements, exhibit extreme tail events whereexcess kurtosis greater than 5 can be expected, but the simulation model does not capture this reality. Future simulation design can expand on both with adding complex correlation structures, such as DCC[67] and more dynamic adaptive bounds where the limits are set via relationships between variables rather than set as constants.

The study is further limited in scope. While 315 datasets is sufficient for statistically significant results, more robust simulation study could be run across at least 1000000 datasets. The scope was limited by hardware architecture as larger simulation study would require more advanced hardware. This limitation is further exacerbated by the use of CUDA that makes it impossible to achieve perfect reproducibility of results. CUDA operations introduce minor non-determinism even when fixed seeds are implemented. While these variations are negligible(less than 1%) and do not affect aggregate conclusions, they still mean the simulation is not perfectly reproducible. Lastly, the metrics. While the metrics presented can be thought of as a gold-standard and are more than adequate for this study, much larger selection and inclusion of more advanced metrics would make the study more robust.

4. RESULTS AND DISCUSSION

This chapter will present three key results: *Pretraining* results that were used for final model selection; *Forecasting Test* results which discusses the results for comparing forecasting power; and lastly an illustrative case study that showcases method-dependent inference techniques that are associated with BVAR and TCN.

4.1. Pretraining Results

Pretraining results confirm that running a pretraining routine was a smart choice in model selection. The full results can be seen in *Appendix D*.

4.1.1. BVAR

Looking at table D.1, the results are mixed, where most results are very close to each other. DIC values are negative which suggests that the BVAR models fit data well as the log-likelihood is high. DIC penalises more complex models thus given the close proximity in performance between lags, DIC favours *Lag 3*, but with overall spread between values being approximately 0.2%. It is also not surprising that the least complex architecture of *Lag 3* leads to smallest runtime of 1715.00 seconds which is approximately 29 minutes, while the longest running *Lag 9* ran for approximately 53 minutes. However, this is where the metrics stop favouring *Lag 3*.

The overall results from table D.1 narrow down the selection to be between two lags: *Lag 5* and *Lag 7*. In favour of *Lag 7* are the overall means for RMSE and MAE which suggest highest prediction power. However, *Lag 5* shows the best standard deviations for overall RMSE and MAE which suggests *Lag 5* has more precise predictions with less uncertainty in the model's estimates. All RMSE/MAE ratios are around 1.19 with the lowest being *Lag 6*. This suggest the models are not overly sensitive to outliers as the ratio is fairly close to 1.0 (ratio for uniform errors).

Despite all that, the models all perform similarly withe very tiny differences between them. DIC ranges from -27607.22 to -27557.44, giving 0.2% spread. RMSE ranges from 2.1668 to 2.1764, giving 0.44% spread, while MAE gives spread of 0.45% for metrics ranging from 1.8197 to 1.8280. This suggest very close overall performance between lags and ultimately shows that the choice of lag is not a crucial decision in terms of fit quality. It also suggests that the simulated data are well-behaved thanks to the introduced limits and stable correlation structure as there are no extreme dynamics that would favour one specific lag. Lastly, it shows that the Minnesota prior is working effectively and was a good choice as it is regularising effectively regardless of lag choice. However, it is still possible a more advanced prior could lead to much better predictions.

To narrow down the selection between *Lag 5* and *Lag 7*, differences in coverage and per-variable metrics are analysed. The uncertainty quantification results in table D.2 show that all lags have coverage around 0.97 which suggest models are well-calibrated, but remain slightly conservative. In terms of coverage, the closest to 0.95 is the best performing model which in this case is *Lag 5*. Neither of the models dominates other uncertainty metrics, which again remain very close. Standard deviation is lower for *Lag 7* by 0.00146 and for mean width by 0.032. *Lag 5* performs better in width standard deviation with a lower value by 0.0226 points. In terms of per-variable metrics that are shown in table D.3, *Lag 7* clearly dominates in RMSE and MAE for variables 2 and 3. *Lag 5* shows best performance only in MAE in variable 1 while RMSE for variable 1 is best under *Lag 8*. Nevertheless, the differences are again tiny with around 0.002 to 0.01 point differences in metrics.

In terms of choice between the two lags, it boils down to preference. Overall, *Lag 7* model has the strongest predictive power over the 15 datasets while *Lag 5* model seems to be a better fitted model with tighter predictions and uncertainties. In the end, *Lag 5* was chosen as the better model for the following reasons: **(1)** the runtime was quite decent with 2461.41 seconds per 15 models; **(2)** all metrics are very close across lags with minute differences; **(3)** is less complex than *Lag 7* and thus can be considered more parsimonious; **(4)** has the best coverage for 95% interval; **(5)** dominates in standard deviations for RMSE and MAE which shows much; tighter predictions across models; and **(6)** while given results it is unlikely to occur for *Lag 7* the lower number of lags prevents overfitting.

In short, *Lag 5* has better calibrated uncertainty and is more consistent than *Lag 7* with small differences in accuracy between them. This means that despite lower forecasting performance across 15 pretraining datasets, the model should have more consistent performance across the *Forecasting Test's* 300 datasets.

4.1.2. TCN+MC Dropout

Unlike with BVAR, selection process for the best model was straightforward with TCN as the Grid Search had more fruitful results. The final model that was chosen was *model 16*, whose specification can be found in the table A.1.

While the model was not the fastest with 262.44 seconds compared to the fastest model 8's 192.79, the model dominated RMSE and MAE's mean metrics with them being 7.98% and 9.16% lower than than the second best performing model (*model 7*) respectively. *Model 7* performed best in standard deviation of RMSE and MAE, indicating more precise predictions, but with only 3.85% and 3.30% improvements respectively compared to *model 16*. All trained models had RMSE/MAE ratio ranging from 1.05 to 1.1 indicating that the models are not sensitive to outliers with having near uniform errors.

Uncertainty quantification displayed in table D.5 fundamental weakness of Bayesian approximation methods that will be discussed later. Based on data, all models are over-

conservative as their coverage is nearly 1. *Model 16* still remains the 2nd best performing model in terms of width mean(best is *model 7*) and width standard deviation (best is 25) with It performing 0.56 and 4.58% worse respectfully.Nevertheless, *model 16* outperformed other models in per-variable statistics that are seen in table D.6. It was only 2nd best performing in RMSE(mean) and MAE(mean) for variable 3 by 1.31% and 0.88% respectively compared to the best performing *model 7*.

While *model 7* had a potential with a possibly much precise predictions, its uncertainty advantage was negligible and its per-variable performance poor. Therefore, *model 16* had the best performance overall by far compared to other models and was a clear choice to compete against BVAR *Lag 5*

4.2. Forecasting Test

This section will discuss the centre of the study: the *Forecasting Test* results. Unlike in previous section where the aggregate overall performance was preferred to individual metrics, this section will go through detailed per-variable and per excitement level performance to find the strengths and weaknesses of each model. This section will discuss individual Excitement level results first and then summarise the results overall. Full results can be seen in *Appendix B*.

4.2.1. Results for Excitement Level 0.25

Looking at the results in table B.1, its interesting that *BVAR Lag 5* model only needed to go through 1st-order differencing. *TCN Model 16* performed significantly faster, as its runtime was lower by 88.70%. However, *TCN Model 16* shows clear weaknesses as its overall means for RMSE and MAE are 39.35% and 52.57% larger and their standard deviations are 61.41% and 82.51% larger. This shows that overall predictive power is higher for *BVAR Lag 5* model, but BVAR seems to be more sensitive to outliers as *TCN Model 16* has lower RMSE/MAE ratio by 8.66% showing its slightly less sensitive to outliers as its closer to 1.0.

This trend is reflected in per variable breakdowns in tables B.4 and B.5. the closest performance of *TCN Model 16* to *BVAR lag 5* model was in the variable 2, where TCN was only 25.62% and 36.3% larger in RMSE and MAE means, but the worst performance was in variable 1 with 47.1% and 62.35% increase in metrics.

In terms of uncertainty, *BVAR Lag 5* is well-calibrated with coverage near their intended target where BVAR achieves near-nominal coverage at 95% (0.956-0.962). On the other hand *TCN Model 16* is extremely overconservative with the coverage values being much larger than intended target values and are approaching 100% actual coverage. Furthermore, the TCN widths are approximately 275% wider than BVAR. In short, *BVAR Lag 5* has much tighter and well-calibrated uncertainty quantification while *TCN Model*

16 tends to favour overconservatism.

4.2.2. Results for Excitement Level 0.50

Again, *BVAR Lag 5* model only needed to go through 1st order differencing, but *TCN Model 16* was still 88.92% faster. What is interesting is that RMSE mean had slightly lower percentage increase to 27.69% from *BVAR Lag 5* model to *TCN Number 16*, While also having much lower increase in standard deviation being 21.61% compared to Excitement level 0.25's 61.41%. MAE metrics also experienced lower increases compared to 0.25 Excitement level, where TCN's respective MAE means and standard deviations only increased by around 38%. In terms of per-variable statistics, there is an interesting outlier where Variable 3 had comparatively smallest increases in RMSE and MAE means of only 13.57% and 21.7% respectively, but the RMSE standard deviation actually decreased by 3.14% with MAE having lowest increase in standard deviation of only 9.33% from BVAR to TCN. The uncertainty metrics are also interesting. BVAR has a slightly lower coverage, for example at 95% 0.932-0.960) compared to Excitement level 0.25 (0.956-0.962), with wider intervals. The combination of lower coverage and wider widths suggests that the model finds it more difficult to create accurate predictions as the Excitement level increases. TCN meanwhile remains overconservative, with much stronger coverage approaching 100% (0.999-1.000). The standard deviations across interval metrics have increased from 0.25 Excitement. *TCN Model 16* still has 2.7 times wider intervals (approx. 27.4 vs 10.2), meaning the ratio is approximately the same as in Excitement level 0.25.

4.2.3. Results for Excitement Level 0.75

Following the trend so far, *BVAR Lag 5* model under 0.75 Excitement level also needed only to be differenced once with *TCN Model 16* runtime being 89.24% faster. . The runtimes are also approximately same with excitement levels 0.25 and 0.50. What is interesting though is that instead of continuing the trend of improving *TCN Model 16* metrics, Excitement level instead forms a middle ground between before mentioned Excitement levels. RMSE and MAE mean increases from BVAR to TCN are 35.17% and 47.03% respectively, while having the highest percentage increases in standard deviations being 68.46% and 86.37% respectively. In terms of per-variable statistics, the results show much tighter spreads for BVAR across variables than for other Excitement Levels. Looking at RMSE standard deviation for *BVAR Lag 5* at each Excitement level, Excitement level 0.75 has the smallest standard deviation in variable 1 and 2. In variable 3, it has the 2nd smallest with a difference by approximately 0.15 from variable 1.

- *Variable 1* (1.24846; 1.23073; **1.19328**)
- *Variable 2*(1.32814; 1.52342;**1.30682**)
- *Variable 3* (**1.13787**;1.61616; 1.28575)

More interestingly, RMSE standard deviation for *TCN Model 16* at each Excitement level shows it being worst performing in variable 2 and 3. In variable 1, Excitement level 0.75 is 2nd only by a small margin of 0.04 compared to the worst performing Excitement level 0.25. In general, it would be expected that both models would behave as TCN, where with increasing Excitement level, there would be larger standard deviation. The possible causes of this discrepancy will be discussed in the next section.

- *Variable 1* (2.24748;**1.788906**;2.20354)
- *Variable 2* (**1.87671**; 1.96122;2.09318)
- *Variable 3* (1.97143;**1.56540**;2.08100)

Uncertainty is also interesting, BVAR is now much better calibrated across all coverage compared to Excitement level 0.50. For example, BVAR's 95% coverage spread is 0.951-0.959, compared to Excitement level 0.50's 0.932-0.960, and closer to the nominal 95% target. TCN remains overconservative with results approaching 100% actual coverage for all coverage. The width ratio between models remains approximately the same (2.65 times larger for TCN), with TCN's 95% width averaging approximately 27.8 compared to BVAR's approximate width of 10.5. What is interesting and expected is that, according to table 4.1 the width of BVAR and TCN is increasing across Excitement levels which suggest the uncertainty quantification is well calibrated across Excitement levels for BVAR, and behaving appropriately for TCN+MC Dropout.

4.2.4. Forecasting Results Discussion

Excitement Level	BVAR Width (95%)	TCN Width (95%)
0.25	10.0	27.2
0.50	10.2	27.4
0.75	10.5	27.8

TABLE 4.1. APPROXIMATE 95% PREDICTION INTERVAL WIDTHS ACROSS EXCITEMENT LEVELS

Model	Runtimes per excitement level			Overall Runtime
	0.25	0.50	0.75	
BVAR	16165.32	16425.75	13024.58	45615.65
TCN	1826.41	1820.39	1401.18	5047.98

TABLE 4.2. COMPARISON OF RUNTIMES ACROSS MODELS

The results are very interesting. On paper, BVAR is much better model overall.It beats *TCN Model 16* across all metrics for both mean and standard deviations, where BVAR beats TCN by 27-39% on RMSE and 38-53% on MAE,and creates much better uncertainty

	MEAN			STD		
	var1	*var2*	*var3*	*var1*	*var2*	*var3*
Excitement Level 0.25						
RMSE	47.09780	25.62270	45.89568	80.02018	41.30363	64.46782
MAE	62.35325	36.30348	59.65070	101.20322	61.24842	86.38137
Excitement Level 0.50						
RMSE	44.74273	26.07375	13.56602	45.28451	28.73797	−3.14078
MAE	59.02754	37.11682	21.70057	67.43453	46.25676	9.33431
Excitement Level 0.75						
RMSE	41.03261	27.52888	36.96690	84.66244	60.17355	61.85106
MAE	53.52229	38.28334	49.23132	105.29249	75.06313	80.47476

TABLE 4.3. PER VARIABLE STATISTICS % CHANGE FROM BVAR TO TCN

Excitement Level	Runtime	RMSE		MAE		RMSE/MAE
		mean	*std*	*mean*	*std*	
0.25	−88.70168	39.35029	61.41340	52.56750	82.50914	−8.66324
0.50	−88.91746	27.69193	21.60876	38.73348	38.45589	−7.95888
0.75	−89.24203	35.17435	68.46230	47.03098	86.36637	−8.06449

TABLE 4.4. OVERALL STATISTICS % CHANGE FROM BVAR TO TCN

quantification, with TCN intervals being approximately 2.7 times wider while achieving near 100% coverage vs BVAR's calibrated 95%. Facing these results, it is important to ask the question why BVAR dominates across metrics.

Since the simulation design was inspired by minute-trading data, BVAR might have unintentionally gained competitive edge. Firstly, the data was very stable with all 315 datasets only requiring first order differencing. Secondly, the strange decrease in performance under 0.50 Excitement level. This phenomenon could be attributed to the three key design choices (Cholesky-based correlation structure, stochastic volatility, jump diffusion) mimicking how in periods of high stress, the correlations between stocks increases as they move together when facing the stress. Combining that with the simulation model being based on returns, it gives a competitive edge to VAR models which have advantage in terms of detecting these correlation structures.

BVAR is also truly Bayesian framework and thus can create proper Bayesian interval whereas MC Dropout is just an approximation of Bayesian inference. This is the main reason why *TCN Model 16* remained overconservative throughout the process with prediction intervals being only useful in capturing worst case scenario for each point forecast.

Despite BVAR's dominance across metrics, the results are more nuanced. In practice BVAR is not only significantly slower than TCN to prototype, train, test and implement, its design can never truly compete within current hardware architecture frameworks. BVAR design utilises MCMC which currently does not allow for GPU acceleration due to MCMC being sequential. In other words, there is a limit to how much data can BVAR process with no clear sign of improving performance in terms of hardware architecture. MCMC's utilisation of Markov chains, which are innately sequential, with clear accept/reject logic, creates unique combination of sequential processing with logical overhead that results in irregular memory usage. This makes GPU acceleration nearly impossible. Meanwhile TCN and neural networks in general: allow for much more modular design; are much faster with GPU acceleration allowing for use of Grid Search and rapid prototyping; and provide reasonable results in terms of forecasting and uncertainty quantification. Combining the above, it is not surprising that the total runtime for *BVAR Lag 5* was 45616 seconds (approximately 12.7 hours) vs *TCN Model 16* 5048 seconds (approximately 1.4 hours). While recent developments,such as JAX-based probabilistic programming, better vectorisation and MCMC algorithms are actively developed, and led to considerable decrease in runtimes, the fundamental sequential bottleneck of MCMC still persist. Thus, without major paradigm shift, neural networks will always beat Bayesian MCMC-based models due to their massive parallelisation and GPU acceleration capabilities on large datasets.

The results suggest that due to limitations of BVAR, it is best suited for cases where: *(a)* dataset is clearly suitable for BVAR, such as financial datasets; *(b)* Bayesian uncertainty quantification is non-negotiable; and *(c)* point-forecast accuracy takes precedent over speed. In other cases, TCN is more than suitable to be used in production and time-

sensitive research. Given these results and modular nature of TCN, a possible solution would be to use BVAR to find credible intervals from sampled data, and then leveraging the TCN's modularity to approximate found intervals with Bayesian approximation methods such as MC Dropout.

4.3. Case Study: Comparison Between Model-Specific Inference Techniques

While this study focuses on forecasting, it is also important to give a brief illustrative example of tools and frameworks that are available within each model for inference.

It should be noted that there are many method-agnostic inference techniques that may outperform the methods stated below. These methods include, but are not limited to: Granger Causality, a weak form of causality based on whether past values of one variable improve linear predictions of another; Transfer Entropy, Non-linear method describing how information from one stochastic process affects values of another stochastic process; Correlation Analysis, techniques focusing on non-temporal relationships between variables; and Feature Importance Methods, an entire family of techniques such as permutation-based methods, Shapely Additive Explanations (SHAP) and many more.

4.3.1. Case Study Dataset: Excitement Level 0.50 - Pretraining Dataset 1

While more detailed analysis of the dataset could be carried out, since the dataset is simulated and used for illustrative purposes, there is no need for detailed analysis. After all, there is no strict meaning behind Variable 1 and 2 and their relationship. Instead, in this subsection, a graphical analysis of the dataset will be carried out to get a bit of an understanding of the specific mechanics of the dataset. Same graphs can be found for all *Pretraining* datasets in the *Appendix C*.

In the top-left graph in figure C.6 , the time-series for prices for all three variables can be seen. They are labelled as *asset_1, asset_2* and *asset_3*. All start from indexed value 100. Around a 3000 time value, an Excitement level process is starting to take shape where all variables start to decrease and end on values lower than at time 0. Variable 1 ends near 84, variable 2 near 80 and variable 3 near 95. In the top-right corner, distributions for returns (calculated via $(P_t - P_{t-1})/P_{t-1}$) shows that all distributions are symmetric and approximately normal. While visual inspection is in general not enough to establish normality, it can be said that a) simulated data is well-behaved given it generates approximately normal returns b) BVAR is a suitable model to use as the data suggest non-extreme outliers and Minnesota priors Gaussian likelihood is well calibrated for this dataset under 1st order differencing of price data. Bottom-left graph shows time-varying volatility(calculated via standard deviation of returns) for 30-period rolling window. From the graph, it is clear that volatility is not constant, but instead clusters to periods of high and low volatility. This confirms that the Ornstein-Uhlenbeck process was implemented correctly in the

simulation and represents a common assumption about financial data behaviour. On the graph, there is a visible spike around 3500 timestep that corresponds to the price drop visible in top-left graph. The volatility ranges from 0.0005 to 0.00225. The presence of this stylised fact is very important for BVAR. BVAR assumes homoskedastic errors, but based on the graph the errors are heteroskedastic. Nevertheless, this error is often assumed as non-consequential and BVAR continues to be used on financial data by econometricians, despite the violation of assumptions. Lastly, bottom-right graph is a Pearson's correlation matrix heat map that shows variable correlations (variable 1-2: 0.37; variable 1-3: 0.38; variable 2-3: 0.35) which suggest the Cholesky-based correlation structure and Excitement level hyperparameter are working as intended and thus the dataset is by definition multivariate.

4.3.2. BVAR Inference Showcase

A part of any inference procedure, as well as model assessment, is to analyse diagnostic plots. For *BVAR Lag 5*, these plots can be found in the figure L.1. The first row contains three plots: trace plot, posterior β_1 distribution and R-hat plot. The trace plot shows good mixing in the chains. There is no trends, drifts or divergences and all β coefficients are exploring the parameter space well. This suggest proper function of the MCMC implementation and its stability. The middle β_1 posterior distribution shows a unimodal distribution, although with some skewness present, which is roughly symmetric. The peak is around -0.005. All of this suggest a functioning posterior. It should be noted that under proper exploration, β posteriors should be explored and to see if they resemble the appropriate distribution. Last on the row is R-hat plot. R-hat compares variance within each chain to variance between chain to see if the chains are working properly. Based on the graph, not only are the first 10 parameters under the 1.10 threshold, they are all very close to 1.00 which suggests excellent performance of MCMC chains. The second row shows the forecast of the 200 timesteps with 95% prediction intervals. The forecast and prediction intervals are created from the 8000 posterior samples(2000 draws x 4 chains) thus the prediction interval includes both β parameter uncertainty and forecast uncertainty (noise from Σ). These plots show that for all variables, the predictions are relatively flat with compounding uncertainty over time, which is represented by the widening fan shape of the prediction interval. For all variables, the actual values lie within the 95% prediction interval which is a good calibration. The last row shows error distributions for each variable. Under ideal conditions, the distribution would have a mean error around zero with symmetry around the mean. This is roughly seen in variable 1. However, distributions of errors for variable 2 are right-skewed with mean 1.126 and variable 3 being left-shifted with mean -1.466. This suggest that variable 2 has systematic underprediction and variable 3 has systematic overprediction, both consistent with forecast plots from second row. This is not surprising given Minnesota prior conservative nature and the error compounding over the 1250 time-step horizon.

Impulse Response Function

One of the classic inference tools associated with VAR-based methods are Impulse Response Functions (IRF). IRF traces shock to one variable and its effects on all variables over time. The identification and shock mechanism uses the Cholesky decomposition of the error covariance matrix Σ. Key aspect of this approach is that it is unidirectional, meaning that shocked variable affects other variables and not vice versa.

In a classic VAR(p) model, the IRF at horizon h is computed recursively as follows:

$$\Phi_0 = P, \quad \Phi_h = \sum_{j=1}^{\min(h,p)} B_j \Phi_{h-j} \tag{4.1}$$

where P is the Cholesky factor of Σ (i.e., $\Sigma = PP'$) and B_j are the VAR coefficient matrices at lag j. The response of variable i to a shock in variable j at horizon h is given by element (i, j) of Φ_h. Thus, the overall design of IRF uses a deterministic shock propagation mechanism.

In the inference showcase example, IRF was computed over 8000 posterior samples. The shock size was 1 standard deviation to variable 1. The credible interval sizes presented are 68% and 95%. Since the percentiles are computed across posterior samples Of the IRF and not from noise, the intervals are truly Bayesian and quantify parameter uncertainty. Given easy implementation of the sampling algorithm via PyMC and relatively easy maths of Cholesky decomposition, IRF can be easily implemented without the need for additional libraries. This means IRF paths' distribution is easily implemented compared to other parts of BVAR implementation.

The showcase example of IRF in figure 4.1 clearly shows the relationship between variables under a shock. While variable 1 recovers relatively soon within 2 time steps with tight credible intervals, both variable 2 and 3 recover later between steps 5 and 7. The spillover effect peaks around 0.035 and 0.04 for variables 2 and 3 respectively. All variables decay to zero by approximately step 7. The credible intervals for variables 2 and 3 are wider than for variable 1 which means there is a greater uncertainty in the spillover effect of variable 1 to variables 2 and 3.

The advantage of this approach are clear: BVAR allows for Bayesian uncertainty quantification of shock modelling across variables. This is often favoured technique in econometrics, bit has a one single drawback: the model assumes mean-reversion. This assumption is key limitation as not all shocks in reality revert to the mean. Thus, IRF functions should be only applied to data which satisfies that assumption. In the case of simulated data, it is reasonable that the assumption is justified since the IRF is build from 1st order differencing of prices and, while still debated within the field of econometrics, it is often assumed that returns meet this requirement.

Forecast Error Variance Decomposition

Forecast Error Variance Decomposition(FEVD) is a technique that is linked to IRF as it can be derived from IRF samples. It works by quantifying the proportion of each variable's forecast error variance that is attributable to shocks from each variable. For a shock to variable j, the contribution to variable i's forecast error variance at horizon h is:

$$\text{FEVD}_{i \leftarrow j}(h) = \frac{\sum_{s=0}^{h} (\Phi_s)_{ij}^2}{\sum_{s=0}^{h} \sum_{k=1}^{K} (\Phi_s)_{ik}^2} \tag{4.2}$$

Since this section is only illustrative, the implementation only computes FEVD from shocks to variable 1, but in practice should be carried out across all possible shocks. The implementation thus shows how much variable 2 and 3's forecast uncertainty is explained by the shock in the variable 1. The implementation's FEVD was calculated from IRF samples that were previously generated and utilises 68% credible intervals. There is also one key simplification in the implementation as the code normalises FEVD samples by total cumulative variance at final horizon across all variables which was used since the implementation only showcases FEVD in terms of effect of a variable 1.

From figure 4.2, FEVD are constant with proportion of variance being explained mostly by variable 1 (approximately 0.82) with variables 2 and 3 having similar contribution (approximately 0.09). This shows that, despite variable 1 shock explains most of the system variance, the secondary contribution of variable 2 and variable 3 correlation effects contributes meaningfully to the cross-variable system dynamics. The linear nature of the FEVD plots is also expected given the rapid IRF decay because the shock reverts back to the mean by timestep 7, which causes the relative variance contributions stabilise very early.

In real-world applications, FEVD can show interesting results. Hypothetically, while initial shock in variable 1 can create devastating impact over the whole system, FEVD can then show that while variable 1 created the initial shock, it was the secondary shock of variable 2 that is responsible for the entire system as its proportion of variance is significantly greater than that of the variable 1.

4.3.3. TCN+MC Dropout Inference Showcase

Unlike BVAR, TCN is inherently stochastic model which makes it much more difficult for inference. Neural network training involves stochastic elements such as weight initialisation and dropout, but the architecture (CUDA operations, DataLoading etc.) also add to the stochastic issue. This inherent randomness means that, even if seeds are used across architectures, there will still be slight variation between runs. TCN is designed with slight, but bounded variations which capture the overall trends. This lack of determinism means that in practice neural networks are at a strong disadvantage for inference. While RMSE

and predictions have inconsequential variations, each run puts different weights on each feature and thus can lead to drastically different results of TCN run on a single dataset. In practice, it is highly recommended to do multiple runs on a single dataset and aggregate results for inference. This should average-out variations and reveal consistent trends and relationships across data.

Looking at figure L.2, the model appears to be well-calibrated with expected behaviour. In the first row, there are three graphs: training loss curve, final loss distribution and prediction interval coverage. The training loss curve shows rapid convergence within first five epochs to a stable level by 20th epoch which suggest the early stopping is working properly. The final loss distribution shows that MSE loss has a tight distribution around the final value (49.72) with the interval [47, 52] with a left-skewed distribution. This again confirms the model is working properly. The 95% prediction interval coverage graph shows that the intervals are overconservative with them having approximately 100% coverage which is MC dropout's is expected behaviour. Second row contains actual vs. predicted plots for assets. These plots show the classic variance-bias trade-off where variables 1 and 2 have marginal bias, but larger variance. The variable 3 is the opposite: smaller variance and large bias. All plots show reasonable tracking along the diagonal which suggest that support the model being well-calibrated. Last row shows error distribution plots. The small bias is now clearer for plots for variables 1 and 2 whose means are 0.271 and 0.259 respectively. Meanwhile variable 3 has a stronger positive bias with a mean 3.904. The errors appear to be near-normally distributed with symmetric distributions and only slight right-skewness in variable 2 errors. This suggest near-normal and well-behaved errors with no significant outliers.

Figure 4.3 shows prediction plot including actual vs. predicted. The bias in variable 3 is again clearly shown with predictions having values approximately 4 points higher than the actual values. The 95% prediction intervals are wide as expected. What is also interesting that while variable 1 and variable 2 are less biased, they do no follow the trends as accurately as the response in variable 3. Overall, the plot suggest well-calibrated forecasts across variables.

Shock Propagation: Alternative to TCN

Compared to BVAR, TCN does not have an explicit covariance matrix Σ or coefficient matrices B_j with no structural assumptions build into the model. TCN does not allow for the IRF analytical approach shown by equation 4.1. Instead, to achieve a similar analysis, empirical shock propagation analysis can be used that simulates how the TCN responds to an exogenous shock.

Shock Identification For the purposes of this showcase, the size and nature of the shock must be identical to the one in BVAR's IRF. The shock is identified using Cholesky decomposition of the empirical covariance matrix. Let $\hat{\Sigma}$ be the sample covariance matrix

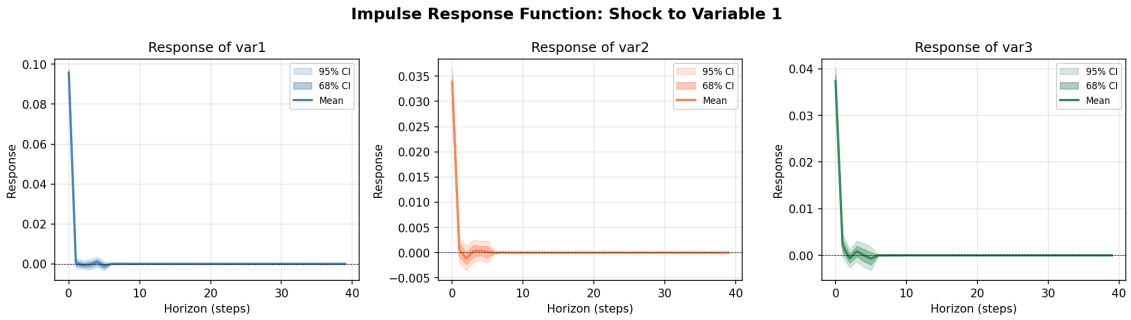


Fig. 4.1. BVAR(Lag 5) - IRF Plots

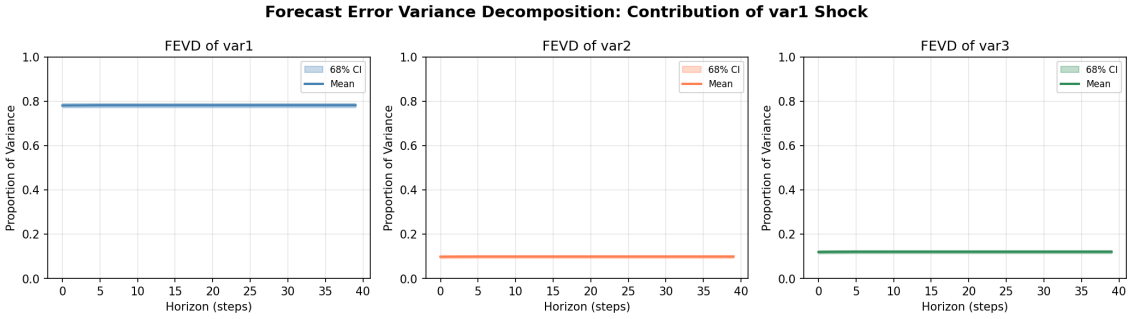


Fig. 4.2. BVAR(Lag 5) - FEVD Plots

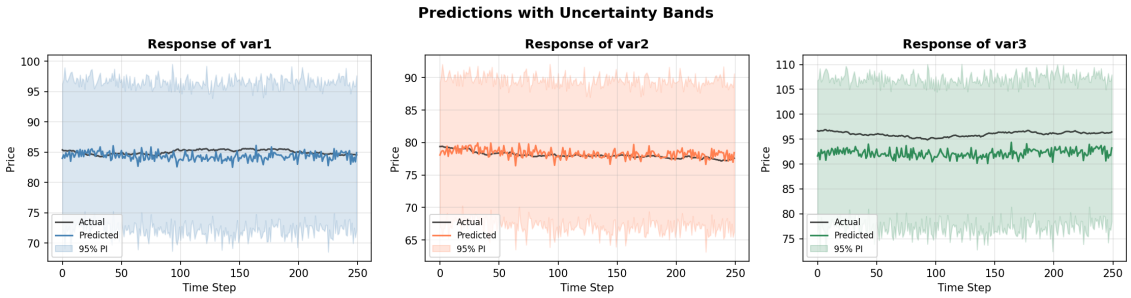


Fig. 4.3. TCN+MC Dropout (Model 16) - Uncertainty Predictions

of training-set returns:

$$\hat{\Sigma} = \frac{1}{T-1} \sum_{t=1}^{T} (r_t - \bar{r})(r_t - \bar{r})' \tag{4.3}$$

where r_t represents the return vector at time t. The Cholesky decomposition yields:

$$\hat{\Sigma} = LL' \tag{4.4}$$

where L is a lower triangular matrix. A one standard deviation orthogonalised shock to variable j is given by the j-th column of L:

$$\epsilon_j^{shock} = L_{\cdot,j} \tag{4.5}$$

The above shock affects all variables through their contemporaneous correlations which is a method that is equivalent to the identification strategy utilised by BVAR.

Propagation Mechanism Unlike BVAR's recursion, TCN shock propagation is computed via forward simulation. Let $X_t = (x_{t-k+1}, \ldots, x_t)$ denote the input sequence of length k (lookback window). The procedure is:

1. **Baseline:** Select a baseline sequence X_t^{base} from the test set
2. **Shock injection:** Create shocked sequence X_t^{shock} by adding ϵ_j^{shock} to the final observation:
$$x_t^{shock} = x_t^{base} + \epsilon_j^{shock} \tag{4.6}$$
3. **Forward propagation:** For each horizon $h = 0, 1, \ldots, H-1$, generate predictions using the pre-trained TCN model and roll the sequence forward using predicted values
4. **Response computation:** The response at horizon h is the difference from baseline prediction:
$$R_h = \hat{y}_{t+h+1}^{shock} - \hat{y}_{t+h+1}^{base} \tag{4.7}$$
5. **Uncertainty bands:** Calculated via MC Dropout for each horizon h, $M = 100$ forward passes to generate distribution of responses $\{R_h^{(m)}\}_{m=1}^{M}$

In figure 4.4, the red dashed line shows the shock at $h = 0$. Unlike with the IRF, the scale means that the initial shock magnitude is not clearly visible hence the red dashed line. For all variables, the stochastic oscillation around the zero mean can be observed with varying credible intervals.

There are key differences from BVAR. Shock propagation does not assume mean-reversion. This not only allows the oscillation around zero shown in figure 4.4, but it also

means that if a shock is large enough, exponential explosion or decay from the mean could be observed as a result of a shock. This is due to the the lack of strong assumptions present with TCN and allows for purely data-driven shock modelling.

TCN's shock propagation also has no equivalent of FEVD as TCN lacks structural coefficient matrices. While there are ways to achieve similar-looking visualisations that could be constructed, the results do not share the same interpretability and have limited utility for inference. For variance contributions that are statistically meaningful, another approach using a structured model should be chosen over TCN.

Lastly, shock propagation suffers from the same weakness that has been observed throughout this chapter, overconservative uncertainty quantification due to MC Dropout.

Regime Change Detection

Since TCN cannot do FEVD, it seems fitting to showcase a technique that cannot be effectively mirrored by BVAR. MC Dropout allows for input-dependent uncertainty quantification which, unlike BVAR's prediction intervals being derived from a fixed posterior distribution and structural covariance matrix Σ, allows to generate intervals specific to a chosen sequence. This means that, when there is a divergence between patterns in training data and patterns in generated forecasts, MC Dropout's samples also diverge thus creating higher prediction standard deviation.

This allows for uncertainty regime detection method to be utilised which identifies periods where the model is less confident in its predictions.

Methodology For each test observation t, the prediction standard deviation is computed across $M = 100$ MC Dropout samples:

$$\sigma_t = \sqrt{\frac{1}{M-1}\sum_{m=1}^{M}(\hat{y}_t^{(m)} - \bar{\hat{y}}_t)^2} \tag{4.8}$$

where $\hat{y}_t^{(m)}$ is the m-th MC sample prediction and $\bar{\hat{y}}_t$ is the mean prediction. This yields a time series of uncertainty that can be analysed for regime patterns.

In the figure 4.5, variable 3 has the highest baseline uncertainty with approximately 7.5 units. Variable 1 and 2 baseline are between 5.5-6 unis. Furthermore, there is a clear variation in uncertainty over time with spikes that may indicate difficult-to-predict periods. The figure also shows that there is no clear "regime" structure with periods of high and low uncertainty, but rather oscillates with occasional spikes in uncertainty. These spikes may represent periods where jump-diffusion process creates unpredictable jumps within the simulated data.

4.3.4. Strengths and Weaknesses Compared Across Models

The strengths and limitations of BVAR are due to its explicit design choices. Because BVAR is a truly Bayesian model, it allows for Bayesian inference methods to be used. Furthermore, the presence of covariance matrix Σ and coefficient matrices allows for straight-forward inference based via IRF and FEVD. However, the strong-built in assumptions limit where the model is used for inference, such as if the data has heteroskedastic errors. The built-in mean-reversion within IRF also means that the analysed dataset should satisfy that assumption as not all shocks decay towards the mean. Due to the limitations based on strong assumptions and explicit design elements, the usefulness of BVAR-based tools for inference should be considered in light of method-agnostic approaches that may lead to better inference on case-by-case basis.

TCN on the other hand, does not have explicit assumptions build into it. It does not include mean-reversion, nor any specific covariate structure. It does not rely on strong assumptions about data and thus allows for using non-linear methods, such as the presented uncertainty regime detection. TCN also allows for shock propagation analysis that does not rely on the assumption of mean-reversion under a shock and thus may model shock dynamics more accurately. However, this is where the advantages end. The lack of explicit structure, such as covariance matrix, means that it is very difficult to extract information about specific variables and their relationships between each other. This is why TCN does not have its own technique equivalent to FEVD. This means that, while TCN can more accurately model shocks and precise dynamics, it cannot say precisely why these dynamics occur. At best it may say which variable's shock is the most devastating. Furthermore, as observed throughout this chapter, MC Dropout produces non-callibrated uncertainty estimates, limiting the precision of any inference drawn from prediction intervals.

Adding the inherent stochasticity of neural network modelling, it means that each trained model diverges in their inference results. One run may put more importance on variable 1 while other on variable 2 and these relationships may lead to drastically different shock propagation results. The only way to address this is to run the model multiple times on a single dataset and aggregate the results. This makes already difficult inference methodology much more complex and shows an inherent weakness when using neural networks for inference.

Both BVAR and TCN inference methods showcase was just a taster of the methodology implemented for inference within each paradigm. Each of these models build upon the aforementioned foundation a myriad of hypothesis tests, methodologies, inference models and many more tools. Both frameworks offer distinct advantages and limitations, but one fundamental distinction persists: BVAR provides truly Bayesian inference with interpretable structural parameters, while TCN offers flexibility at the cost of inferential clarity.

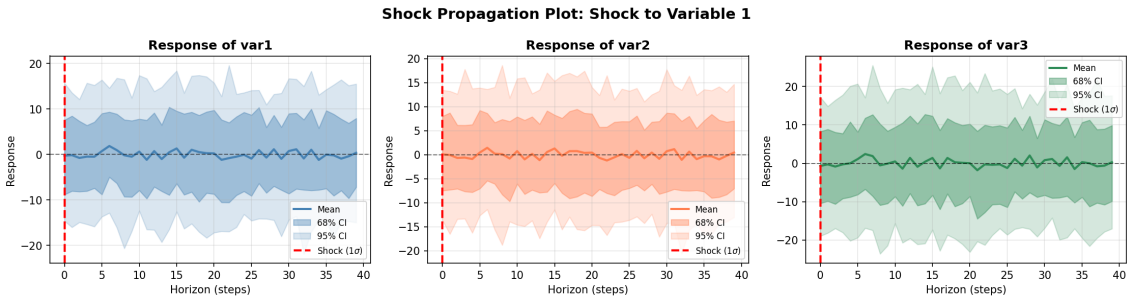


Fig. 4.4. TCN + MC Dropout (Model 16) - Shock Propagation Plots

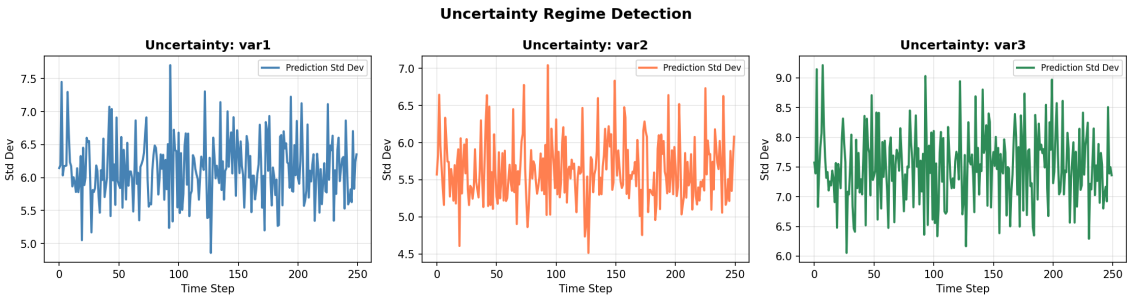


Fig. 4.5. TCN + MC Dropout (Model 16) - Uncertainty Regime Detection

5. CONCLUSION

This simulation study was able to fulfil its initial aims successfully. The forecasting results show that not only BVAR frameworks remain competitive, but BVAR outperformed TCN+MC Dropout across all forecasting metrics: BVAR beat TCN by 27-39% on RMSE and by 38-53% on MAE, while also creating much better uncertainty quantification. TCN intervals were approximately 2.7 times wider than BVAR's while achieving near 100% coverage vs BVAR's calibrated 95%. The simulation design, that was based on minute-level trading data, might have unintentionally gave BVAR a competitive edge. Furthermore,BVAR created proper credible and prediction intervals whereas MC Dropout is only a Bayesian approximation and thus cannot reach BVAR'S abilities in uncertainty quantification. This is the main reason why *TCN Model 16* remained overconservative throughout the study with prediction intervals being only useful in capturing worst case scenarios for each point forecast.

Despite BVAR's clear dominance in forecasting, the results are more nuanced. BVAR remains to be significantly slower to prototype, cross-validate and implement. MCMC also limits BVAR's computational performance as its inherent sequential nature means it cannot use GPU accelerated architectures. In other words, there is a ceiling in terms of BVAR implementation in production and applied settings. The runtime of both models illustrate this appropriately as *BVAR Lag 5*'s 45616 seconds (approximately 12.7 hours) is significantly slower than *TCN Model 16* 5048 seconds (approximately 1.4 hours). This limits the scope of relevant applications. TCN models further benefit from their modular design and GPU acceleration.

In terms of inference, while not being the focus of this study, it was shown that both model frameworks have their own advantages. BVAR allows for much cleaner inference due to its Bayesian structure, but relies on strong assumptions that need to be present in data. TCN does not rely on assumptions and thus can employ techniques, such as shock propagation, more easily. However, TCN's inherent stochasticity means inference requires multiple runs on the same dataset, and thus inference can only be done through aggregation and usage of Bayesian approximation methods. In practice, this means that TCN's lack of mean-reversion assumption implies that it can more effectively model the impact of a shock, but cannot describe precisely what is the mechanism behind the shock.

The results of this simulation study suggest that, due to slower runtime, BVAR is best suited for cases where: *(a)* dataset is clearly suitable for BVAR, such as financial datasets; *(b)* Bayesian uncertainty quantification is non-negotiable; and *(c)* point-forecast accuracy takes precedent over speed. TCN+MC Dropout appears to be suitable for production-level code and time-sensitive research. Furthermore, both methodologies have their advantages and disadvantages in terms of inference, where BVAR's Bayesian design has clear advantages over MC Dropout Bayesian approximation.

The simulation study had a few key limitations. Firstly, the scope of using 315 datasets may not be enough for achieving truly robust results. A much larger simulation study would have allowed for more robust results. Secondly, the aforementioned stochasticity of TCN hurts reproducibility of the results. CUDA operations introduce minor non-determinism even when fixed seeds are implemented. While these variations are negligible(less than 1%) and do not affect aggregate conclusions of this study, they still mean the simulation is not perfectly reproducible. Another limitation of the study is that it does not compare two truly Bayesian frameworks. While TCN+MC Dropout is a fair Bayesian approximation method, compared to methods that utilise full posterior distributions, the simulation study can only contribute to the field of applied statistics as this methodology does not add to the theoretical understanding of Bayesian statistics. Lastly, the simulation design has explicit limitations. The final design was chosen due to preference for computational tractability and numerical stability over realism. More advanced correlation structures could have been utilised, such as DCC and less restrictive limits could have been implemented for more realistic, but less stable simulation. As a consequence, simulation was not able to capture extreme tail events. Future simulation design can expand on both with more dynamic adaptive bounds where the limits are set via relationships between variables rather than set as constants.

The results also open a few questions: What would be the performance of a TCN model if its runtime would be nearly identical to BVAR? Would BVAR still dominate? Would the results be similar if the simulation design reflected a different data paradigm such as biostatistics data? Would other Bayesian approximation methods create better uncertainties than MC Dropout? These questions can be used as a starting point of a further research in this field. Given the results of this study, another possible approach to Bayesian approximation could be explored. Recent suggestion by by Papamarkou et al. [75] suggest a pursuit of hybrid Bayesian approaches. In that spirit, the study results and modular nature of TCN, suggest possible solution where both methodologies could be combined into one hybrid approach: first use BVAR to find credible intervals from sampled data, and then leveraging the TCN's modularity to approximate found intervals with Bayesian approximation methods such as MC Dropout. Such hybrid approaches could bridge the gap between traditional Bayesian methods and Bayesian approximations for BNN. The feasibility of this approach is also a possible avenue of further research from this study.

In conclusion, while well-calibrated BVAR on well-behaving data significantly outperforms TCN, in practice TCN is better choice due to computational efficiency. Choice of a model should be on case-by-case basis as neither model clearly dominates the other across all metrics and applications.

BIBLIOGRAPHY

[1] P. Izmailov, S. Vikram, M. D. Hoffman, and A. G. Wilson, 'What are bayesian neural network posteriors really like?' *CoRR*, vol. abs/2104.14421, 2021. arXiv: 2104.14421. [Online]. Available: https://arxiv.org/abs/2104.14421

[2] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, 'Weight uncertainty in neural network,' in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., ser. Proceedings of Machine Learning Research, vol. 37, Lille, France: PMLR, Jul. 2015, pp. 1613–1622. [Online]. Available: https://proceedings.mlr.press/v37/blundell15.html

[3] P. L. McDermott and C. K. Wikle, 'Bayesian recurrent neural network models for forecasting and quantifying uncertainty in spatial-temporal data,' *Entropy*, vol. 21, no. 2, p. 184, 2019. DOI: 10.3390/e21020184 [Online]. Available: https://www.mdpi.com/1099-4300/21/2/184

[4] Y. Gal and Z. Ghahramani, 'Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,' in *International Conference on Machine Learning*, PMLR, 2016, pp. 1050–1059.

[5] B. Lakshminarayanan, A. Pritzel, and C. Blundell, 'Simple and scalable predictive uncertainty estimation using deep ensembles,' in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 6402–6413. arXiv: 1612.01474. [Online]. Available: https://dl.acm.org/doi/10.5555/3295222.3295387

[6] A. Graves, 'Practical variational inference for neural networks,' in *Advances in Neural Information Processing Systems*, J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, Eds., vol. 24, Curran Associates, Inc., 2011. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2011/file/7eb3c8be3d411e8ebfab08eba5f49632-Paper.pdf

[7] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter, *Transformers can do bayesian inference*, 2024. arXiv: 2112.10510 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2112.10510

[8] S. Bai, J. Z. Kolter, and V. Koltun, *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*, 2018. arXiv: 1803.01271 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1803.01271

[9] C. A. Sims, 'Macroeconomics and reality,' *Econometrica*, vol. 48, no. 1, pp. 1–48, 1980. Accessed: Aug. 14, 2025. [Online]. Available: http://www.jstor.org/stable/1912017

[10] R. B. Litterman, ‘Forecasting with bayesian vector autoregressions: Five years of experience,’ *Journal of Business & Economic Statistics*, vol. 4, no. 1, pp. 25–38, 1986. Accessed: Aug. 8, 2025. [Online]. Available: http://www.jstor.org/stable/1391384

[11] G. Koop and D. Korobilis, ‘Bayesian multivariate time series methods for empirical macroeconomics,’ *Foundations and Trends® in Econometrics*, vol. 3, no. 4, pp. 267–358, 2010. DOI: 10.1561/0800000013 [Online]. Available: http://dx.doi.org/10.1561/0800000013

[12] M. Bayes and M. Price, ‘An essay towards solving a problem in the doctrine of chances. by the late rev. mr. bayes, f. r. s. communicated by mr. price, in a letter to john canton, a. m. f. r. s.,’ *Philosophical Transactions (1683-1775)*, vol. 53, pp. 370–418, 1763. Accessed: Aug. 5, 2025. [Online]. Available: http://www.jstor.org/stable/105741

[13] H. Raiffa and R. Schlaifer, *Applied Statistical Decision Theory*. Boston: Division of Research, Graduate School of Business Administration, Harvard University, 1961.

[14] M. H. DeGroot, *Optimal Statistical Decisions*. New York: McGraw-Hill, 1970.

[15] H. Jeffreys, ‘An invariant form for the prior probability in estimation problems,’ *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences*, vol. 186, no. 1007, pp. 453–461, Sep. 1946. DOI: 10.1098/rspa.1946.0056

[16] A. Zellner, *An Introduction to Bayesian Inference in Econometrics*. New York: John Wiley & Sons, 1971, Wiley Classics Library Edition Published 1996.

[17] A. Zellner, ‘On assessing prior distributions and Bayesian regression analysis with g prior distributions,’ in *Bayesian Inference and Decision Techniques: Essays in Honor of Bruno de Finetti*, ser. Studies in Bayesian Econometrics and Statistics, P. K. Goel and A. Zellner, Eds., vol. 6, New York: Elsevier, 1986, pp. 233–243.

[18] J. O. Berger and J. M. Bernardo, ‘On the development of reference priors,’ in *Bayesian Statistics 4*, J. M. Bernardo, J. O. Berger, A. P. Dawid, and A. F. M. Smith, Eds., Oxford: Oxford University Press, 1992, pp. 35–60.

[19] A. Gelman, ‘Prior distributions for variance parameters in hierarchical models,’ *Bayesian Analysis*, vol. 1, Sep. 2006. DOI: 10.1214/06-BA117A

[20] G. Casella, ‘An introduction to empirical bayes data analysis,’ *The American Statistician*, vol. 39, no. 2, pp. 83–87, 1985. Accessed: Aug. 8, 2025. [Online]. Available: http://www.jstor.org/stable/2682801

[21] C. N. Morris, ‘Parametric empirical bayes inference: Theory and applications,’ *Journal of the American Statistical Association*, vol. 78, no. 381, pp. 47–55, 1983. DOI: 10.1080/01621459.1983.10477920

[22] T. J. Mitchell and J. J. Beauchamp, 'Bayesian variable selection in linear regression,' *Journal of the American Statistical Association*, vol. 83, no. 404, pp. 1023–1032, 1988. Accessed: Aug. 8, 2025. [Online]. Available: http://www.jstor.org/stable/2290129

[23] C. M. Carvalho, N. G. Polson, and J. G. Scott, 'The horseshoe estimator for sparse signals,' *Biometrika*, vol. 97, no. 2, pp. 465–480, 2010. Accessed: Aug. 8, 2025. [Online]. Available: http://www.jstor.org/stable/25734098

[24] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, 'Equation of state calculations by fast computing machines,' *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, Jun. 1953. DOI: 10.1063/1.1699114 eprint: https://pubs.aip.org/aip/jcp/article-pdf/21/6/1087/18802390/1087_1_online.pdf. [Online]. Available: https://doi.org/10.1063/1.1699114

[25] S. Geman and D. Geman, 'Stochastic relaxation, gibbs distributions, and the bayesian restoration of images,' *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, pp. 721–741, 1984. [Online]. Available: https://api.semanticscholar.org/CorpusID:5837272

[26] A. N. Kolmogorov, *Foundations of the Theory of Probability*, 2nd ed. New York: Chelsea Publishing Company, 1956, Translation edited by Nathan Morrison, with added bibliography by A. T. Bharucha-Reid.

[27] G. U. Yule, 'On a method of investigating periodicities in disturbed series, with special reference to wolfer's sunspot numbers,' *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 226, pp. 267–298, 1927. Accessed: Jan. 28, 2026. [Online]. Available: http://www.jstor.org/stable/91170

[28] G. T. Walker, 'On periodicity in series of related terms,' *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, vol. 131, no. 818, pp. 518–532, Jun. 1931. DOI: 10.1098/rspa.1931.0069 eprint: https://royalsocietypublishing.org/rspa/article-pdf/131/818/518/26784/rspa.1931.0069.pdf. [Online]. Available: https://doi.org/10.1098/rspa.1931.0069

[29] H. O. A. Wold, *A Study in the Analysis of Stationary Time Series*. Uppsala, Sweden: Almqvist & Wiksells Boktryckeri, 1938.

[30] G. E. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco, USA: Holden-Day, 1970.

[31] T. Doan, R. B. Litterman, and C. A. Sims, 'Forecasting and conditional projection using realistic prior distributions,' National Bureau of Economic Research, Working Paper 1202, Sep. 1983. DOI: 10.3386/w1202 [Online]. Available: http://www.nber.org/papers/w1202

[32] D. Lewandowski, D. Kurowicka, and H. Joe, 'Generating random correlation matrices based on vines and extended onion method,' *Journal of Multivariate Analysis*, vol. 100, no. 9, pp. 1989–2001, 2009. DOI: https://doi.org/10.1016/j.jmva.2009.04.008 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0047259X09000876

[33] W. K. Hastings, 'Monte carlo sampling methods using markov chains and their applications,' *Biometrika*, vol. 57, no. 1, pp. 97–109, 1970. Accessed: Aug. 15, 2025. [Online]. Available: http://www.jstor.org/stable/2334940

[34] A. E. Gelfand and A. F. M. Smith, 'Sampling-based approaches to calculating marginal densities,' *Journal of the American Statistical Association*, vol. 85, no. 410, pp. 398–409, 1990. Accessed: Aug. 18, 2025. [Online]. Available: http://www.jstor.org/stable/2289776

[35] S. Brooks, A. Gelman, G. Jones, and X.-L. Meng, *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, May 2011. DOI: 10.1201/b10905 [Online]. Available: http://dx.doi.org/10.1201/b10905

[36] M. D. Hoffman and A. Gelman, 'The no-u-turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo,' *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1593–1623, Jan. 2014. DOI: 10.5555/2627435.2638586

[37] K. R. Kadiyala and S. Karlsson, 'Numerical methods for estimation and inference in Bayesian VAR-models,' *Journal of Applied Econometrics*, vol. 12, no. 2, pp. 99–132, 1997. DOI: https://doi.org/10.1002/(SICI)1099-1255(199703)12:2<99::AID-JAE429>3.0.CO;2-A eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/%28SICI%291099-1255%28199703%2912%3A2%3C99%3A%3AAID-JAE429%3E3.0.CO%3B2-A. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291099-1255%28199703%2912%3A2%3C99%3A%3AAID-JAE429%3E3.0.CO%3B2-A

[38] C. A. Sims and T. Zha, 'Bayesian methods for dynamic multivariate models,' *International Economic Review*, vol. 39, no. 4, pp. 949–968, 1998. Accessed: Jan. 28, 2026. [Online]. Available: http://www.jstor.org/stable/2527347

[39] D. Giannone, M. Lenza, and G. E. Primiceri, 'Prior selection for vector autoregressions,' *The Review of Economics and Statistics*, vol. 97, no. 2, pp. 436–451, 2015. Accessed: Jan. 28, 2026. [Online]. Available: http://www.jstor.org/stable/43556184

[40] W. Zhu, J. M. Marin, and F. Leisen, 'A bootstrap likelihood approach to Bayesian computation,' *Australian & New Zealand Journal of Statistics*, vol. 58, no. 2, pp. 227–244, 2016.

[41] P. d. Z. Bermudez, J. M. Marín, H. Rue, and H. Veiga, 'Integrated nested Laplace approximations for threshold stochastic volatility models,' *Econometrics and Statistics*, vol. 30, no. C, pp. 15–35, 2024. DOI: 10.1016/j.ecosta.2021.08.006 [Online]. Available: https://ideas.repec.org/a/eee/ecosta/v30y2024icp15-35.html

[42] W. S. McCulloch and W. Pitts, 'A logical calculus of the ideas immanent in nervous activity,' *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, 1943. DOI: 10.1007/BF02478259 [Online]. Available: https://doi.org/10.1007/BF02478259

[43] F. Rosenblatt, 'The perceptron: A probabilistic model for information storage and organization in the brain,' *Psychol. Review*, vol. 65, pp. 386–408, Nov. 1958. DOI: 10.1037/h0042519

[44] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*, 1st. Cambridge, MA: MIT Press, 1969.

[45] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, 'Learning representations by back-propagating errors,' *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. DOI: 10.1038/323533a0 [Online]. Available: https://doi.org/10.1038/323533a0

[46] J. S. Bridle, 'Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters,' in *Advances in Neural Information Processing Systems 2*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1990, pp. 211–217.

[47] V. Nair and G. E. Hinton, 'Rectified linear units improve restricted Boltzmann machines,' in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ser. ICML'10, Haifa, Israel: Omnipress, 2010, pp. 807–814.

[48] J. L. Elman, 'Finding structure in time,' *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. DOI: https://doi.org/10.1016/0364-0213(90)90002-E [Online]. Available: https://www.sciencedirect.com/science/article/pii/036402139090002E

[49] R. Pascanu, T. Mikolov, and Y. Bengio, 'On the difficulty of training recurrent neural networks,' in *Proceedings of the 30th International Conference on Machine Learning*, S. Dasgupta and D. McAllester, Eds., ser. Proceedings of Machine Learning Research, vol. 28, Atlanta, Georgia, USA: PMLR, Jun. 2013, pp. 1310–1318. [Online]. Available: https://proceedings.mlr.press/v28/pascanu13.html

[50] S. Hochreiter and J. Schmidhuber, 'Long short-term memory,' *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. DOI: 10.1162/neco.1997.9.8.1735 eprint: https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[51] K. Cho et al., 'Learning phrase representations using RNN encoder–decoder for statistical machine translation,' in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds., Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1724–1734. DOI: 10.3115/v1/D14-1179 [Online]. Available: https://aclanthology.org/D14-1179/

[52] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-based learning applied to document recognition,' *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791

[53] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, 'Dropout: A simple way to prevent neural networks from overfitting,' *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html

[54] D. Luo and X. Wang, 'ModernTCN: A modern pure convolution structure for general time series analysis,' in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=vpJMJerXHU

[55] D. J. C. MacKay, 'A practical bayesian framework for backpropagation networks,' *Neural Computation*, vol. 4, no. 3, pp. 448–472, 1992. DOI: 10.1162/neco.1992.4.3.448

[56] R. M. Neal, *Bayesian Learning for Neural Networks* (Lecture Notes in Statistics). New York, NY: Springer, 1996, vol. 118, p. 204. DOI: 10.1007/978-1-4612-0745-0

[57] B. Lakshminarayanan, A. Pritzel, and C. Blundell, *Simple and scalable predictive uncertainty estimation using deep ensembles*, 2017. arXiv: 1612.01474 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1612.01474

[58] Y. Gal and Z. Ghahramani, *Dropout as a bayesian approximation: Appendix*, 2016. arXiv: 1506.02157 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1506.02157

[59] A. Garza, C. Challu, and M. Mergenthaler-Canseco, *Timegpt-1*, 2024. arXiv: 2310.03589 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2310.03589

[60] A. F. Ansari et al., *Chronos: Learning the language of time series*, 2024. arXiv: 2403.07815 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2403.07815

[61] A. Vaswani et al., 'Attention is all you need,' in *Advances in Neural Information Processing Systems*, I. Guyon et al., Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[62] H. Zhou et al., 'Informer: Beyond efficient transformer for long sequence time-series forecasting,' *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 11 106–11 115, May 2021. DOI: 10.1609/aaai.v35i12.17325 [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/17325

[63] A. Gu and T. Dao, *Mamba: Linear-time sequence modeling with selective state spaces*, 2024. arXiv: 2312.00752 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2312.00752

[64] S. Wang et al., *Timemixer: Decomposable multiscale mixing for time series forecasting*, 2024. arXiv: 2405.14616 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2405.14616

[65] S. J. Taylor, *Modelling Financial Time Series*. World Scientific Publishing Co. Pte. Ltd., 2007. [Online]. Available: https://EconPapers.repec.org/RePEc:wsi:wsbook:6578

[66] R. C. Merton, 'Option pricing when underlying stock returns are discontinuous,' *Journal of Financial Economics*, vol. 3, no. 1, pp. 125–144, 1976. DOI: https://doi.org/10.1016/0304-405X(76)90022-2 [Online]. Available: https://www.sciencedirect.com/science/article/pii/0304405X76900222

[67] R. Engle, 'Dynamic conditional correlation: A simple class of multivariate generalized autoregressive conditional heteroskedasticity models,' *Journal of Business & Economic Statistics*, vol. 20, no. 3, pp. 339–350, 2002. Accessed: Aug. 27, 2025. [Online]. Available: http://www.jstor.org/stable/1392121

[68] Y. Aït-Sahalia, J. Cacho-Diaz, and R. J. Laeven, 'Modeling financial contagion using mutually exciting jump processes,' National Bureau of Economic Research, Inc, NBER Working Papers 15850, Mar. 2010. [Online]. Available: https://ideas.repec.org/p/nbr/nberwo/15850.html

[69] S. Chib, F. Nardari, and N. Shephard, 'Analysis of high dimensional multivariate stochastic volatility models,' *Journal of Econometrics*, vol. 134, no. 2, pp. 341–371, 2006. DOI: https://doi.org/10.1016/j.jeconom.2005.06.026 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0304407605001478

[70] R. Tsay, *Analysis of Financial Time Series*, 3rd. Hoboken, NJ: Wiley & Sons, 2010. DOI: 10.1002/0471746193

[71] F. Canova, *Methods for Applied Macroeconomic Research*. Princeton University Press, 2007. Accessed: Jan. 12, 2026. [Online]. Available: http://www.jstor.org/stable/j.ctvcm4hrv

[72] L. Kilian and H. Lütkepohl, *Structural Vector Autoregressive Analysis* (Themes in Modern Econometrics). Cambridge University Press, 2017.

[73] J. Salvatier, T. V. Wiecki, and C. Fonnesbeck, 'Probabilistic programming in Python using PyMC3,' *PeerJ Computer Science*, vol. 2, no. 55, 2016. DOI: 10.7717/peerj-cs.55

[74] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[75] T. Papamarkou et al., *Position: Bayesian deep learning is needed in the age of large-scale ai*, 2024. arXiv: 2402.00809 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2402.00809

APPENDIX A. TCN+MC DROPOUT ARCHITECTURE MAPPING FOR GRID SEARCH

Config	Layers	Channels	Dropout
1	2	16	0.1
2	2	16	0.2
3	2	16	0.3
4	2	32	0.1
5	2	32	0.2
6	2	32	0.3
7	2	64	0.1
8	2	64	0.2
9	2	64	0.3
10	3	16	0.1
11	3	16	0.2
12	3	16	0.3
13	3	32	0.1
14	3	32	0.2
15	3	32	0.3
16	**3**	**64**	**0.1**
17	3	64	0.2
18	3	64	0.3
19	4	16	0.1
20	4	16	0.2
21	4	16	0.3
22	4	32	0.1
23	4	32	0.2
24	4	32	0.3
25	4	64	0.1
26	4	64	0.2
27	4	64	0.3

Note: Red colour represents the chosen model.

TABLE A.1. TCN CONFIGURATION ARCHITECTURE MAPPING

APPENDIX B. FORECASTING TEST RESULTS

B.1. Results for Excitement Level 0.25

Model	Data Tag	Runtime (sec)	RMSE (mean)	RMSE (std)	MAE (mean)	MAE (std)	RMSE/ MAE
BVAR Lag 5	0;100;0(100)	16165.32	2.38321	1.23815	2.03291	1.11624	1.17231
Model 16	na	1826.41	3.32101	1.99854	3.10156	2.03724	1.07075

TABLE B.1. OVERALL PERFORMANCE METRICS
(EXCITEMENT = 0.25)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Coverage 80%				
Variable 1	0.79800	0.24941	0.95889	0.14782
Variable 2	0.79842	0.23019	0.98641	0.04555
Variable 3	0.82018	0.21368	0.98196	0.08409
Coverage 90%				
Variable 1	0.90218	0.17212	0.98600	0.06767
Variable 2	0.90413	0.17875	0.99911	0.00409
Variable 3	0.91880	0.15635	0.99734	0.01622
Coverage 95%				
Variable 1	0.95599	0.10881	0.99478	0.03059
Variable 2	0.95477	0.13219	0.99994	0.00035
Variable 3	0.96220	0.11671	0.99969	0.00199

TABLE B.2. PREDICTION INTERVAL COVERAGE RATES
(EXCITEMENT = 0.25)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Width 80%				
Variable 1	6.59724	0.33923	18.41841	1.65452
Variable 2	6.57931	0.35699	18.31516	1.69149
Variable 3	6.51717	0.30856	18.20723	1.68737
Width 90%				
Variable 1	8.47484	0.43567	23.35376	2.09313
Variable 2	8.44876	0.45609	23.21735	2.13221
Variable 3	8.36792	0.39992	23.08502	2.13667
Width 95%				
Variable 1	10.10192	0.51838	27.38825	2.45698
Variable 2	10.07030	0.54938	27.22826	2.49889
Variable 3	9.97950	0.48020	27.07317	2.50494

TABLE B.3. PREDICTION INTERVAL WIDTHS (EXCITEMENT = 0.25)

	RMSE (mean)			RMSE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	2.41610	2.45181	2.28174	1.24846	1.32814	1.13787
Model 16	3.55403	3.08003	3.32896	2.24748	1.87671	1.87143

TABLE B.4. ROOT MEAN SQUARED ERROR BY VARIABLE (EXCITEMENT = 0.25)

	MAE (mean)			MAE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	2.05877	2.08867	1.95131	1.13529	1.18550	1.02793
Model 16	3.34248	2.84693	3.11528	2.28424	1.91160	1.91587

TABLE B.5. MEAN ABSOLUTE ERROR BY VARIABLE (EXCITEMENT = 0.25)

B.2. Results for Excitement Level 0.50

Model	Data Tag	Runtime (sec)	RMSE (mean)	RMSE (std)	MAE (mean)	MAE (std)	RMSE/ MAE
BVAR Lag 5	0;100;0(100)	16425.75	2.45920	1.45677	2.09803	1.30716	1.17215
Model 16	na	1820.39	3.14020	1.77156	2.91067	1.80984	1.07886

TABLE B.6. OVERALL PERFORMANCE METRICS
(EXCITEMENT = 0.50)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Coverage 80%				
Variable 1	0.82309	0.23760	0.98549	0.06457
Variable 2	0.79003	0.25934	0.97795	0.10723
Variable 3	0.78414	0.29369	0.99302	0.03570
Coverage 90%				
Variable 1	0.91585	0.16604	0.99764	0.01443
Variable 2	0.88376	0.21118	0.99566	0.02801
Variable 3	0.87436	0.22956	0.99944	0.00328
Coverage 95%				
Variable 1	0.96036	0.10294	0.99962	0.00260
Variable 2	0.93696	0.15213	0.99945	0.00422
Variable 3	0.93174	0.16675	0.99997	0.00019

TABLE B.7. PREDICTION INTERVAL COVERAGE RATES
(EXCITEMENT = 0.50)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Width 80%				
Variable 1	6.61651	0.34432	18.41375	1.70789
Variable 2	6.68786	0.37032	18.43544	1.73989
Variable 3	6.63771	0.29902	18.41039	1.63550
Width 90%				
Variable 1	8.49436	0.44312	23.34124	2.16157
Variable 2	8.58296	0.47528	23.36625	2.20074
Variable 3	8.52369	0.38688	23.33341	2.06933
Width 95%				
Variable 1	10.12482	0.53061	27.36983	2.53877
Variable 2	10.22513	0.56606	27.40073	2.58284
Variable 3	10.15700	0.46391	27.36117	2.43166

TABLE B.8. PREDICTION INTERVAL WIDTHS (EXCITEMENT = 0.50)

	RMSE (mean)			RMSE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	2.32963	2.52526	2.52270	1.23073	1.52342	1.61616
Model 16	3.37197	3.18369	2.86493	1.78806	1.96122	1.56540

TABLE B.9. ROOT MEAN SQUARED ERROR BY VARIABLE (EXCITEMENT = 0.50)

	MAE (mean)			MAE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	1.98343	2.15172	2.15893	1.09438	1.37087	1.45624
Model 16	3.15420	2.95037	2.62743	1.83237	2.00499	1.59217

TABLE B.10. MEAN ABSOLUTE ERROR BY VARIABLE (EXCITEMENT = 0.50)

B.3. Forecasting Test Results for Excitement Level 0.75

Model	Data Tag	Runtime (sec)	RMSE (mean)	RMSE (std)	MAE (mean)	MAE (std)	RMSE/ MAE
BVAR Lag 5	0;100;0(100)	13024.58	2.40465	1.26195	2.05994	1.16572	1.16734
Model 16	na	1401.18	3.25047	2.12591	3.02875	2.17251	1.07321

TABLE B.11. OVERALL PERFORMANCE METRICS
(EXCITEMENT = 0.75)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Coverage 80%				
Variable 1	0.82090	0.23361	0.97461	0.10238
Variable 2	0.82246	0.25014	0.97934	0.10654
Variable 3	0.79402	0.24958	0.97856	0.09740
Coverage 90%				
Variable 1	0.91646	0.16595	0.99517	0.03937
Variable 2	0.90755	0.19349	0.99045	0.08792
Variable 3	0.90241	0.17807	0.99691	0.02055
Coverage 95%				
Variable 1	0.95920	0.10694	0.99896	0.00984
Variable 2	0.95090	0.13025	0.99506	0.04888
Variable 3	0.95123	0.13069	0.99966	0.00246

TABLE B.12. PREDICTION INTERVAL COVERAGE RATES
(EXCITEMENT = 0.75)

	BVAR Lag 5		Model 16	
	Mean	SD	Mean	SD
Width 80%				
Variable 1	6.88480	0.39573	18.81218	1.89698
Variable 2	6.83251	0.35558	18.71136	1.78269
Variable 3	6.80006	0.37826	18.58399	1.63511
Width 90%				
Variable 1	8.83433	0.50253	23.85040	2.41577
Variable 2	8.77154	0.45525	23.72360	2.26890
Variable 3	8.72638	0.48543	23.56230	2.08014
Width 95%				
Variable 1	10.52779	0.59867	27.97552	2.83655
Variable 2	10.45461	0.54280	27.82805	2.66574
Variable 3	10.39777	0.57864	27.63622	2.44704

TABLE B.13. PREDICTION INTERVAL WIDTHS (EXCITEMENT = 0.75)

	RMSE (mean)			RMSE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	2.39858	2.40304	2.41232	1.19328	1.30682	1.28575
Model 16	3.38278	3.06457	3.30408	2.20354	2.09318	2.08100

TABLE B.14. ROOT MEAN SQUARED ERROR BY VARIABLE (EXCITEMENT = 0.75)

	MAE (mean)			MAE (std)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
BVAR Lag 5	2.06215	2.05050	2.06718	1.09627	1.21964	1.18124
Model 16	3.16586	2.83550	3.08488	2.25056	2.13514	2.13184

TABLE B.15. MEAN ABSOLUTE ERROR BY VARIABLE (EXCITEMENT = 0.75)

APPENDIX C. PRETRAINING DATASET VISUALISATIONS



Fig. C.1. Pretraining Dataset: Excitement 0.25 Dataset 1



Fig. C.2. Pretraining Dataset: Excitement 0.25 Dataset 2



Fig. C.3. Pretraining Dataset: Excitement 0.25 Dataset 3

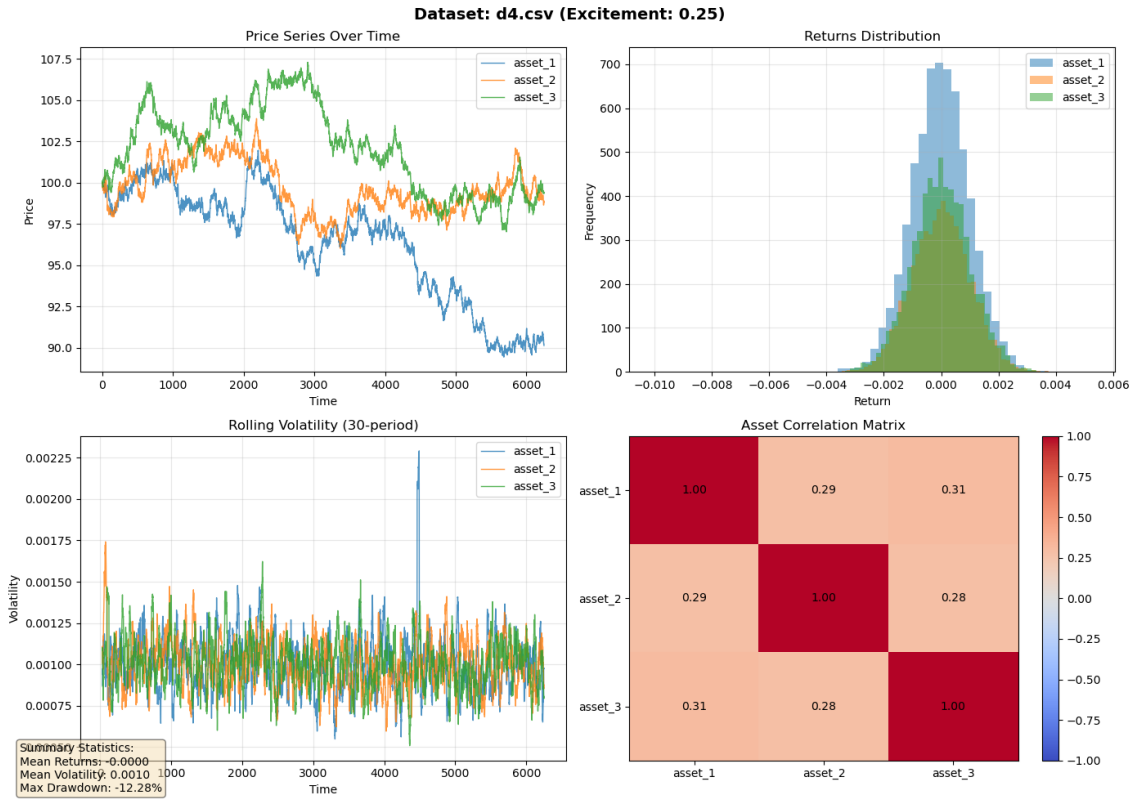


Fig. C.4. Pretraining Dataset: Excitement 0.25 Dataset 4

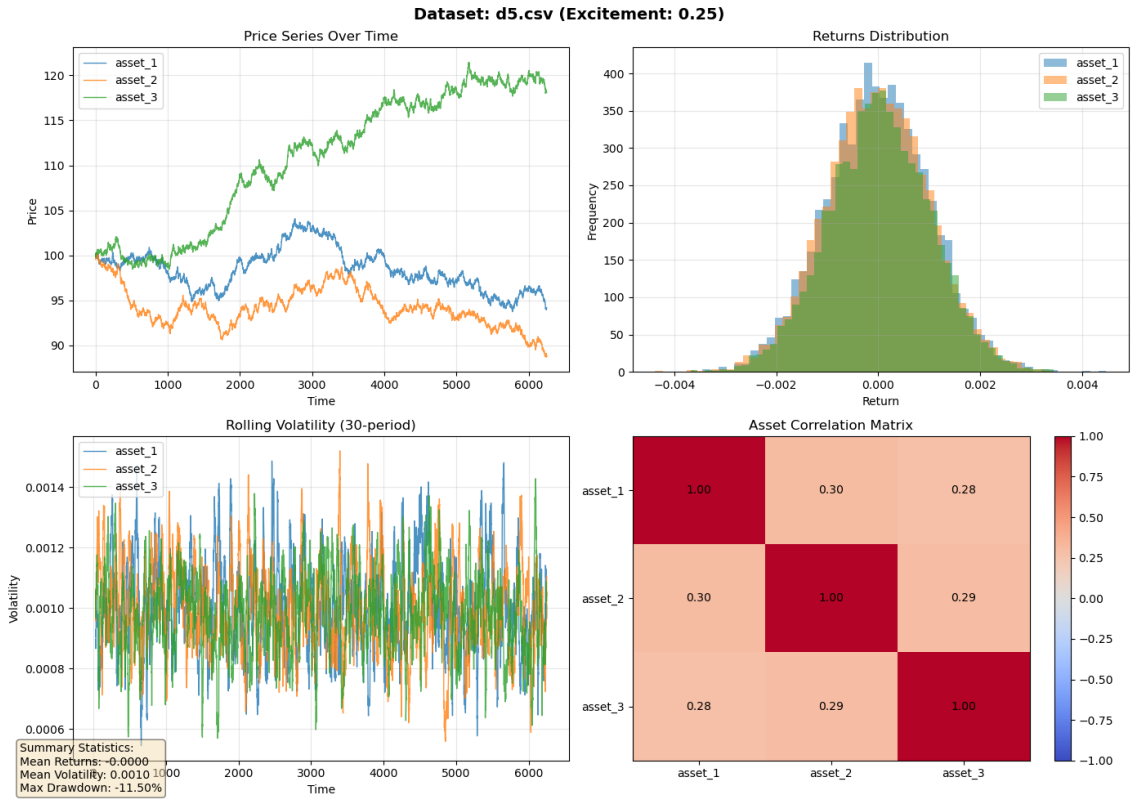


Fig. C.5. Pretraining Dataset: Excitement 0.25 Dataset 5

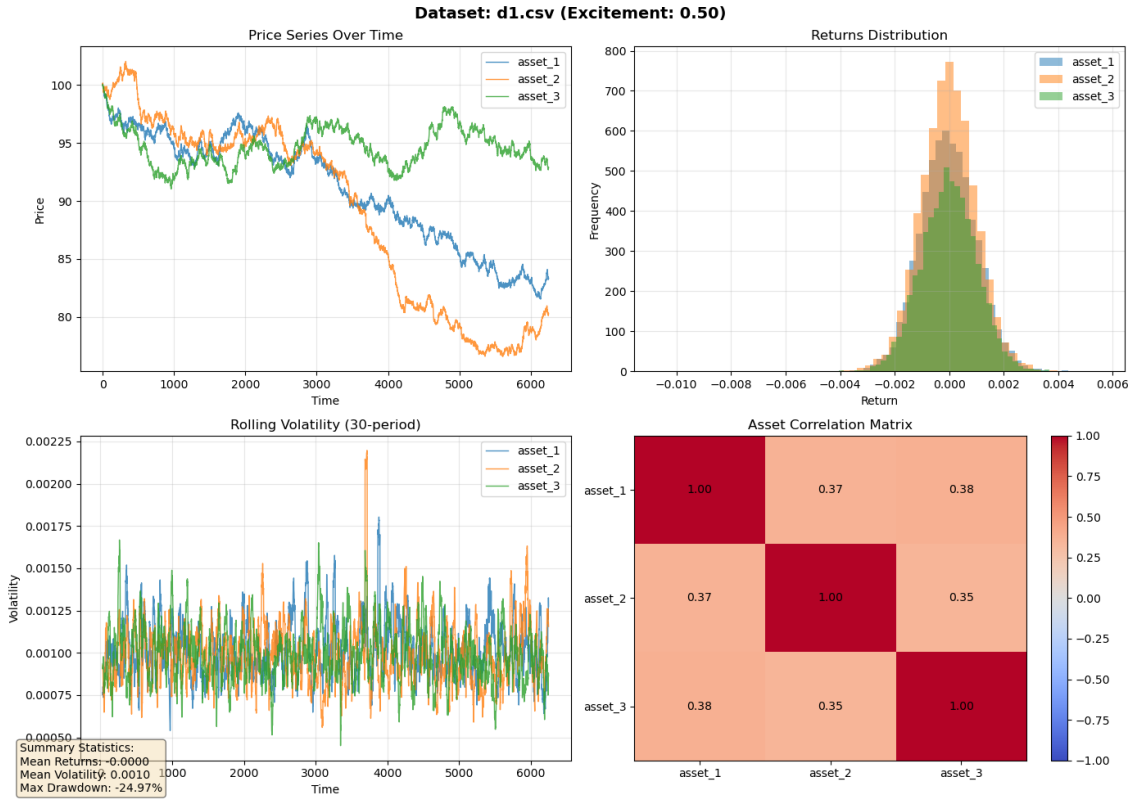


Fig. C.6. Pretraining Dataset: Excitement 0.50 Dataset 1

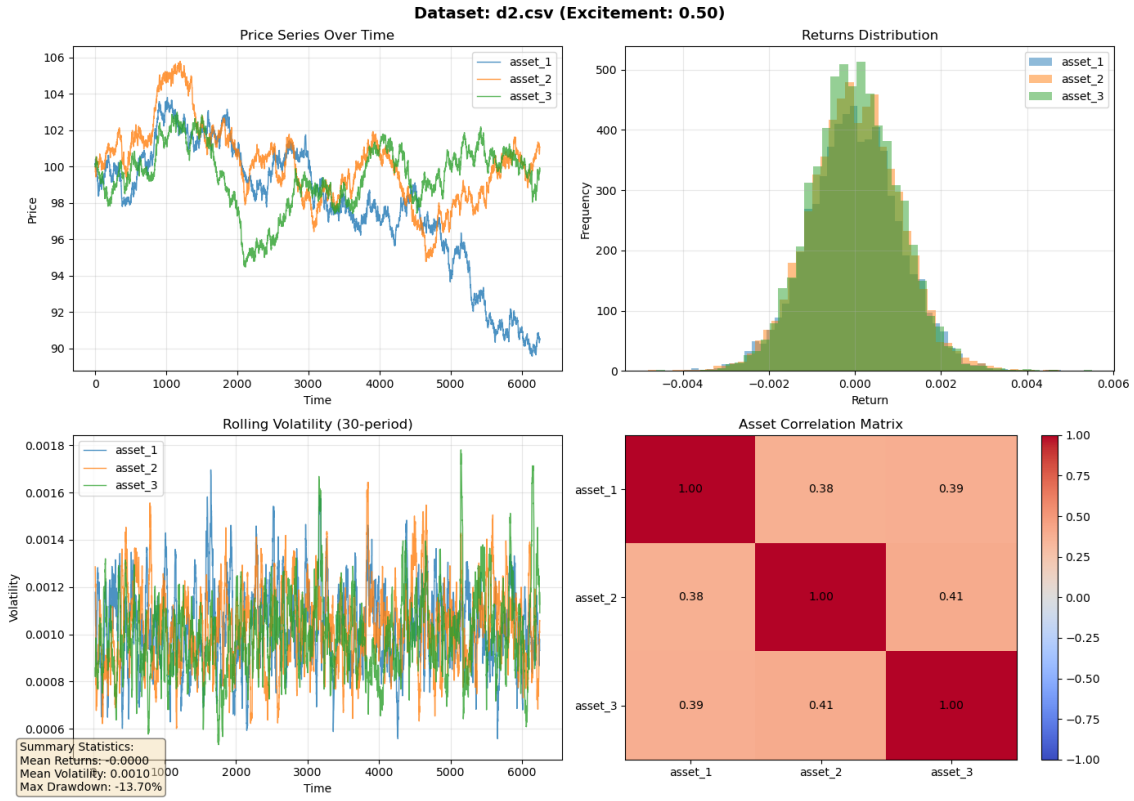


Fig. C.7. Pretraining Dataset: Excitement 0.50 Dataset 2

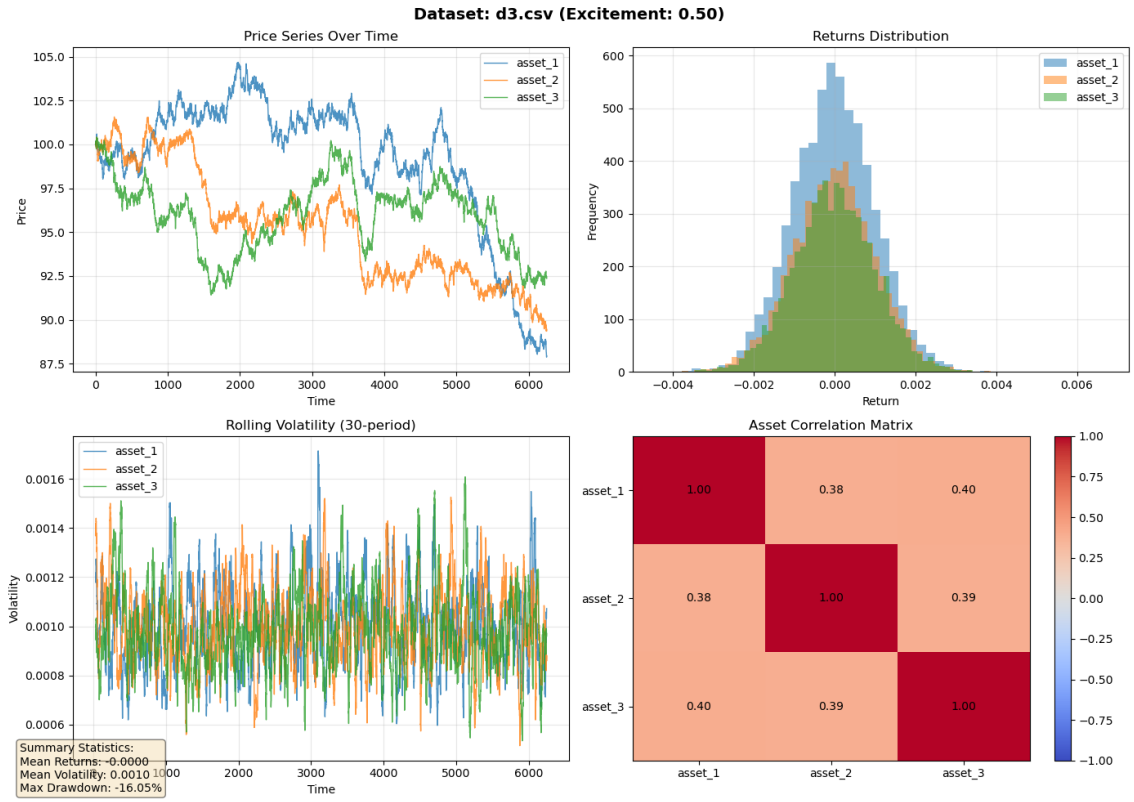


Fig. C.8. Pretraining Dataset: Excitement 0.50 Dataset 3

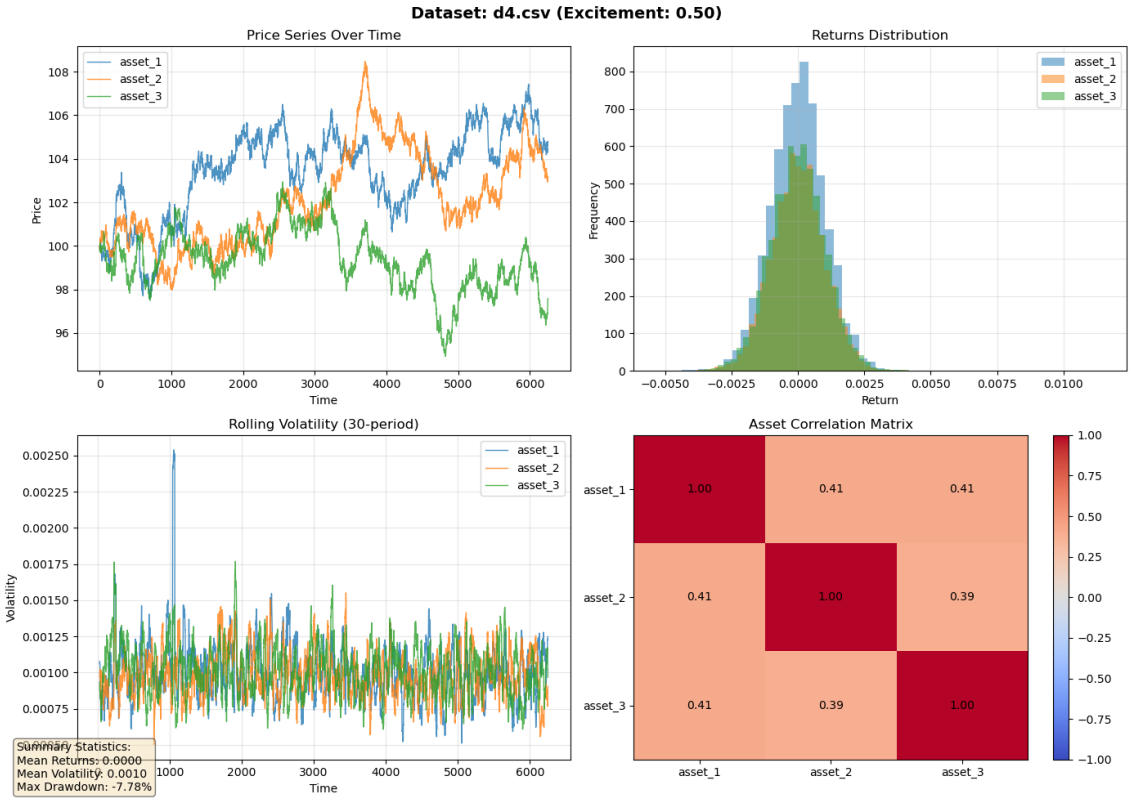


Fig. C.9. Pretraining Dataset: Excitement 0.50 Dataset 4

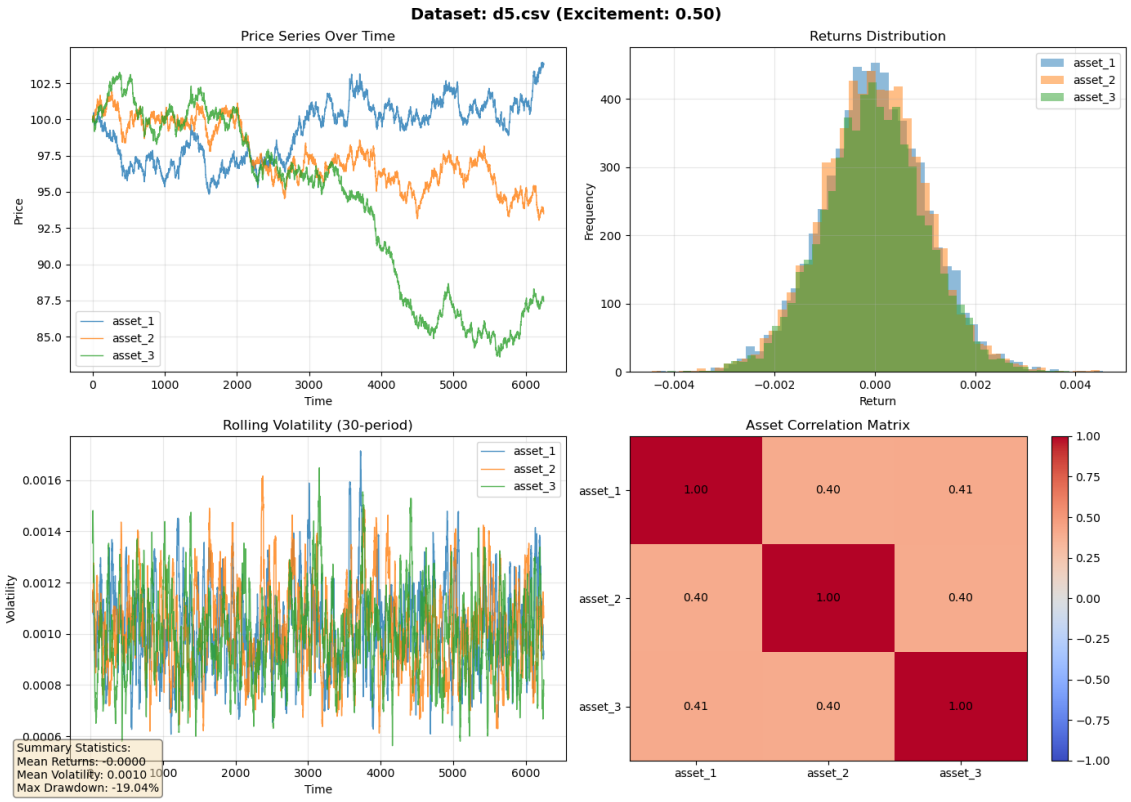


Fig. C.10. Pretraining Dataset: Excitement 0.50 Dataset 5

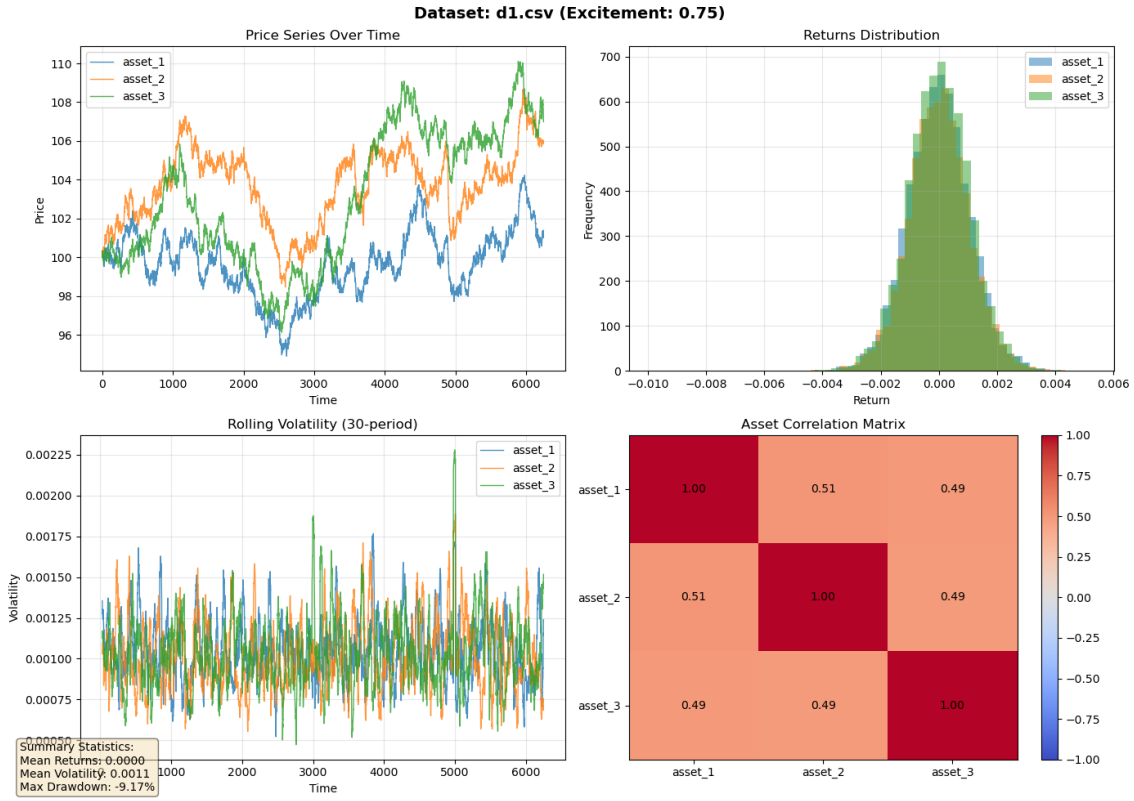


Fig. C.11. Pretraining Dataset: Excitement 0.75 Dataset 1

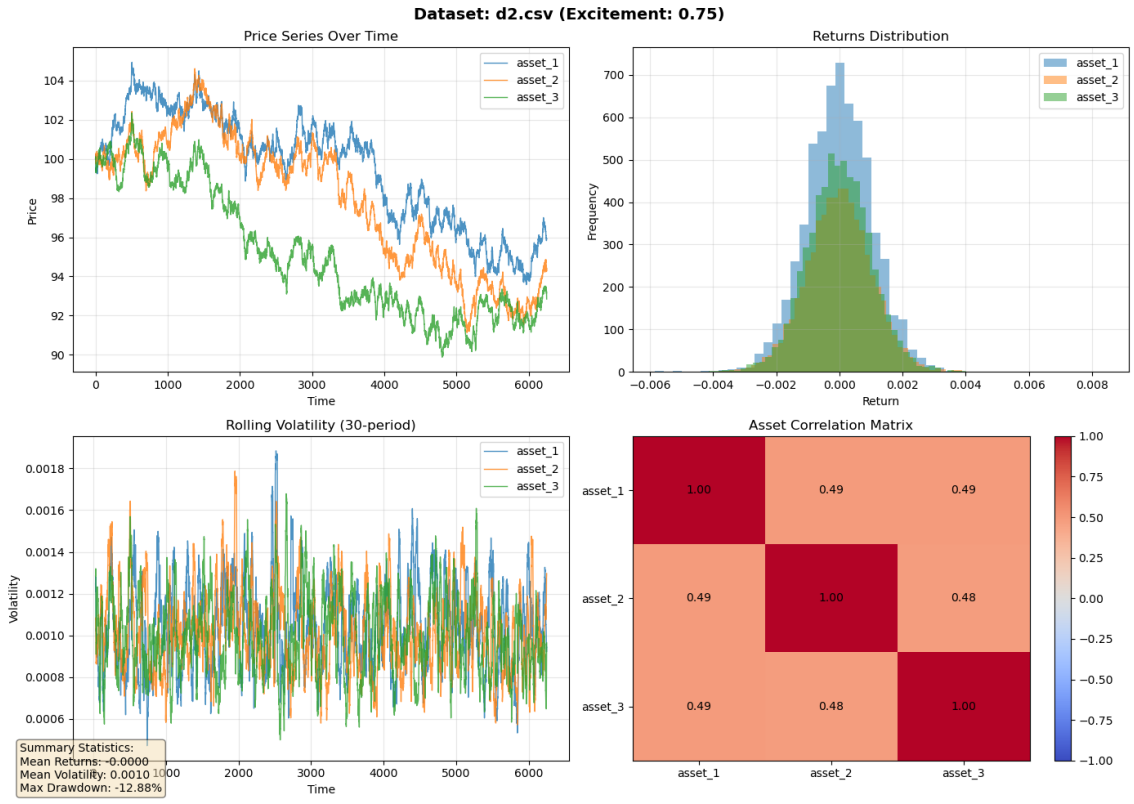


Fig. C.12. Pretraining Dataset: Excitement 0.75 Dataset 2

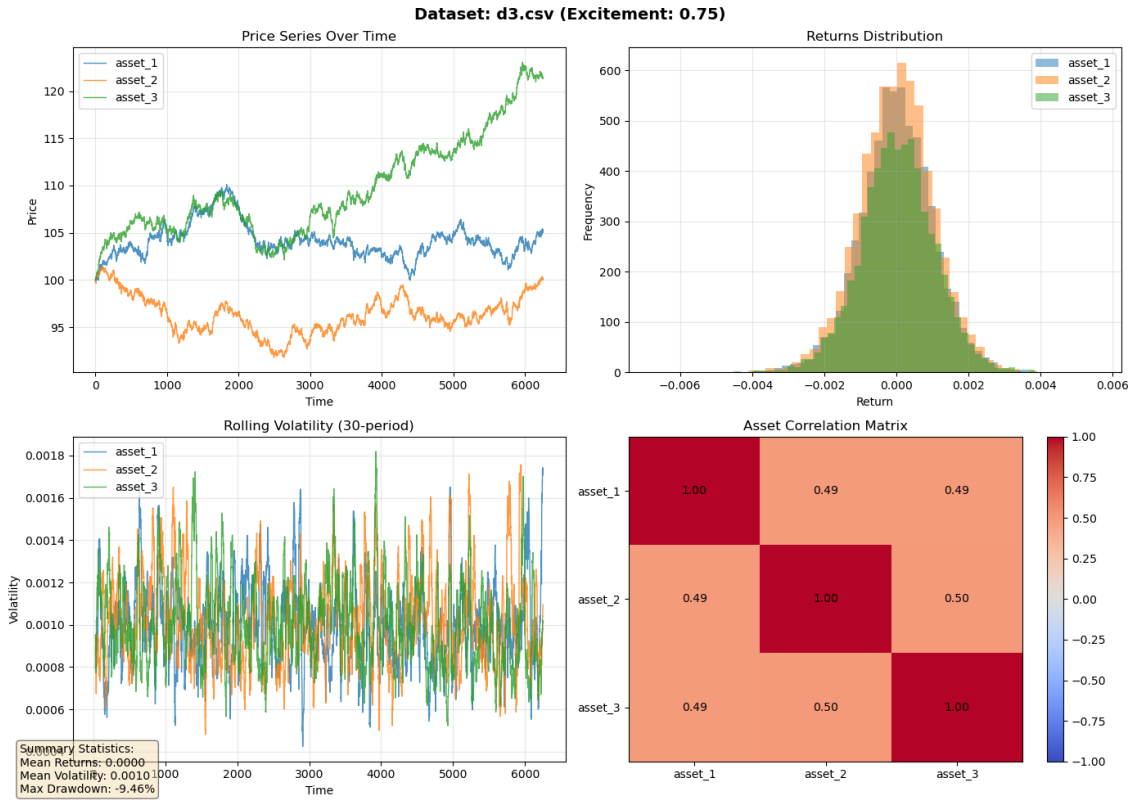


Fig. C.13. Pretraining Dataset: Excitement 0.75 Dataset 3

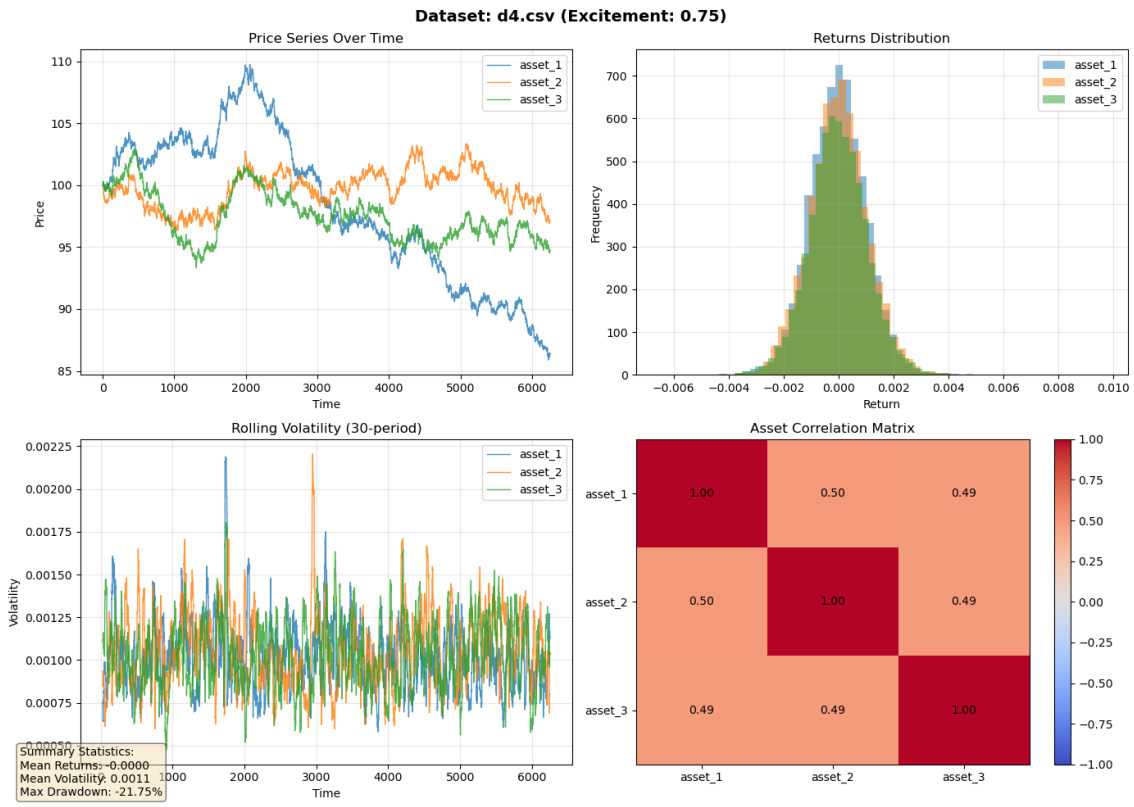


Fig. C.14. Pretraining Dataset: Excitement 0.75 Dataset 4

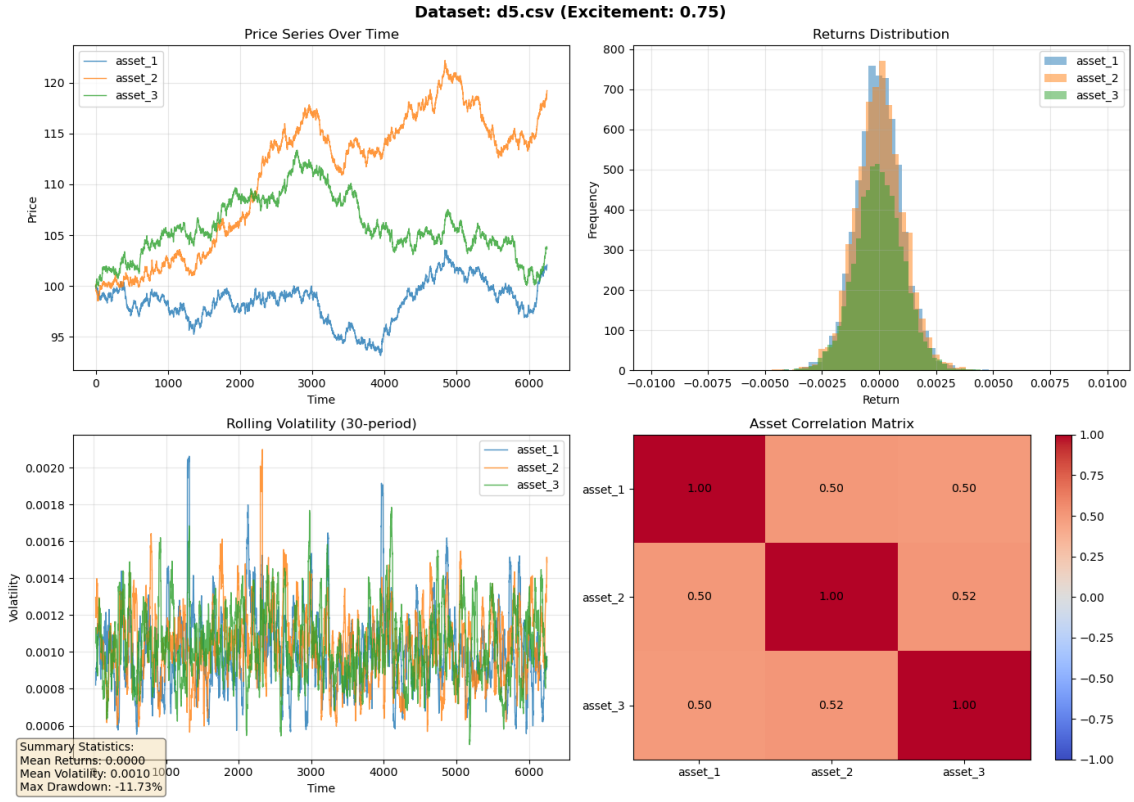


Fig. C.15. Pretraining Dataset: Excitement 0.75 Dataset 5

APPENDIX D. PRETRAINING RESULTS

D.1. BVAR

Lag	DIC (mean)	Runtime (s)	RMSE (mean)	RMSE (std)	MAE (mean)	MAE (std)	RMSE/ MAE
Lag 3	**-27607.22**	**1715.00**	2.17540	1.29327	1.82588	1.15833	1.19143
Lag 4	-27594.72	2385.48	2.17642	1.29119	1.82801	1.15742	1.19060
Lag 5	-27586.54	2461.41	2.17161	**1.29049**	1.82272	**1.15617**	1.19142
Lag 6	-27578.24	2652.43	2.17349	1.30058	1.82587	1.16588	**1.19039**
Lag 7	-27570.21	2779.21	**2.16677**	1.29292	**1.81968**	1.15747	1.19074
Lag 8	-27562.01	3048.64	2.17572	1.29530	1.82732	1.16113	1.19066
Lag 9	-27557.44	3171.03	2.17412	1.29190	1.82553	1.15781	1.19095

Note: Bold values indicate best performance. *Distribution of differencing orders:* $d = 0$ (0 datasets); $d = 1$ (15 datasets); $d = 2$ (0 datasets). All metrics are overall means and standard deviations computed across all three variables and 15 pretraining datasets. Runtime is total time for all 15 datasets.

TABLE D.1. BVAR PRETRAINING RESULTS: MODEL SELECTION (OVERALL METRICS)

Lag	Coverage 95% (mean)	Coverage 95% (std)	Width 95% (mean)	Width 95% (std)
Lag 3	0.97266	0.10284	10.3265	0.4981
Lag 4	0.97275	0.10236	10.3090	**0.4724**
Lag 5	**0.97193**	0.10447	10.2915	0.4830
Lag 6	0.97223	0.10384	10.2701	0.4956
Lag 7	0.97234	0.10301	10.2595	0.5056
Lag 8	0.97239	0.10237	**10.2539**	0.5417
Lag 9	0.97275	**0.10000**	10.2620	0.5606

Note: Bold values indicate best performance. Coverage and width are computed for 95% prediction intervals. Mean and standard deviation are calculated across all variables and 15 pretraining datasets (overall metrics). Best coverage is the one closest to 0.95.

TABLE D.2. BVAR PRETRAINING RESULTS: UNCERTAINTY QUANTIFICATION (OVERALL METRICS)

D.2. TCN+MC Dropout

Lag	RMSE (mean)			MAE (mean)		
	Var 1	**Var 2**	**Var 3**	**Var 1**	**Var 2**	**Var 3**
Lag 3	1.99330	2.60769	1.92521	1.66871	2.22362	1.58531
Lag 4	1.99539	2.61181	1.92207	1.67041	2.22911	1.58451
Lag 5	1.98531	2.60441	1.92513	**1.66209**	2.22075	1.58531
Lag 6	1.99039	2.60787	1.92221	1.66920	2.22502	1.58339
Lag 7	1.98740	**2.59486**	**1.91805**	1.66351	**2.21667**	**1.57887**
Lag 8	**1.98474**	2.61639	1.92603	1.66292	2.23287	1.58617
Lag 9	1.98493	2.61757	1.91986	1.66300	2.23290	1.58069

Note: Bold values indicate best performance. Metrics are means across 15 pretraining datasets. No standard deviations shown for per-variable metrics.

TABLE D.3. BVAR PRETRAINING RESULTS: PER-VARIABLE PERFORMANCE (MEANS)

Config	Runtime (s)	RMSE (mean)	RMSE (std)	MAE (mean)	MAE (std)	RMSE/MAE
1	193.73	4.09786	2.30494	3.80705	2.36361	1.07639
2	194.97	5.80523	3.35462	5.45408	3.44815	1.06438
3	224.87	5.84735	2.96636	5.37858	3.04961	1.08715
4	194.33	3.80145	2.33372	3.55024	2.37159	1.07076
5	209.33	4.72391	3.05007	4.43117	3.11600	1.06606
6	231.04	4.45240	2.32259	4.05620	2.36500	1.09768
7	200.62	3.18359	**1.71393**	2.97384	**1.74846**	1.07053
8	**192.79**	3.63859	1.88588	3.36837	1.90903	1.08023
9	236.22	4.01624	2.44494	3.70985	2.51129	1.08259
10	253.26	4.44902	2.59694	4.15573	2.67119	1.07057
11	277.40	4.60278	2.35810	4.19255	2.39389	1.09785
12	298.13	6.16593	3.16362	5.72563	3.25206	1.07690
13	273.19	4.06460	2.11068	3.82390	2.18054	1.06295
14	253.52	4.28837	2.66629	3.96991	2.73890	1.08022
15	301.57	4.92293	2.62535	4.57067	2.70631	1.07707
16	262.44	**2.92939**	1.78259	**2.70131**	1.80823	1.08443
17	292.37	3.89845	2.17653	3.62511	2.22614	1.07540
18	273.36	4.27220	2.56598	3.96698	2.63127	1.07694
19	321.59	4.26472	2.50794	3.95517	2.56228	1.07826
20	323.59	5.20240	2.94005	4.80324	3.02539	1.08310
21	338.91	6.94670	3.42281	6.51776	3.53424	1.06581
22	327.70	4.09390	2.44558	3.83874	2.51166	1.06647
23	341.18	4.51507	2.64906	4.19237	2.72550	1.07697
24	341.70	5.01253	3.18814	4.63450	3.28730	1.08157
25	321.93	3.60636	2.02844	3.38414	2.07112	1.06566
26	328.94	4.54394	2.53666	4.28190	2.60484	**1.06120**
27	330.21	4.53990	2.52505	4.22835	2.59882	1.07368

Note: Bold values indicate best performance. All metrics are overall means and standard deviations computed across all three variables and 15 pretraining datasets. Runtime is total time for all 15 datasets. See table A.1 for architecture details (Layers-Channels-Dropout mapping). Red colour represents the chosen model.

TABLE D.4. TCN PRETRAINING RESULTS: MODEL SELECTION (OVERALL METRICS)

Config	Coverage 95% (mean)	Coverage 95% (std)	Width 95% (mean)	Width 95% (std)
1	1.00000	0.00000	42.9604	6.2543
2	1.00000	0.00000	64.1292	6.5406
3	1.00000	0.00000	80.9096	8.3061
4	1.00000	0.00000	34.7421	3.1245
5	0.99996	0.00014	49.2540	4.7355
6	1.00000	0.00000	63.1799	4.7742
7	1.00000	0.00000	**27.0921**	2.7286
8	1.00000	0.00000	38.8346	4.5054
9	1.00000	0.00000	45.9797	3.3100
10	1.00000	0.00000	44.8247	6.8706
11	1.00000	0.00000	64.9578	8.2162
12	1.00000	0.00000	79.3467	10.7286
13	1.00000	0.00000	33.8447	2.7722
14	0.99993	0.00028	49.6977	5.9046
15	1.00000	0.00000	59.9869	6.0712
16	0.99996	0.00014	27.2447	2.6032
17	1.00000	0.00000	39.0371	3.7770
18	1.00000	0.00000	47.4447	4.0203
19	1.00000	0.00000	45.5307	6.2841
20	1.00000	0.00000	65.5738	7.4218
21	1.00000	0.00000	77.9049	6.2859
22	0.99916	0.00324	33.8037	2.8811
23	1.00000	0.00000	49.3127	4.1473
24	0.99996	0.00014	61.4946	5.8941
25	0.99975	0.00089	27.5689	**2.4893**
26	0.99973	0.00103	37.4919	3.3539
27	1.00000	0.00000	47.9945	4.7664

Note: Bold values indicate best performance. Coverage and width are computed for 95% prediction intervals. Mean and standard deviation are calculated across all variables and 15 pretraining datasets (overall metrics). Red colour represents the chosen model.

TABLE D.5. TCN PRETRAINING RESULTS: UNCERTAINTY QUANTIFICATION (OVERALL METRICS)

Config	RMSE (mean)			MAE (mean)		
	Var 1	Var 2	Var 3	Var 1	Var 2	Var 3
1	2.98499	4.41636	4.89224	2.66576	4.11238	4.64301
2	4.71032	6.68921	6.01615	4.29087	6.38022	5.69115
3	4.45533	5.73093	7.35579	3.93136	5.27562	6.92876
4	3.04288	3.56334	4.79812	2.76252	3.29001	4.59821
5	4.55243	4.45549	5.16381	4.26957	4.12426	4.89968
6	3.75573	4.14263	5.45884	3.32936	3.76558	5.07366
7	3.04224	3.53893	**2.96959**	2.81752	3.31669	**2.78730**
8	3.08603	3.26365	4.56610	2.78396	2.96565	4.35549
9	3.59670	4.10048	4.35155	3.25426	3.80374	4.07156
10	4.44900	3.82570	5.07236	4.13288	3.53137	4.80294
11	3.57060	4.48371	5.75405	3.16840	4.02286	5.38639
12	4.60163	6.24522	7.65095	4.09702	5.80270	7.27716
13	4.05823	3.96570	4.16987	3.80783	3.70950	3.95437
14	3.85327	3.26981	5.74203	3.53029	2.91097	5.46846
15	3.53487	5.08315	6.15077	3.14019	4.73432	5.83749
16	**2.82196**	**2.95768**	3.00854	**2.59747**	**2.69465**	2.81182
17	3.60246	3.98853	4.10437	3.29481	3.72296	3.85758
18	3.98895	3.35277	5.47489	3.63838	3.03333	5.22924
19	3.27817	4.35034	5.16564	2.92720	4.02815	4.91016
20	4.10486	4.73401	6.76832	3.65477	4.32657	6.42839
21	5.17572	7.43236	8.23204	4.65139	7.04645	7.85543
22	4.35970	3.67720	4.24481	4.08448	3.40537	4.02637
23	4.67178	4.02674	4.84670	4.34041	3.67813	4.55859
24	4.62850	4.06395	6.34515	4.23478	3.66167	6.00705
25	4.24143	3.32895	3.24870	4.02555	3.09503	3.03185
26	3.33102	4.72602	5.57476	3.00338	4.45976	5.38254
27	3.64369	4.15533	5.82069	3.29045	3.83425	5.56034

Note: Bold values indicate best performance. Metrics are means across 15 pretraining datasets. No standard deviations shown for per-variable metrics. Red colour represents the chosen model.

TABLE D.6. TCN PRETRAINING RESULTS: PER-VARIABLE PERFORMANCE (MEANS)

APPENDIX E. SIMULATION STUDY DESIGN TABLE

Component	Specification	Pretraining	Finals
Dataset Configuration			
Total Datasets	–	15	300
Datasets per Regime	$\lambda \in \{0.25, 0.50, 0.75\}$	5 each	100 each
Observations per Dataset	N_{obs}	6,250	6,250
Number of Assets	N_{vars}	3	3
Data Split			
Training Set	80%	5,000 obs	5,000 obs
Testing Set	20%	1,250 obs	1,250 obs
Reproducibility			
Random Seed	–	42	666
Purpose			
Objective	–	Hyperparameter	Model
	–	Selection	Comparison

Total observations: 1,968,750 across 315 datasets
All code is available in the following repository
Gitlab Username:@zfontaine
Gitlab Link: https://gitlab.com/zfontaine/uc3m-master-thesis-bvar-vs-tcn

TABLE E.1. SIMULATION STUDY DESIGN SUMMARY

APPENDIX F. BVAR IMPLEMENTATION PIPELINE AND CODING SPECIFICATIONS

BVAR Implementation Pipeline

1. **Input:** price data (6250 times 3)
2. **Stationarity check(ADF/KPSS):** If non-stationary, then difference and track via *data_tag*
3. **Estimate AR(1) parameters:** use for Minnesota prior means
4. **Build prior covariance matrix** (λ, τ, δ)
5. **MCMC sampling:** NUTS with 4 chains and 2000 draws per chain
6. **Generate forecast distribution:** 1250 steps
7. **Integrate forecasts if differencing was used:** forecasts are returned to prices
8. **Output:** predictions + quantiles

Key Software, Programming Languages and Libraries

- *Bash:* used for writing piping and orchestration scripts utilising native Linux parallelisation
- *Python:*NumPy, Pandas, PyMC, adfuller from statsmodels, multivariate_normal from scipy.stats
- *Fixed Seed:* 1964

Fig. F.1. BVAR Implementation Pipeline and Coding Specifications

APPENDIX G. TCN+MC DROPOUT IMPLEMENTATION AND CODING SPECIFICATIONS

TCN Implementation Pipeline

1. **Input:** price data (6250×3)
2. **Sequence preparation:** sliding window with lookback=50
3. **Train/test split:** 5000 train / 1250 test (temporal, no shuffle)
4. **Model initialisation:** TCN with specified layers, channels, dropout
5. **Training:** Adam optimiser + MSE loss + early stopping
6. **MC Dropout inference:** 100 forward passes with dropout active
7. **Compute quantiles:** from MC sample distribution
8. **Output:** predictions + quantiles

Key Software, Programming Languages and Libraries

- *Bash:* piping and orchestration scripts with GNU Parallel for Cartesian product grid search
- *Python:* PyTorch, NumPy, Pandas
- *Hardware:* CUDA GPU acceleration
- *Fixed seed:* 1964

Fig. G.1. TCN Implementation Pipeline and Coding Specifications

APPENDIX H. METRICS EQUATIONS AND HARDWARE CONFIGURATION TABLE

H.1. Root Mean Square Error(RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{H.1}$$

where y_i denotes the actual value, $\hat{y}_i$ the predicted value, and n the number of observations.

H.2. Mean Absolute Error(MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{H.2}$$

H.3. Prediction Interval Coverage Rate (80%, 90%, 95%)

$$\text{PICR}_\alpha = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\left(L_i^{(\alpha)} \le y_i \le U_i^{(\alpha)}\right) \tag{H.3}$$

where $L_i^{(\alpha)}$ and $U_i^{(\alpha)}$ denote the lower and upper bounds of the $(1-\alpha)\times 100\%$ prediction interval, respectively, and $\mathbf{1}(\cdot)$ is the indicator function.

H.4. Average Prediction Interval Width (80%, 90%, 95%)

$$\text{APIW}_\alpha = \frac{1}{n}\sum_{i=1}^{n}\left(U_i^{(\alpha)} - L_i^{(\alpha)}\right) \tag{H.4}$$

H.5. Hardware Configuration for Model Training

H.6. Deviance Information Criterion (DIC)

$$\text{DIC} = \bar{D}(\theta) + p_D = D(\bar{\theta}) + 2p_D \tag{H.5}$$

where $\bar{D}(\theta) = \mathbb{E}_{\theta|y}[D(\theta)]$ is the posterior mean of the deviance, $D(\theta) = -2\log p(y|\theta)$ is the deviance, $\bar{\theta}$ is the posterior mean of the parameters, and $p_D = \bar{D}(\theta) - D(\bar{\theta})$ represents the effective number of parameters.

Component	Specification
GPU	NVIDIA GTX 1660 Ti Mobile
CUDA Cores	1,536
VRAM	6 GB GDDR6
CPU	Intel i7-9750H (6-core, 12-thread)
Base Clock	2.6 GHz
RAM	32 GB

TABLE H.1. HARDWARE CONFIGURATION FOR MODEL TRAINING

H.7. Mean Across Datasets

$$\overline{m}_i = \frac{1}{N} \sum_{d=1}^{N} m_{i,d} \tag{H.6}$$

where $N \in \{N_{pre}, N_{final}\}$ depending on the phase.

H.8. Standard Deviation Across Datasets

$$\sigma_{m_i} = \sqrt{\frac{1}{N-1} \sum_{d=1}^{N} (m_{i,d} - \overline{m}_i)^2} \tag{H.7}$$

Note that the sample standard deviation ($N - 1$ in denominator) is used to provide an unbiased estimate of variability.

H.9. Aggregated (Overall) Mean

$$\overline{m}_{overall} = \frac{1}{k} \sum_{i=1}^{k} \overline{m}_i \tag{H.8}$$

where $k = 3$ is the number of variables.

H.10. Aggregated (Overall) Standard Deviation

$$\sigma_{m,overall} = \frac{1}{k} \sum_{i=1}^{k} \sigma_{m_i} \tag{H.9}$$

APPENDIX I. DERIVATION OF BAYES THEOREM

Since Bayes Theorem is the cornerstone of Bayesian statistics, this appendix is included for completeness. While its contents are basic, it is useful to include for non-Bayesian readers.

The beginning of the derivation is conditional probability and its rearrangement for finding an equation for an intersection:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \tag{I.1}$$

$$P(A|B) \cdot P(B) = P(A \cap B) \tag{I.2}$$

By the commutative property of intersection and by the new equation for $P(B \cap A)$, the Bayes Theorem can be derived as follows;

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B \cap A)}{P(B)} = \frac{P(B|A) \cdot P(A)}{P(B)} \tag{I.3}$$

Therefore, Bayes' Theorem in its final form is:

$$\boxed{P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}} \tag{I.4}$$

The above equation can be conceptually understood as following:

Conceptually:

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}} \tag{I.5}$$

Prior Prior is often described as belief or an assumption, about the probability of event A occurring without any new data. It represents prior knowledge of any kind, regardless of accuracy. The important concept of the prior is that it is understanding before the statistical experiment takes place.

Evidence Evidence, or also referred to as marginal likelihood, is the total probability of observing the event B. It also functions as a scaling factor that ensures the posterior probability is between 0 and 1.

Likelihood Likelihood is the probability of event B occurring, given that event A occurred. It measures how well the hypothesis about event A occurring explains the data observed for B. In practice, likelihood often takes form of a statistical model that is being used within Bayesian framework.

Posterior Posterior is the final result of the Bayes theorem that shows the updated probability of A after B occurring. While in essence it is conceptually equivalent to conditional probability, the key here is that there is no observed evidence about A. This allows for uncertainty quantification where the understanding of A is defined via prior knowledge and its updating through data, rather than using frequentist asymptotic properties.

APPENDIX J. PRIOR DISTRIBUTIONS FOR COVARIANCE MATRICES

J.1. Inverse-Wishart Distribution

The Inverse-Wishart is the conjugate prior for the covariance matrix of a multivariate normal. If $\mathbf{\Sigma} \sim \mathscr{W}^{-1}(\mathbf{S}_0, \nu_0)$, its density is

$$p(\mathbf{\Sigma}|\mathbf{S}_0, \nu_0) \propto |\mathbf{\Sigma}|^{-(\nu_0+k+1)/2} \exp\left(-\frac{1}{2}\operatorname{tr}(\mathbf{S}_0\mathbf{\Sigma}^{-1})\right) \tag{J.1}$$

where $\mathbf{S}_0$ is a $k \times k$ positive definite scale matrix and $\nu_0 > k-1$ is the degrees of freedom.

J.2. LKJ Distribution

The LKJ distribution [32] is used as a prior for correlation matrices. For a $k \times k$ correlation matrix $\mathbf{R}$, the probability density function is:

$$p(\mathbf{R}|\eta) = C * |\mathbf{R}|^{\eta-1} \tag{J.2}$$

where C is a normalising constant that includes η and k. The variable η can be used to set desired correlation behaviour by setting: $\eta = 1$ to give a uniform distribution over valid correlation matrices; $\eta > 1$ to shrink correlations toward zero and; $\eta < 1$ to favour stronger correlations.

Unlike the Inverse-Wishart, using LKJ distribution as a prior allows to separate correlations from scales. The covariance matrix is recovered via

$$\mathbf{\Sigma} = \operatorname{diag}(\boldsymbol{\sigma}) \cdot \mathbf{R} \cdot \operatorname{diag}(\boldsymbol{\sigma}) \tag{J.3}$$

where each σ_i gets its own prior which allows for separating prior beliefs about correlations and standard deviations of individual variables.

APPENDIX K. MCMC ALGORITHMS SHOWCASE

Metropolis Algorithm

Goal: Given target distribution $\pi(x)$, generate samples from said given distribution

1. Choose starting value x_0
2. Propose a value x^* from symmetric proposal distribution $q(x^*|x_t)$
3. Calculate acceptance ratio $\alpha = \frac{\pi(x^*)}{\pi(x_t)}$
4. Sample random number u from $U(0, 1)$
   Depending on the acceptance ratio:
5. Accept if u is less than $min(1, \alpha)$
   $x_{t+1} = x^*$
6. Reject otherwise $x_{t+1} = x_t$
7. Repeat from **step 2** until convergence

Metropolis-Hastings Algorithm

Goal: Given target distribution $\pi(x)$, generate samples from said distribution using potentially asymmetric proposals

1. Choose starting value x_0
2. Propose a value x^* from proposal distribution $q(x^*|x_t)$ (can be asymmetric)
3. Calculate acceptance ratio $\alpha = \frac{\pi(x^*)q(x_t|x^*)}{\pi(x_t)q(x^*|x_t)}$
4. Sample random number u from $U(0, 1)$
   Depending on the acceptance ratio:
5. Accept if u is less than $\min(1, \alpha)$
   $x_{t+1} = x^*$
6. Reject otherwise
   $x_{t+1} = x_t$
7. Repeat from **step 2** until convergence

Gibbs Sampling Algorithm

Goal: Given target distribution $\pi(x)$ where $x = (x_1, x_2, ..., x_d)$, generate samples from said distribution using conditional distributions

1. Choose starting values $x_0 = (x_0^{(1)}, x_0^{(2)}, ..., x_0^{(d)})$
2. For each component $i = 1, 2, ..., d$:
    - Sample $x_{t+1}^{(i)}$ from conditional distribution $\pi(x^{(i)}|x_{t+1}^{(1)}, ..., x_{t+1}^{(i-1)}, x_t^{(i+1)}, ..., x_t^{(d)})$
3. Update state:
    - $x_{t+1} = (x_{t+1}^{(1)}, x_{t+1}^{(2)}, ..., x_{t+1}^{(d)})$
4. Repeat from **step 2** until convergence

Note: Each conditional update is always accepted (acceptance probability = 1), making this a special case of Metropolis-Hastings where the proposal for component i is its full conditional distribution

APPENDIX L. INFERENCE SHOWCASE: DIAGNOSTIC PLOTS

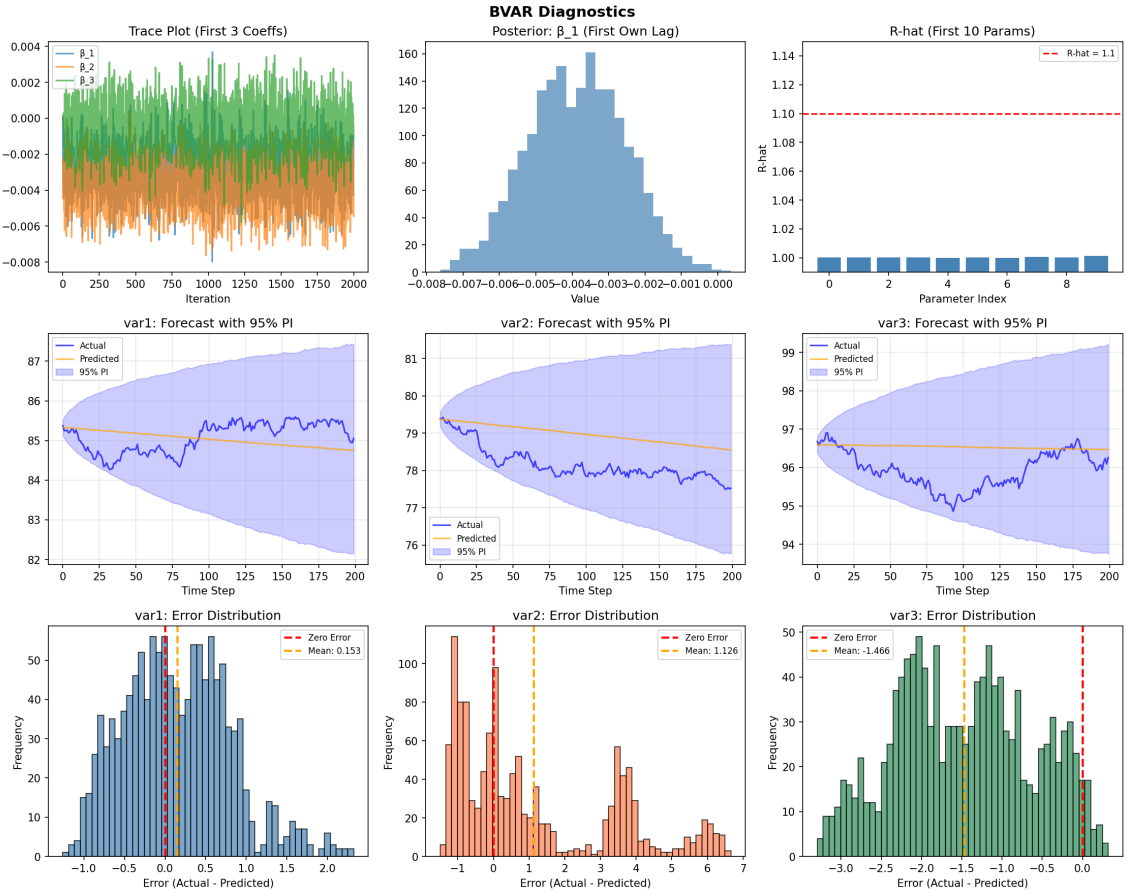


Fig. L.1. BVAR(Lag 5) - Diagnostic Plots

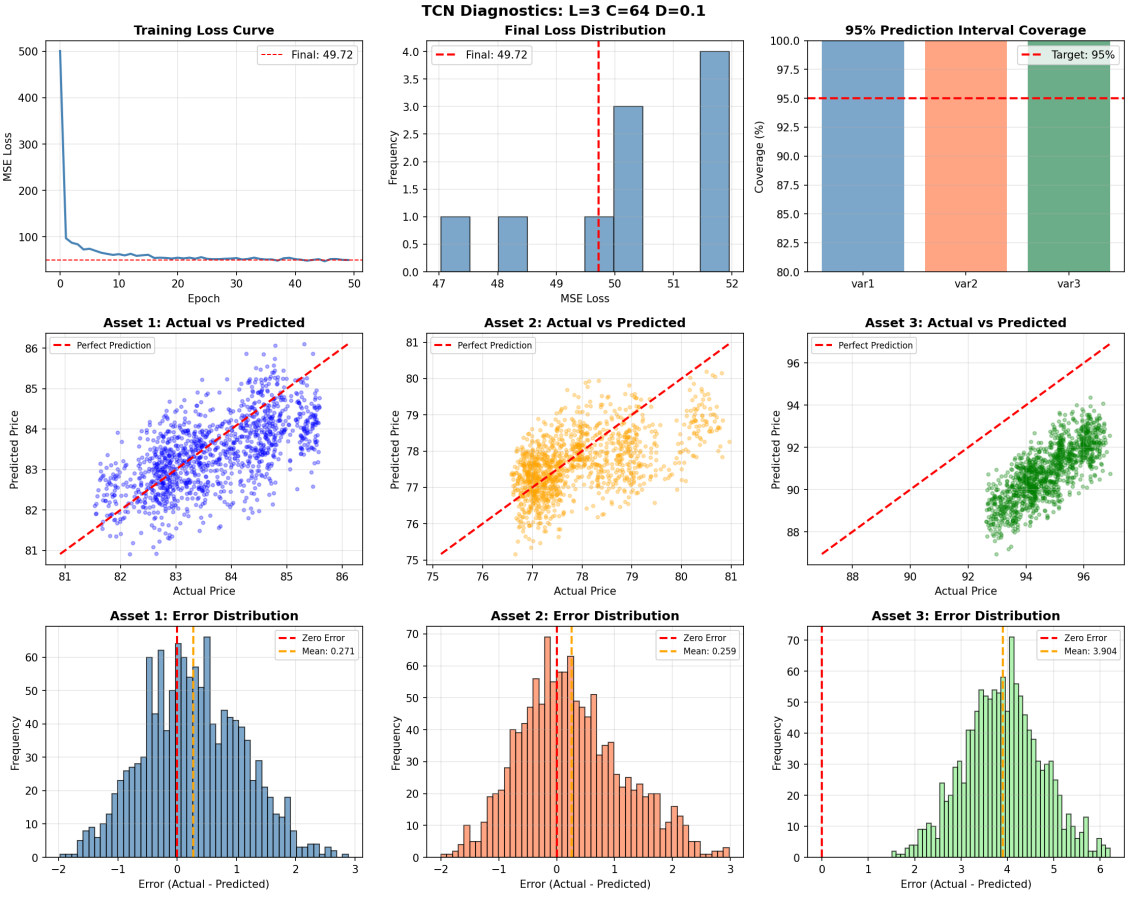


Fig. L.2. TCN+MC Dropout (Model 16) - Diagnostic Plots

APPENDIX M. CODE REPOSITORY AND AUTHOR ALIAS DECLARATION

The complete code repository associated with this thesis has been submitted and is publicly available on GitLab. All code, data generation scripts, and analysis tools developed for this research are included in the repository to ensure full reproducibility of the results presented herein.

The repository is published under the established professional and academic pseudonym "Zeno Fontaine", which serves as an official alias for the author of this work. This pseudonym has been used consistently across academic platforms and publications.

Repository Details:

Platform:	GitLab
Username:	@zfontaine
Repository URL:	`https://gitlab.com/zfontaine/uc3m-master-thesis-bvar-vs-tcn`
Thesis Submission Commit SHA:	485e673f7d15316ef15052e3e62bac3918502a8e

All code is licensed under GNU GPL v3 and is available for academic and research purposes.